

А.А. ТЮГАШЕВ

НЕЙРОННЫЕ СЕТИ

Учебное пособие

Оглавление

ВВЕДЕНИЕ	3
1. ОСНОВЫ МАШИННОГО ОБУЧЕНИЯ	5
ИСТОРИЯ МАШИННОГО ОБУЧЕНИЯ	8
КЛАССИЧЕСКИЕ ЗАДАЧИ МАШИННОГО ОБУЧЕНИЯ	10
ЯЗЫК ПРОГРАММИРОВАНИЯ ПИТОН В ИИ	15
2. ВВЕДЕНИЕ В ИСКУССТВЕННЫЕ НЕЙРОННЫЕ СЕТИ	25
ИСТОРИЯ ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЕЙ	25
ОСНОВНЫЕ КЛАССЫ ЗАДАЧ, РЕШАЕМЫЕ ИНС	27
БИОЛОГИЧЕСКИЙ НЕЙРОН	29
СТРУКТУРА ФОРМАЛЬНОГО НЕЙРОНА	30
АРХИТЕКТУРЫ НЕЙРОННЫХ СЕТЕЙ	34
ТЕОРЕМЫ КОЛМОГОРОВА – АРНОЛЬДА И ХЕХТ-НИЛЬСЕНА	37
ОБУЧЕНИЕ НЕЙРОННОЙ СЕТИ	39
<i>Обучение нейросети с учителем</i>	<i>40</i>
<i>Обучение без учителя</i>	<i>44</i>
ПРИМЕР СОЗДАНИЯ НЕЙРОСЕТИ НА ЯЗЫКЕ ПИТОН	46
<i>Контрольные вопросы и упражнения главы</i>	<i>56</i>
3. ПРИМЕНЕНИЕ СОВРЕМЕННЫХ НЕЙРОСЕТЕЙ	57
СВЕРТОЧНЫЕ СЕТИ	57
РЕКУРРЕНТНЫЕ НЕЙРОННЫЕ СЕТИ	67
АРХИТЕКТУРА «ТРАНСФОРМЕР»	73
УСПЕХИ ОБУЧЕНИЯ С ПОДКРЕПЛЕНИЕМ	76
ТЕНДЕНЦИИ РАЗВИТИЯ БОЛЬШИХ ЯЗЫКОВЫХ МОДЕЛЕЙ	85
ИСПОЛЬЗОВАНИЕ БИБЛИОТЕК ЯЗЫКА ПИТОН ДЛЯ СОЗДАНИЯ ИНС	98
<i>Применение библиотеки Keras</i>	<i>98</i>
<i>Использование библиотеки PyTorch</i>	<i>109</i>
<i>Распознавание медицинских изображений</i>	<i>115</i>
<i>PyTorch для предсказания стоимости квартир</i>	<i>121</i>
СОВРЕМЕННЫЕ ДОСТИЖЕНИЯ КОМПЬЮТЕРНОГО ЗРЕНИЯ	130
НЕЙРОСЕТИ ДЛЯ АНАЛИЗА ЕСТЕСТВЕННОГО ЯЗЫКА	134
<i>Одномерные сверточные нейронные сети</i>	<i>141</i>
<i>Пример определения тональности текста</i>	<i>143</i>
АУТОМЛ – ИИ СОВЕРШЕНСТВУЕТ СЕБЯ	145
НЕЙРОСТРУДНИКИ	154
ВВЕДЕНИЕ В ПРОМПТ-ИНЖИНИРИНГ И КОНТЕКСТ-ИНЖИНИРИНГ	160
<i>Контрольные вопросы и упражнения главы</i>	<i>165</i>
11. ФИЛОСОФСКИЕ, ЭТИЧЕСКИЕ И СОЦИАЛЬНЫЕ ПРОБЛЕМЫ НЕЙРОСЕТЕЙ	166
НЕРЕШЕННЫЕ ВОПРОСЫ ИНС	166
НЕСКОЛЬКО СЛОВ О КИБОРГАХ	171
<i>Контрольные вопросы главы</i>	<i>173</i>
ЗАКЛЮЧЕНИЕ	174
БИБЛИОГРАФИЧЕСКИЙ СПИСОК	175

ВВЕДЕНИЕ

Учебное пособие предназначено для изучения основ искусственных нейронных сетей, являющихся базой для большинства современных систем искусственного интеллекта (ИИ).

В качестве языка программирования в примерах используется Питон (Python), обладающий богатыми библиотеками в данной предметной области — Sklearn, Keras, TensorFlow, и пр.

Приводятся вопросы и задания для обучающихся.

Написание данного пособия относится к периоду революции искусственного интеллекта, прежде всего связанных с успехами так называемых больших языковых моделей, лежащих в основе интеллектуальных чат-ботов, а также генеративных нейросетей, создающих изображения и видео. Искусственный интеллект пишет программы для ЭВМ, сочиняет музыку, стихи, «на лету» переводит видео с одного языка на другой, анализирует структуру белка, и прочая, и прочая. В современном мире искусственный интеллект становится все более значимым, его влияние заметно в самых разных аспектах.

Нейросети сейчас играют ключевую роль в создании персонализированных услуг и продуктов, причем речь идет не просто о рекомендации купить тот или иной товар на основе истории покупок, а широкий спектр приложений - от персональных рекомендаций в медицине до индивидуально составленных учебных планов.

Искусственный интеллект значительно ускоряет процесс исследований и разработок, от фармацевтики до аэрокосмической индустрии. Машина способна обрабатывать огромные объемы данных быстрее и точнее, чем это мог бы сделать человек. Это может повлечь за собой революцию в науке.

В области безопасности ИИ позволяет улучшать системы видеонаблюдения и мониторинга, что помогает быстрее реагировать на возможные инциденты и предотвращать преступления (с другой стороны — есть опасность чрезмерного контроля над личностью).

Важность ИИ сегодня трудно переоценить. Он стал неотъемлемой частью современного технологического ландшафта, важной для промышленности, сельского хозяйства, науки, культуры, военной сферы. Понимание основных принципов и возможностей искусственного интеллекта может открыть перед каждым новые горизонты и возможности для развития и профессионального роста.

Известные профессионалы в области информатики, инноваторы, такие как Билл Гейтс, Илон Маск, сравнивают происходящее по степени влияния на будущее человечества с появлением компьютеров. А некоторые специалисты – даже с открытием огня и колеса. По некоторым оценкам, ИИ может вытеснить 73% людей, занятых в информационных технологиях.

Развитие искусственного интеллекта (ИИ) стало вопросом государственной важности, залогом конкурентоспособности страны. На высшем уровне – Указом Президента РФ № 490 утверждена Национальная стратегия развития искусственного интеллекта до 2030 года [1]. Надо заметить, что и на сайте Белого дома США есть раздел по ИИ, и во Франции, Японии и других странах существуют государственные программы развития искусственного интеллекта – «ИИ для французов», «ИИ для американской нации» и т.п.

С одной стороны, все это внушает веру в будущее развитие техники. С другой – вызывает опасения. Если в принципе возможно появление искусственного разума, не уступающего человеческому, или даже превосходящего его по своему потенциалу, что нам принесет появление мыслящих машин? Не станет ли это серьезной угрозой для человечества?

Читателю предлагается сделать свой собственный вывод по итогам ознакомления с приводимыми материалами. Автор надеется, что настоящее пособие станет неплохим введением в мир нейросетей.

1. ОСНОВЫ МАШИННОГО ОБУЧЕНИЯ

В настоящее время, когда произносят слова «искусственный интеллект», в большинстве случаев подразумеваются системы машинного обучения (англ. *machine learning*). Что же это за системы? Как они соотносятся с другими подходами, применяемыми в методах ИИ?

К характерным и одним из базовых свойств интеллектуальных систем относится адаптивность, приспособляемость, способность действовать гибко в зависимости от складывающейся ситуации. В чем источники этой гибкости? Какими различными способами можно добиться вариативного поведения ЭВМ?

Во-первых, варианты поведения могут быть непосредственно запрограммированы. Одной из основных конструкций языков программирования ЭВМ являются операторы «ЕСЛИ-ТО-ИНАЧЕ» и ВЫБОР (из нескольких альтернатив – case, switch, и т.п.). Наличие большого числа условных конструкций в программе может позволить добиться значительного числа ветвей в алгоритме и тем самым сделать ее поведение весьма гибким.

Однако реализация вариантов непосредственно в программе – далеко не лучший вариант реализации интеллектуальной системы. При необходимости настройки на другие варианты потребуются менять исходные файлы программ, заново осуществлять трансляцию, отладку новой получившейся версии программного обеспечения, внедрение и т.п.

Кроме того, еще одним важнейшим качеством интеллектуальных систем должна быть способность к обучению. Ясно, что реализовать ее в случае использования условных операторов весьма непросто.

Значительно более перспективным представляется построение программы как комбинации некой «машины вывода» и базы знаний (БЗ), как в экспертных системах. База знаний при этом является пополняемым и уточняемым хранилищем фактов, правил и т.п. При данном подходе появляется возможность сначала построить некую «оболочку», или «скелет» интеллектуальной системы, а затем, наполняя базу знаний, получать прикладные программы, пригодные для применения в различных сферах.

Обратим внимание на ключевую черту – в алгоритме, реализованном в данном случае в программе, непосредственно нет инструкций для решения конкретных задач. Иными словами, изначально при создании подобной интеллектуальной системы мы знаем лишь общий подход, но *не знаем в деталях*, как именно она в дальнейшем будет приходить к своим выводам.

Системы подобного рода можно снабжать специальными модулями для приобретения знаний. Приобретение знаний при этом может происходить под контролем человека – как отдельно взятого эксперта в некоторой области, который вводит в БЗ, например, правила продукционной модели вида «ЕСЛИ P_1 и P_2 и... P_n ТО $Z_1, Z_2... Z_m$ », где P_i – посылки, а Z_j – следствия, которые затем, в свою очередь, могут использоваться в качестве посылок в других правилах, так и при помощи представителя новой профессии эпохи ИИ – *когнитолога*, или *инженера по знаниям*, умеющего структурировать и формализовывать знания, используемые, но зачастую не осознаваемые в полной мере самим экспертом, для ввода их в машину в виде, пригодном для использования. Процесс пополнения базы знаний вышеописанных систем, безусловно, является разновидностью машинного обучения – так называемым *дедуктивным* обучением. Дедуктивное обучение предполагает формализацию знаний экспертов и их перенос в компьютер в виде структурированной базы знаний, в коей легко можно выделить и понять отдельные правила.

Другой способ – извлечение знаний (data mining, knowledge discovery, knowledge mining) автоматически путем за счет выявления зависимостей в больших объемах данных.

Что такое data mining, или, по-русски, **интеллектуальный анализ данных** (ИАД)? Читателю будет интересна следующая история [7]. Говорят, что по субботам на Западе в супермаркетах был отмечен феномен: всплеску подвергаются продажи двух товаров – памперсов и пива, присутствующих при этом в одном чеке, но почему - никто объяснить не мог. Оказалось, что суббота в Европе – в основном "папин день". Когда жены окончательно устают «бороться» с отпрысками на протяжении недели, они говорят: «муж, сегодня ты идешь за памперсами». Он идет. Но для него этот поход – важная

социальная нагрузка, он чувствует себя в некотором смысле «героем». Он «мужественно» берет упаковку памперсов и немедленно заключает, что заслуживает вознаграждения, для чего приобретает пиво. Когда это поняли, по субботам к стеллажу с подгузниками начали подставлять ящики пива. Продажи «набора» сильно возросли – и принесли магазинам повышенную прибыль.

Особую роль интеллектуальный анализ данных стал играть сейчас, когда в самых разных сферах уже накоплены большие объемы информации в электронном виде. Это – так называемые «большие данные», или Big Data, основными чертами которых считают три «V» – Volume, или по-русски размер, Velocity, по-русски скорость, имеется в виду как скорость накопления, так и необходимое для обработки быстрое действие вычислительных средств, и Variety – разнообразие.

Результатом работы алгоритмов интеллектуального анализа данных являются опять же правила, пригодные для использования машиной вывода.

Однако все же в подавляющем числе случаев, когда упоминают машинное обучение сегодня, имеют в виду индуктивное обучение, основанное на выявлении общих закономерностей по частным эмпирическим данным. Поговорим об этом подробнее.

Машинное обучение находится на стыке математической статистики, методов оптимизации и иных классических математических дисциплин, но имеет и собственную специфику, связанную с остро стоящими проблемами вычислительной эффективности и переобучения [31].

Машинное обучение – не чисто математическая, но и практическая, инженерная дисциплина [32]. Теоретические исследования, как правило, не приводят немедленно к методам и алгоритмам, применимым на практике. Чтобы заставить их хорошо работать, приходится изобретать и применять дополнительные *эвристики*, компенсирующие несоответствие сделанных в теории предположений условиям реальных задач. Под эвристикой здесь понимается не имеющее строгого теоретического обоснования, но полезное при решении задач соображение «здравого смысла», например, отражающее специфику данной области. Поскольку

решение большинства реальных задач методом прямого перебора невозможно, а эвристики позволяют сократить объем дерева поиска решения, они играют весьма важную роль в ИИ. Практически ни одно исследование в машинном обучении не обходится без эксперимента на модельных или реальных данных, подтверждающего практическую работоспособность метода.

Общая постановка задачи *обучения на основе прецедентов* сводится к нижеследующему. Дано конечное множество прецедентов (объектов, ситуаций), по каждому из которых собраны (измерены) некоторые данные. Данные о прецеденте называют также его описанием. Совокупность всех имеющихся описаний прецедентов называется обучающей выборкой. Требуется по этим частным данным выявить общие зависимости, закономерности, взаимосвязи, присущие не только этой конкретной выборке, но вообще всем прецедентам, в том числе тем, которые ещё не наблюдались.

Говорят еще о восстановлении *зависимостей* по эмпирическим данным. Иными словами, на входе имеется множество объектов (ситуаций), существует также множество возможных значений на выходе (откликов, реакций). Между входом и выходом существует некоторая зависимость, но она неизвестна.

При этом известна конечная совокупность прецедентов – «правильных» пар «вход, ответ», называемая обучающей выборкой. На основе этих данных нужно построить процедуру, способную для любого объекта на входе выдать соответствующий ответ. Для оценки точности ответов обычно вводится мера – так называемый функционал качества.

Замечание. Нетрудно заметить, что данная постановка является обобщением классических математических задач аппроксимации и интерполяции функций. Однако в классических задачах аппроксимации на входе имеются действительные числа или векторы. В практически значимых прикладных задачах данные об объектах могут быть неполными, неточными, нечисловыми, разнородными. Эти особенности приводят к значительному разнообразию методов машинного обучения.

История машинного обучения

Термин «машинное обучение» в 1959 году ввел в научный обиход Артур Сэмюэль (Arthur Samuel), исследователь в области искусственного интеллекта и изобретатель первой самообучающейся компьютерной программы игры в шашки (рис. 1.1). Он определил его как процесс, в результате которого компьютеры способны *показать поведение, которое в них не было явно запрограммировано*.



Рис. 1.1. Артур Сэмюэль – пионер машинного обучения

В 1952 году Сэмюэль создает первую программу, играющую в шашки, для ЭВМ IBM 701. Сэмюэль выбрал шашки, потому что правила игры относительно просты, но в ней значима развитая стратегия. Основой программы являлось дерево поиска игровых позиций, достижимых из текущего состояния.

В 1955 году он добавляет в программу способность к обучению. Сэмюэль разрабатывал различные методы, которые должны были сделать его программу лучше. Например, он назвал зубрежкой (*rote learning*) процесс, когда программа запоминала каждую позицию игры, в которую уже играла, вплоть до конечного результата игры. Программа Checkers-playing, таким образом, стала одной из первых

практических демонстраций базовых понятий искусственного интеллекта. Более поздние программы Сэмюэля переоценивали веса оценочной функции на основе игр профессионалов. Также он заставлял программу играть *против себя самой* и таким образом самообучаться. В итоге программа Сэмюэля достигла достаточно высокого любительского уровня. Его метод обучения через игру продолжал развиваться как для игры в шашки (где к 2007 году компьютер уже мог исследовать все положения на доске), так и для других игр, таких как шахматы и го.

Классические задачи машинного обучения

В рамках машинного обучения можно выделить целый ряд традиционно решаемых задач – как с точки зрения особенностей постановки задачи и применяемых методов, так и практических потребностей.

Сначала рассмотрим подвиды так называемого **обучения с учителем** (англ. *supervised learning*), представляющего собой наиболее распространённый случай в машинном обучении. Каждый прецедент представляет собой пару {объект, ответ} с известным «правильным», или «желаемым», ответом. Требуется найти зависимость ответов от описаний объектов и построить систему, принимающую на вход описание объекта и выдающую на выход ответ. Функционал качества обычно определяется как средняя ошибка ответов, выданных алгоритмом, по всем возможным входным объектам. Перечислим основные варианты решаемых методами машинного обучения задач при обучении с учителем [31, 32].

Задача *классификации* (classification) отличается тем, что множество допустимых ответов конечно. Их называют метками классов (class label). Иными словами, получая на входе некоторый набор признаков, на выходе нужно получить номер или наименование класса, к которому относится предъявленный объект (например, горилла, банан, автобус или человек).

Задача *регрессии* (regression) отличается тем, что допустимым ответом является произвольное действительное число или числовой вектор. Иными словами, имеется набор признаков, описывающих

входной объект и представленных действительными числами, и на выходе — число.

Задача *прогнозирования* (forecasting) отличается тем, что объектами являются отрезки временных рядов, и нужен прогноз на будущее. Например, известен курс акций некоторого предприятия за некий предшествующий период, и нам интересно, каким будет курс завтра. Для решения задач прогнозирования часто удаётся приспособить методы регрессии или классификации.

Бывает, на входе имеется последовательность данных, и на выходе необходимо получить последовательность — задачи, называемые условно *seq2seq*. К данному классу относится *машинный перевод* — здесь одна цепочка слов преобразуется в другую.

Если на входе и выходе изображения, получается задача преобразования изображений. Другой пример — задача распознавания речи, при которой на вход модели подаётся последовательность звуковых амплитуд, а на выходе получается текстовая расшифровка речи [5,6].

Может быть и так, что на входе — текст, а на выходе изображение, это задача *генерации* изображения по описанию, где сейчас достигнуты весьма яркие успехи, или на входе изображение, а на выходе — текст, например, описание, что изображено на рисунке. Генеративные модели сейчас достигли большого успеха в создании рисунков, музыки, даже видео (например, Sora).

В случае комбинации (смеси) разных видов данных (включая сейчас сюда и звуки, и видео) на входе и на выходе, можно говорить о мультимодальных моделях.

Задачи регрессии и классификации на самом деле можно представить в виде задачи преобразования последовательности в последовательность. В этом случае входная последовательность будет содержать значения входных факторов (или весь набор значений факторов в виде единственного элемента-вектора), а выходная - состоять из одного элемента — метки класса или значения регрессии.

Мы видим некоторую условность выделения категорий задач машинного обучения. Когда мы относим ту или иную задачу к категории *seq2seq*, то обычно хотим тем самым подчеркнуть, что

входные и выходные данные модели могут иметь переменную размерность. Если же, например, на входе нашей модели последовательность переменной длины, а на выходе — метка класса, то такая задача будет скорее отнесена к задачам классификации последовательностей (sequence classification). Примером такой задачи может быть выявление языка, на котором написан некоторый текст переменной длины. Аналогичным образом говорят о регрессии последовательностей (sequence regression), в случаях когда на входе модели — последовательность, а на выходе — некоторая величина, например на входе — текст комментария в социальной сети, а на выходе — предполагаемый возраст его автора [5].

В машинном обучении возможно и так называемое **обучение без учителя** (unsupervised learning). В этом случае «правильные» ответы не задаются, а искать нужно некоторые зависимости между объектами. Перечислим решаемые здесь задачи:

Задача *кластеризации* (clustering) заключается в том, чтобы сгруппировать объекты в так называемые кластеры, или группы «схожих» между собой объектов. Задачу можно назвать классификацией с заранее неизвестным набором различаемых классов. Функционалы качества могут определяться по-разному, например, как отношение средних расстояний между объектами внутри построенных кластеров и между объектами различных кластеров.

Задача *поиска ассоциативных правил* (пример — рассмотренный выше «закон пива и памперсов») (association rules learning). Исходные данные представлены в виде признаковых описаний. Требуется найти наборы признаков, особенно часто (неслучайно часто) встречающихся в описаниях объектов.

Задача *фильтрации выбросов* (outliers detection) — обнаружение в обучающей выборке небольшого числа особых, или нетипичных объектов. В некоторых приложениях их поиск является самоцелью (например, обнаружение мошенничества). В других приложениях эти объекты являются следствием ошибок в данных или неточности модели, то есть шумом, мешающим настраивать модель, и должны быть удалены из выборки.

Задача *сокращения размерности* (dimensionality reduction) заключается в том, чтобы по исходным признакам с помощью

некоторых функций преобразования перейти к меньшему числу новых признаков, не потеряв при этом существенной информации об объектах выборки.

Есть еще два класса задач, решаемых в рамках обучения без учителя — это задачи понижения размерности и поиск ассоциативных правил.

В задаче понижения размерности требуется из исходных признаков получить новый набор признаков, чтобы потерять как можно меньше информации от исходных объектов, но чтобы признаков при этом стало меньше. Это позволяет нам упростить задачи машинного обучения. Если признаков станет меньше, то и найти правильный алгоритм будет проще.

В задачах *поиска ассоциативных правил* требуется найти закономерности между связанными событиями. Примером может служить правило: если произошло событие X, то должно произойти событие Y. Самым простым примером является анализ чеков покупателей в магазинах. Ведь на основе этой информации мы можем сделать вывод, если покупатель купил товар №1, значит, он купит и товар №2.

Ассоциативные правила могут быть сложными. Они могут состоять не из одного правила (если произошло событие X, то произойдет событие Y), а из нескольких последовательных таких правил. Именно поэтому ассоциативные правила могут хорошо использоваться не только в расстановке товаров на полках, но еще и в анализе паттернов поведения клиентов на веб-сайтах и для анализа товаров, покупаемых вместе.

Представим себе свидетеля преступления, который описывает внешность преступника словами: сообщает цвет глаз, рассказывает о причёске, форме носа и глаз, наличии или отсутствии усов, бороды, воспроизводит другие особенности внешности, называет пол преступника, его ориентировочные рост и возраст. Весь этот набор значений признаков куда более компактен, чем фото преступника, составленное из миллионов пикселей. В данном случае мозг свидетеля выполняет роль модели, решающей задачу сокращения размерности входных данных

В классе линейных преобразований наиболее известным примером является метод главных компонент.

Задача *заполнения пропущенных значений* (missing values) – замена недостающих значений в матрице «объекты – признаки» прогнозными значениями.

Частичное обучение (semi-supervised learning) занимает промежуточное положение между обучением с учителем и без учителя. Прецедент представляет собой пару «объект, ответ», но ответы известны только для части прецедентов. Пример прикладной задачи – автоматическая рубрикация большого количества текстов при условии, что некоторые из них уже отнесены к каким-то рубрикам.

Важнейшую роль в современных системах ИИ играет *обучение с подкреплением* (англ. *reinforcement learning*). Роль объектов играют пары {ситуация, принятое решение}, ответами являются значения функционала качества, характеризующего правильность принятых решений (реакцию среды). Как и в задачах прогнозирования, здесь существенную роль играет фактор времени. Примеры прикладных задач: формирование инвестиционных стратегий, автоматическое управление технологическими процессами, компьютерные игры, самообучение роботов и т.д.

Динамическое обучение (online learning) может быть обучением как с учителем, так и без учителя. Специфика в том, что прецеденты поступают потоком. Требуется немедленно принимать решение по каждому прецеденту и одновременно доучивать модель зависимости с учётом новых прецедентов. Как и в задачах прогнозирования, здесь существенную роль играет фактор времени.

В вышеприведенной классификации акцент сделан на методологических и теоретических сторонах. При этом современное машинное обучение – прикладная наука, и имеет смысл привести перечень наиболее часто решаемых сегодня с его помощью практических задач.

Машинное обучение имеет широкий спектр приложений:

- [Приложения в биоинформатике.](#)
- [Приложения в медицине,](#) в том числе [медицинская диагностика.](#)
- [Приложения в геологии и геофизике.](#)
- [Приложения в социологии.](#)

- Приложения в экономике, включая:
 - a. Кредитный скоринг;
 - b. Предсказание ухода клиентов;
 - c. Обнаружение мошенничества;
 - d. Биржевой технический анализ;
 - e. Подбор потенциальных клиентов
- Приложения в технике, в том числе:
 - a. Техническая диагностика;
 - b. Робототехника;
 - c. Интеллектуальное управление
 - d. Компьютерное зрение;
 - e. Распознавание речи.
 - f. Генерация речи
- Приложения в офисной автоматизации, включая:
 - a. Распознавание текста;
 - b. Обнаружение спама;
 - c. Категоризация документов;
 - d. Распознавание рукописного ввода;
 - e. Определение эмоциональной окраски публикаций;
 - f. Суммаризация (реферирование) текстов.

Некоторые задачи, возникающие в прикладных областях, имеют черты сразу нескольких стандартных типов задач обучения, поэтому их трудно однозначно отнести к какому-либо одному типу. Например, формирование инвестиционного портфеля имеет черты динамического обучения с подкреплением, в котором очень важен отбор информативных признаков. Роль признаков играют финансовые инструменты. Состав оптимального набора признаков (портфеля) может изменяться со временем. Функционалом качества является долгосрочная прибыль от инвестирования.

Коллаборативная фильтрация – прогнозирование предпочтений пользователей на основе их прежних предпочтений и предпочтений схожих пользователей. Применяются элементы классификации, кластеризации и восполнения пропущенных данных.

Язык программирования Питон в ИИ

В книге в дальнейшем рассмотрение моделей и алгоритмов машинного обучения будет производиться с использованием языка программирования Питон, что вполне логично в связи с наличием именно в нем большого числа специализированных библиотек для данной области, посему приведем здесь по нему небольшую справку, достаточную для понимания приведенных примеров. Мы будем использовать в дальнейшем именно написание «Питон», а не Python, поскольку именно к змее в конечном счете имеет отношение наименование языка.

Язык Питон (Python) создан в 1989-91 гг. Гвидо Ван Россумом. Автор был фанатом комедийного телешоу «Летающий цирк Монти Питона», почему и назвал так свой язык. С тех пор язык постепенно набирал популярность, и на данный момент (2025 г.) занял первое место в мире по рейтингу Tiobe [...], обойдя такие заслуженные инструменты, как C++ и Java. Важнейшую роль в этом, видимо, сыграли именно доступные обширные библиотеки, а также доступность языка практически на всех известных платформах.

Если говорить о базовых характеристиках языка и его месте среди существующих средств программирования, следует отметить ключевые особенности: язык *интерпретируемый* (изначально даже скорее скриптовый), с динамической типизацией, т.е. без необходимости предварительного объявления типов для переменных, зато с богатым выбором встроенных структур данных (список, кортеж, словарь, и т.п.). Питон является объектно-ориентированным, но нам для использования популярных в области машинного обучения и искусственных нейронных сетей глубокого погружения в эту его особенность не понадобится.

Интересной особенностью языка является использование для выделения логической структуры программы отступов (число пробелов имеет важное значение!) и отсутствие «операторных скобок» вроде «{ ... }» в Си или «begin ... end» в языке Паскаль. Как правило, используется два или четыре пробела.

Для Питона существует несколько доступных инструментов разработки, включая Visual Studio Code, Anaconda, PyCharm и пр. Нужно отметить также, что сегодня все более популярным становится использование онлайн-трансляторов и даже интегрированных сред разработки в Интернете. Естественно, они доступны и для Питона, здесь имеется достаточно широкий выбор.

Весьма интересным и эффективным с практической точки зрения является применение так называемых «блокнотов» (Jupyter Notebook), сочетающих фрагменты программы на Питоне и размеченный текст, в коем могут содержаться пояснения и прочая информация, причем компания Google в порыве невиданной щедрости разрешает использование подобных блокнотов онлайн бесплатно всем пользователям, имеющим подписку на их услуги, включая трансляцию и запуск программ онлайн, даже с предоставлением для этого высокопроизводительных аппаратных средств, что как раз важно для машинного обучения и нейронных сетей, где объем необходимых вычислений может быть весьма значительным. Инструмент называется Google Colab.

Питон относится к языкам традиционной (процедурной, императивной) парадигмы. Основным действием в них является присваивание. Оно записывается с одним знаком равенства, например:

```
a = 5
```

Соответственно, аналогично языку программирования Си, в условиях равенство записывается с помощью удвоения знака: '=='.

Как правило, присваивание используется для вычислений. В Питоне используются следующие обозначения основных операций:

- Сложение (+).
- Вычитание (-).
- Умножение (*).
- Деление (/).
- Целочисленное деление (//).
- Остаток от деления (%).
- Возведение в степень (**).

Напомним, что предварительного объявления переменной с указанием типа в Питоне не требуется, тип переменная получит по результату первого присвоения ей значения. Обращаем повторно внимание и на отсутствие специального разделителя операторов, например, в этом качестве точки с запятой в Питоне **нет**.

Вообще, в языке много встроенных изначально типов данных, включая строки, работа с коими интуитивно понятна из следующих примеров:

```
s = "ма"
s *= 2
s += " мыла раму"
print(s)
```

Результат:

мама мыла раму

Кстати, Питон обладает ещё одной любопытной особенностью: строки в нем можно заключать как в одинарные кавычки (апострофы), так и в двойные, причем допускается это делать вперемишку в одной программе — впрочем, закрывающий знак для каждой строки должен соответствовать открывающему.

Знающий Си читатель уже, наверное, усмотрел ещё несколько аналогий между языками, например сокращенную запись операторов: `s*=2` вместо `s = s * 2`. Впрочем, отсутствуют инкремент и декремент в стиле Си, нет `a++` или `--i`;

Печать (вывод сообщений на консоль) в Питоне организована с помощью функции `print`. Опять же, как и в Си, предусмотрена возможность форматирования выводимых данных, включая мощный механизм так называемых f-строк.

Рассмотрим примеры:

```
pi = 3.1415926
print("Значение пи равно ",pi)
print("С округлением: ",round(pi,2))
print(f"Или так: {pi:.3}")
```

Результат:

Значение пи равно 3.1415926

С округлением : 3.14

Или так: 3.14

Для ввода пользователем значений применяется функция `input`. Необходимо обратить внимание на то, что функция возвращает введенное значение как строку, поэтому в случае необходимости, его нужно преобразовать в нужный тип данных.

В Питоне можно создать подпрограмму с помощью механизма функций. Для определения функции используется ключевое слово `def`. Тело функции выделяется, как и в других ситуациях в Питоне, с помощью отступов вправо (пробелов).

Что достаточно необычно, одна функция может легко возвращать несколько значений одним оператором `return`:

```
# функция „убивает двух зайцев” – сразу и складывает, и
    перемножает параметры
def sum_um(x, y):
    return x+y, x*y
```

```
a = 2
```

```
b = 3
```

```
sum, um = sum_um(a, b)
```

```
print (f'{a} + {b} = {sum}, {a} * {b} = {um}')
```

Результат:

```
2 + 3 = 5, 2 * 3 = 6
```

Конечно, как и в других языках, «хороший тон» программирования подразумевает, что при объявлении функции мы снабжаем ее кратким комментарием - пояснением.

Комментарии в Питоне записываются с помощью символа диез `#`, вся дальнейшая строка рассматривается, как примечание.

Посмотрим на ключевые встроенные типы данных в Питоне.

Простые:

- Целые числа, обозначаются как `int`
- Числа с плавающей запятой (точкой) обозначаются как `float`:
- Строки для хранения текстовой информации.

- Булевы значения обозначаются как `bool`, допустимы значения `True` (ИСТИНА) и `False` (ЛОЖЬ) .

Перейдем к структурным типам данных, способным содержать несколько элементов.

Прежде всего, это список. Причем можно сохранить список в одной переменной, как в Лиспе (члены списка при этом, правда, разграничиваются запятой, и берутся в квадратные скобки):

```
s = [1, 2, 3, 'Маша', 'Витя']
s.append(3.14)
print(s)
```

Результат:

```
[1, 2, 3, 'Маша', 'Витя', 3.14]
```

Здесь интересна конструкция `s.append()`. Функция `append()` является типовой для списков, добавляя в конец ещё элементы. А запись переменной `s` с точкой означает применение объектно-ориентированного подхода к программированию, именно что мы вызываем метод `append()`, имеющийся в классе «список».

Фактически, «списки» Питона не совсем являются списками в классическом понимании, принятом в программировании, они обладают и свойствами массива, в частности, возможен произвольный доступ. Мы можем считывать и перезаписывать элементы по *индексу*-номеру в списке по порядку (нумерация с 0), например: `skills[0] = 'стрессоустойчивость'`.

Словарь — структура данных, в которых значения хранятся по ключу. Записываются как пары `ключ: значение` через запятую в фигурных скобках. Например: `{'profession': 'Python-разработчик', 'months': 6}`.

Считывать и перезаписывать элементы мы можем по ключу:

```
course['profession'] = 'Python Developer'.
```

Замечание. Обратим внимание, что формат словаря в Питоне очень напоминает формат представления данных json.

Замечание. Типы данных в Питоне могут комбинироваться как угодно, например допустим список словарей с вложенными списками словарей.

Питон позволяет в ходе выполнения проверять тип данных, причем у одной переменной динамически тип может меняться, см. пример:

```
# Определение типа переменной
val = 42
print("Тип переменной val:", type(val))
val = "А сейчас уже строка"
print("Тип переменной val:", type(val))
```

Результат:

```
Тип переменной val: <class 'int'>
Тип переменной val: <class 'str'>
```

Функции преобразования типов позволяют перейти от одного типа данных к другому:

- `int("42")`: преобразует строку в целое число.
- `float("3.14")`: преобразует строку в число с плавающей запятой.
- `str(42)`: преобразует число в строку.
- `bool(0)`: преобразует число в логическое значение False.

В следующем примере это демонстрируется, причем нужно обратить внимание на полиморфизм операции „+“

```
# Пример преобразования строки в число с помощью int
number = "42"
print(number+number)
number = int(number)
print(number+number)
```

Результат:

Говоря о логических значениях, отметим, что ноль (и любые другие значения, оцениваемые как пустые, например, `None`, пустой список или пустая строка) преобразуются в `False`, а любые другие числа или непустые объекты преобразуются в `True`.

Применение операторов сравнения иллюстрируется следующим примером:

```
# Примеры операторов сравнения
print("5 == 5:", 5 == 5)
print("5 != 3:", 5 != 3)
print("10 > 5:", 10 > 5)
print("3 < 1:", 3 < 1)
print("10 >= 10:", 10 >= 10)
print("3 <= 2:", 3 <= 2)
```

Результат:

```
5 == 5: True
5 != 3: True
10 > 5: True
3 < 1: False
10 >= 10: True
3 <= 2: False
```

Логические операторы `and`, `or`, `not` позволяют комбинировать несколько условий.

Важнейшими конструкциями языков программирования являются условное выполнение и циклы для повторения действий.

Условный оператор ‘ЕСЛИ .. ТО .. ИНАЧЕ’ в Питоне выглядит так (сокращённый вариант без ветви ИНАЧЕ, полный вариант и вариант с несколькими условиями одно за другим — используется `elif`):

```
if a<b:
    print("a<b")

if a<b:
    print("a<b")
else:
    print("не выполняется a<b")
```

```

if a<b:
    print("a<b")
elif a>b:
    print("a>b")
else:
    print("a равно b")

```

Необходимо здесь обратить внимание, что ветви условия начинаются с символа **двоеточия**.

Цикл ПОКА, думается, не вызовет у читателей вопросов:

```

a = 5
while a>0:
    print ('a = ',a)
    a-=1

```

Результат:

```

a = 5
a = 4
a = 3
a = 2
a = 1

```

Зато классический цикл ДЛЯ нужно записывать весьма специфическим образом, поскольку в Питоне 'главным' является цикл ДЛЯ ИЗ:

```

N = 10
s = 0

for i in range(1,N+1):
    s+=i

print (f"Сумма {N} чисел натурального ряда равна {s}")

```

Результат:

Сумма 10 первых чисел натурального ряда равна 55

Для задания границ изменения переменной цикла используется конструкция `in range`, причем следует отметить, что по умолчанию

отсчет, как в Си, идет от нуля, и необходимо в данном случае было написать именно `in range(1, N+1)`. Встроенная функция `range` возвращает последовательность чисел от указанного начального до конечного значения, но не включая само значение `N`.

Питон — язык объектно-ориентированный, в нем есть классы и методы. Создадим класс, описывающий автомобиль:

```
class Car(object):
    # Наименование класса
    Name_class = "Автомобиль"

    def __init__(self, brand, weight, power):
        self.brand = brand # Марка автомобиля
        self.weight = weight # Вес автомобиля
        self.power = power # Мощность двигателя

    # Метод "двигаться прямо"
    def drive(self):
        # Здесь команды двигаться прямо
        print("Поехали, двигаемся прямо!")

    # Метод "повернуть направо"
    def pravo(self):
        # Здесь команды повернуть руль направо
        print("Едем, поворачиваем руль направо!")

    # Метод "повернуть налево"
    def levo(self):
        # Здесь команды повернуть руль направо
        print("Едем, поворачиваем руль налево!")
```

В этом примере мы создали класс, добавили в него три атрибута и пять методов. На основе этого класса создадим объект — автомобиль с именем `Auto1` с атрибутами: Мерседес, вес— 1200 кг, мощность двигателя— 250 лошадиных сил:

```
Auto1 = Car('Мерседес', 1200, 250)
print('Параметры автомобиля, созданного из класса- ',
      Auto1.Name_class)
```



```

print('Марка (модель) - ', Auto1.brand)
print('Вес (кг) - ', Auto1.weight)
print('Мощность двигателя (лс) - ', Auto1.power)

# Поехали!
Auto1.drive()    # Двигается прямо
Auto1.pravo()    # Поворачиваем направо
Auto1.levo()     # Поворачиваем налево

```

Для подключения библиотечных функций (чем мы будем активно пользоваться, ведь Питон нам нужен именно по причине наличия в нем популярных библиотек машинного обучения и построения нейросетей) используются два варианта: либо подключение библиотеки полностью, либо указывается конкретная функция, нужная из неё:

```

# импортируется библиотека matplotlib
import matplotlib
import pandas as pd
from math import atan # импорт арктангенса

```

В примере с библиотекой `pandas` используется так называемый псевдоним, для дальнейшего быстрого доступа к библиотечным функциям через сокращенное наименование `pd`. А в примере с математической библиотекой `math` из нее импортируется конкретно функция арктангенса.

2. ВВЕДЕНИЕ В ИСКУССТВЕННЫЕ НЕЙРОННЫЕ СЕТИ

Как уже неоднократно отмечалось, машинное обучение может осуществляться несколькими принципиально отличающимися по подходам путями. Мы в настоящем пособии делаем акцент на применении так называемых искусственных нейронных сетей (ИНС), где упор делается на попытке воссоздания (весьма грубого!) того, как работает нервная система человека.

При рассмотрении сложного явления достаточно правильным представляется рассмотреть его истоки и историю развития.

История искусственных нейронных сетей

Итак, к истокам искусственных нейронных сетей можно отнести 1943 год, когда МакКаллок и Питтс опубликовали пионерскую работу «Логическое исчисление идей, относящихся к нервной деятельности» [12], где предложили модель искусственного нейрона.

В 1949 году Хебб предложил правило (способ) обучения нейронной сети.

Важной вехой стал 1957 год, когда Розенблатт разработал принцип построения классического варианта ИНС – *персептрона*. Розенблатт представил первый *нейрокомпьютер* – «Марк-1», который был способен распознавать некоторые буквы английского алфавита. «Марк-1» стал первой аппаратной реализацией нейросети. Нейрокомпьютеры и нейропроцессоры, одной из отличительных черт коих являются массовый параллелизм и устойчивость к ошибкам, и сейчас продолжают создавать, но большую часть современных ИНС представляют программные эмуляции их работы на ЭВМ с традиционной последовательной архитектурой.

Персептроны стали очень активно исследовать. На них возлагали большие надежды. Однако, как оказалось, они имели серьезные ограничения. Известные ученые в области информатики Минский и Пейперт опубликовали книгу о персептронах [12], где показали принципиальную ограниченность возможностей простых однослойных персептронов. Это вызвало определенный спад интереса к ИНС, что может рассматриваться как первые «заморозки» в использовании нейросетевых подходов. В науке ИИ стали перспективными и «модными» другие области. Про нейросети подзабыли, финансирование стали направлять на другие направления исследований. Но потом, по прошествии ряда лет, были открыты новые виды сетей, а также алгоритмы их обучения, что вновь возродило интерес к этой области.

Вопрос читателю. Известны ли Вам другие случаи влияния «моды» на научные исследования? Как Вы оцениваете данное явление, идет ли оно на пользу науке? Обоснуйте свое мнение.

Весьма важную роль в многослойных ИНС играет алгоритм обратного распространения ошибки. Основная идея этого метода состоит в распространении сигналов ошибки от выходов сети к её входам, в направлении, обратном прямому распространению сигналов в обычном режиме работы. Впервые метод был описан в 1974 г. в СССР Галушкиным [33]; существенно развит в 1986 г. Румельхартом, Хинтоном и Вильямсом [34] и независимо Барцевым и Охониным [35].

В 1982 году Хопфилд предложил семейство нейронных сетей, моделирующих ассоциативную память.

Финский ученый Кохонен в 1984 году предложил свой вариант ИНС – нейронную сеть с обучением без учителя, успешно решающую задачи визуализации и кластеризации.

Важной вехой в распознавании изображений стала концепция сверточных нейронных сетей (англ. convolutional neural network) – архитектура, предложенная Яном Лекуном в 1988 году [36]. Она использует некоторые особенности зрительной коры головного мозга человека, в которой были открыты так называемые простые клетки, реагирующие на прямые линии под разными углами, и сложные клетки, реакция которых связана с активацией определённого набора простых клеток. Название архитектура сети получила из-за операции свёртки – специальной обработки небольших фрагментов изображения.

К концу 1990-х можно отнести новый всплеск интереса к искусственным нейронным сетям. Долгая краткосрочная память (англ. Long short-term memory; LSTM) – разновидность архитектуры рекуррентных (с наличием обратных связей) нейронных сетей, предложенная в 1997 году Шмидхубером и Хохрайтером. LSTM-сеть является универсальной в том смысле, что при достаточном числе элементов она может выполнить любое вычисление, на которое способен обычный компьютер, для чего необходима соответствующая матрица весов, которая может рассматриваться как программа.

Наконец, период примерно с 2012 года по настоящее время можно назвать эрой бурного внедрения ИНС, огромные успехи в этой области за счет синергии – суммирующего эффекта взаимодействия нескольких факторов. К этим факторам следует отнести, с одной стороны, планомерное повышение производительности массово доступных ЭВМ

(реализация и обучение ИНС требует весьма значительных вычислительных ресурсов). С другой стороны, стали доступны весьма значительные объемы накопленных в электронном виде используемых в самых разных областях человеческой деятельности данных (Big Data – «большие данные»), которые с успехом можно применять для обучения нейросетей. Были достигнуты новые теоретические успехи в области глубокого обучения сверточных и рекуррентных нейронных сетей, внедрения гибридных архитектур нейронных сетей и т.п.

Основные классы задач, решаемые ИНС

Класс задач, активно и успешно решаемых с использованием искусственных нейронных сетей, во многом соответствует задачам машинного обучения вообще [32], но поскольку ИНС – важнейший инструмент современного ИИ, приведем их в данном параграфе [37].

1. Классификация (распознавание) образов.

На входы ИНС поступает набор сигналов (представляющий образ – изображение, звук, данные радиолокационной или гидроакустической станции, набор показаний различных датчиков и пр.). ИНС должна отнести его к конкретному классу путем активации одного из своих выходов. Пример: на входе – представленная набором пикселей (точек) картинка, на выходе активируется один из сигналов: «кошка» или «собака». Может быть и задача, когда интересующую область на изображении необходимо распознать и выделить, например, обвести.

Замечание. Данная задача – отличить кошку от собаки – легко решается человеком начиная с дошкольного возраста, но до недавнего времени ставившая в тупик ЭВМ.

Вопросы читателю: 1. Считаете ли Вы данную задачу действительно интеллектуальной? 2. Представьте, что на вход подаются не обязательно фотографии, а в том числе – условные схематические рисунки, включая детские, изображающие кошку или собаку.

2. *Кластеризация/категоризация* (задача классификации образов без учителя). Набор классов, к которым необходимо отнести входной объект, заранее неизвестен. Кластеризация основана на подоби и группирует схожие образы в один кластер. Объектами могут быть изображения, звуки, размер зарплаты, заявители на получение кредита и т.д. Применяется при интеллектуальном анализе данных, например, расчете страховки для автовладельцев, и весьма широко в маркетинге (сегментация клиентов).

3. *Аппроксимация функций*. Имеется некоторая функциональная зависимость переменной от нескольких других, например $y=x^2$. Однако нам она неизвестна, есть лишь ряд пар чисел, например $\{ (2, 3.999), (1, 1.001), (-2, 4.001), (5, 24.051), (-3, 8.997) \dots \}$, и нужно затем получить ответ для произвольного числа на входе, например (30, 900).

4. *Предсказание/прогноз*. У нас имеется ряд моментов времени (кои принято называть *отсчетами*) $t_1, t_2, \dots, t_n$ и набор некоторых измеренных в данные моменты значений: $f(t_1), f(t_2), \dots, f(t_n)$. Необходимо найти значение в будущем, например, в следующий момент времени $f(t_{n+1})$. На практике подобная задача возникает при игре на бирже, при предсказании курса валют и пр.

5. *Оптимизация*. Многие проблемы в математике, статистике, технике, медицине, экономике могут быть рассмотрены как задачи оптимизации или нахождения решения задачи, которое должно удовлетворять системе ограничений и при этом максимизировать или минимизировать заданную целевую функцию.

6. *Ассоциативная память* (память, адресуемая по содержанию). В традиционной «фоннеймановской» модели ЭВМ обращаться к памяти можно лишь по адресу, причем адрес никак не связан с содержанием. Содержимое ассоциативной памяти может быть восстановлено по частичному или искаженному содержанию.

7. *Управление*. Имеется некая система, имеющая набор входов X и выходов (параметров) Y . В задачах целевого управления производится поиск такого входного воздействия $X(t)$, при котором система следует по желаемой траектории эталонной модели $Y(t)$.

Биологический нейрон

Современная наука на самом деле еще очень мало знает о том, как устроено мышление человека. Тем не менее есть уверенность в некоторых базовых вещах, например, что процесс мышления тесно связан с головным мозгом.

Мозг человека состоит из нервных клеток – нейронов, связанных между собой так, что импульс возбуждения может передаваться от одного другому. Все процессы передачи раздражения (информации) от кожи, ушей, глаз к мозгу, процессы мышления и управления действиями с помощью мышц реализованы в живом организме с помощью передачи импульсов между нейронами.

На рис. 13.1 представлен биологический нейрон (нервная клетка). Он состоит из тела (сомы), содержащего ядро, и большого числа отростков (нервных волокон, некоторые из них могут иметь весьма значительную длину). С помощью нервных волокон нейрон принимает и передает импульсы, причем большинство связей работает «на прием» сигналов, передача осуществляется с помощью единственного *аксона*.

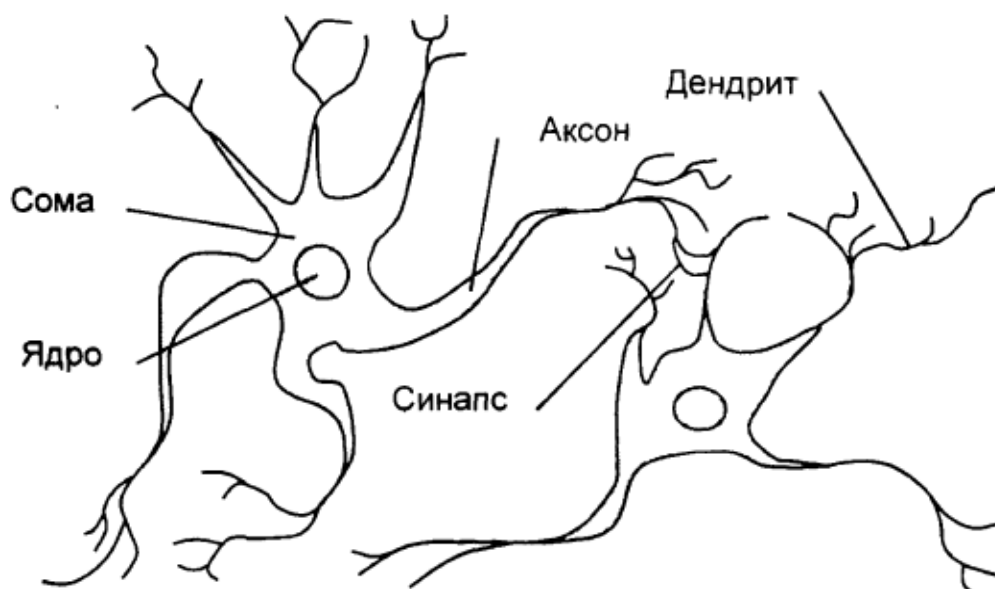


Рис. 2.1. Биологический нейрон

На окончаниях нервных волокон имеются специальные образования – синапсы, отвечающие за величину передаваемого импульса.

Активность синапса в зависимости от процессов, в которых он участвует, меняется, именно на этой основе становятся возможными память и обучение.

По современным представлениям, кора головного мозга человека насчитывает около 80 млрд нейронов, при этом каждый нейрон связан с 10^3 – 10^4 нейронов [37]. Получается, что общее число связей между нейронами в головном мозге оценивается величиной 10^{14} – 10^{15} . Это – поистине большое число. Особенно впечатляющим становится представление о происходящих в мозгу постоянных активациях миллионов нейронов и изменениях в сотнях миллионов синапсов.

Головной мозг человека – наиболее сложная и совершенная из изучаемых наукой систем.

Замечание. Есть мнение, что в некотором смысле он эквивалентен Вселенной, поскольку человек с помощью мозга определил свое место в Галактике, построил модели эволюции Вселенной – например, теорию Большого взрыва, оценил ее возраст. Что Вы думаете по этому поводу?

Структура формального нейрона

Перейдем от нейронов естественных, встречающихся в нервной системе человека и животных, к нейронам искусственным, из коих и складывают искусственные нейронные сети. Подобный элемент сети принято называть *формальным нейроном*, или *нейроноподобным элементом*. Структура его изображена на рис. 13.2.

С математических позиций формальный нейрон реализует скалярную функцию векторного аргумента. Имеется n входов x_n и единственный выход y . Значения сигналов на входах предварительно умножаются на *веса* w_i и суммируются. Иногда к полученной сумме добавляется смещение b , получается значение s . Это значение подается на следующий блок – функциональный преобразователь f . Функция, реализуемая данным блоком, называется функцией активации формального нейрона.

Значение на выходе $y=f(s)$ определяется видом функции активации, оно может быть как непрерывным, так и целым числом.

Существуют бинарные (двоичные) нейронные сети, в которых сигналы принимают значения 1 или 0, бывают и аналоговые сети (сигнал имеет произвольное значение).

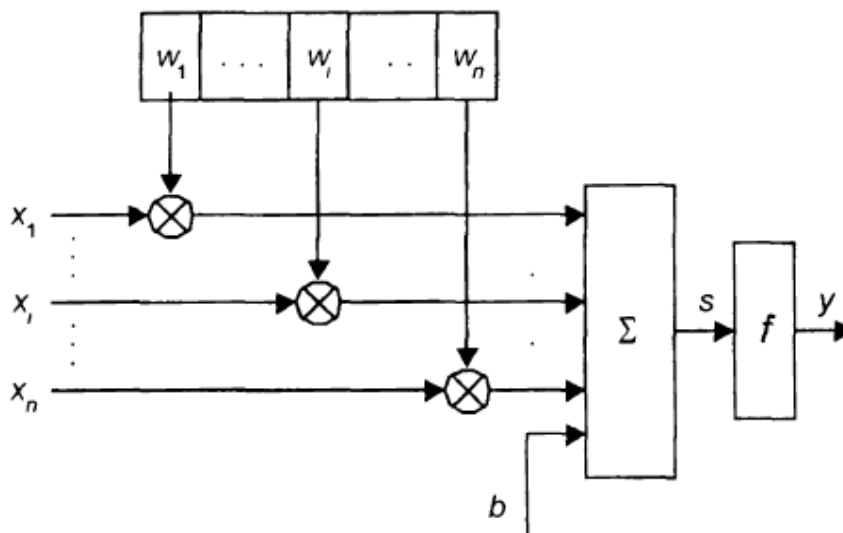


Рис. 2.2. Схема формального нейрона

Замечание. Возможна как аппаратная реализация (электронные схемы, нейропроцессоры) нейроподобного элемента, так и его программное моделирование.

Формулой преобразование формального нейрона может быть выражено как

$$s = b + \sum_{i=1}^n x_i \cdot w_i, y = f(s).$$

Активационная функция f подбирается специальным образом, чтобы реализовать следующее поведение: значения возбуждений на входах могут нарастать длительное время, при этом значение на выходе остается y неизменным, и лишь при преодолении определенного заданного уровня (порога) выход «скачком» переходит в другое состояние. Это соответствует модели функционирования естественного нейрона – формальный нейрон «возбуждается».

На рис. 13.3 представлены примеры активационных функций, используемых в современных ИНС (a – функция единичного скачка, b – линейный порог (гистерезис), c – сигмоид (логистическая функция), d – сигмоид (гиперболический тангенс)).

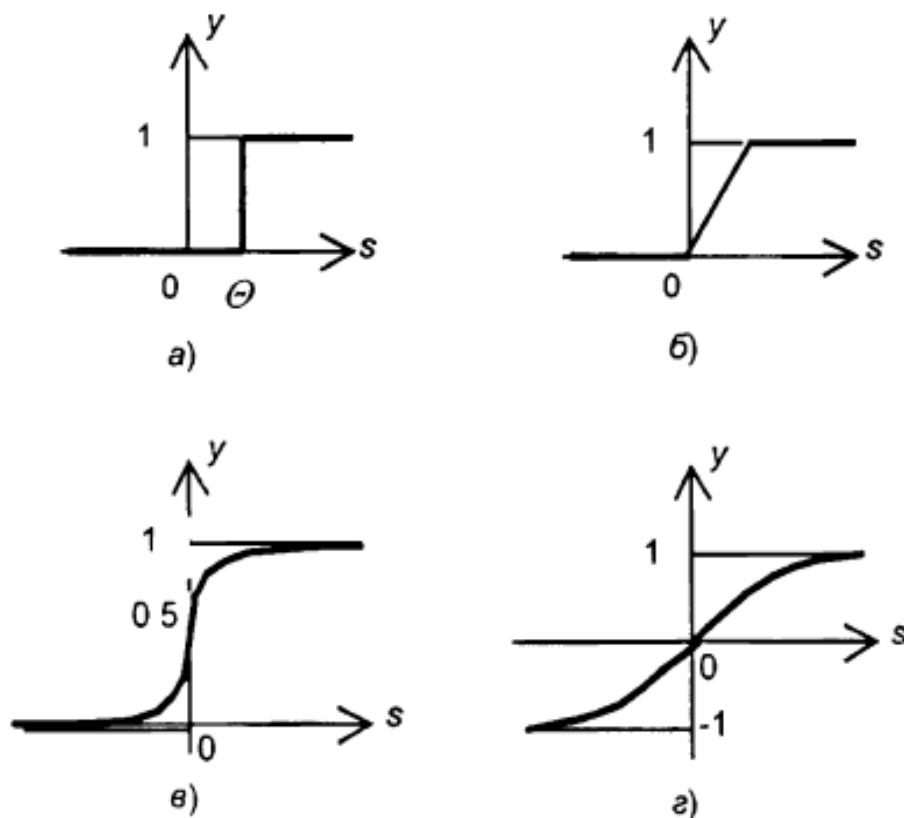


Рис. 2.3. Активационные пороговые функции

Функция единичного скачка – недифференцируемая, равна нулю при всех значениях аргумента, меньших θ , и скачком принимающая значение 1 начиная с момента, когда аргумент становится равен θ .

Линейный порог похож, за исключением того, что «ступенька» не прямая, а имеется небольшой интервал, на коем функция становится линейной.

Весьма значительный интерес с точки зрения так называемого *глубокого обучения*, когда коррекция ошибки проводится в обратном направлении по слоям ИНС с помощью метода обратного распространения ошибки, представляют дифференцируемые на всем множестве аргументов функции активации – сигмоиды (гладкая монотонная возрастающая нелинейная функция, напоминающая форму латинской буквы «S»). Часто в качестве сигмоиды принимают логистическую функцию:

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

Обратим внимание на важные особенности логистической функции: она испытывает «скачок», но гладкий, в районе нулевого значения аргумента, до этого равна нулю, а после – единице.

Альтернативой, используемой в ИНС, можно считать гиперболический тангенс, задаваемый формулой.

$$f(x) = \operatorname{th} \frac{x}{\alpha} = \frac{e^{\frac{x}{\alpha}} - e^{-\frac{x}{\alpha}}}{e^{\frac{x}{\alpha}} + e^{-\frac{x}{\alpha}}}.$$

Отличием гиперболического тангенса от предыдущего случая является то, что область значений здесь: $[-1; 1]$, а область «скачка» носит несколько более пологий характер.

Задание читателю. На произвольном языке программирования написать небольшой модуль (класс), реализующий поведение искусственного нейрона. Смоделировать различные функции активации.

Архитектуры нейронных сетей

Задачи решаются не отдельно взятым формальным нейроном, а большими совокупностями, при этом нейроны могут быть связаны между собой различными топологиями.

Весьма важной и широко применяемой на практике архитектурой является так называемая *многослойная*, её сейчас популярно называть *глубокой*, отсюда происходит термин *глубокое обучение* (Deep Learning).

В многослойной сети входные сигналы ИНС подаются на нейроны первого слоя. Выходные сигналы данного слоя подаются на следующий слой и т.д., пока не будет достигнут выходной слой. Идея заключается в том, что нейроны слоя n передают выходные сигналы слою $n+1$. Слои, находящиеся между входным и выходным, называют *скрытыми*.

Как именно слои связаны между собой, определяется в зависимости от целевой задачи; варианты связей могут

варьироваться, как и число нейронов в разных слоях. Наиболее часто каждый из выходов предшествующего слоя связан со всеми входами каждого нейрона последующего слоя, но может использоваться и частичная связанность. Если сигнал строго передается по направлению от входных слоев к выходным, сеть называется сетью *прямого распространения* (рис. 13.5).

В многослойных («глубоких») архитектурах нейронных сетей могут быть и *обратные связи*, когда выход формального нейрона слоя n подается на вход нейрона одного из предшествующих слоев m , $m < n$. Подобные сети называют *рекуррентными* нейронными сетями.

Интуитивно ясно, что более сложные задачи решаются сетями, состоящими из большего числа нейронов. Однако эта зависимость не носит жесткого характера. На практике число слоев ИНС ограничено доступными ресурсами.

Конкретным примером нейронной сети является так называемый *однослойный персептрон*. Это – вырожденный случай слоистой сети с $n=1$. Входные нейроны одновременно являются и выходными, все входные сигналы подаются на входы всех нейронов, выходы всех нейронов являются выходами всей нейронной сети.

Можно дифференцировать ИНС и по следующим признакам:

1. **Монотонные.** Каждый слой, кроме выходного, разбит на два блока: возбуждающие и тормозящие. Связи между блоками аналогично делятся на возбуждающие и тормозящие. Для нейронных монотонных сетей необходима монотонная зависимость выхода от параметров входного сигнала.

2. **Сети без обратных связей.** В них нейроны входного слоя получают входной сигнал, их выходы связаны с нейронами первого скрытого слоя, обычно каждый выходной сигнал n -ного слоя подается выходной сигнал $n+1$ слоя, но может быть и «скачок», когда сигнал слоя n передается на слой $n+d$.

3. **Сети с обратными связями.** В этих сетях информация может передаваться с последующих слоев предыдущим, среди них, в свою очередь, выделяют следующие:

а) *слоисто-циклические*, в которых слои замкнуты в кольцо, выходной слой передает сигналы входному, при этом все слои равноправны и могут служить как входами, так и выходами;

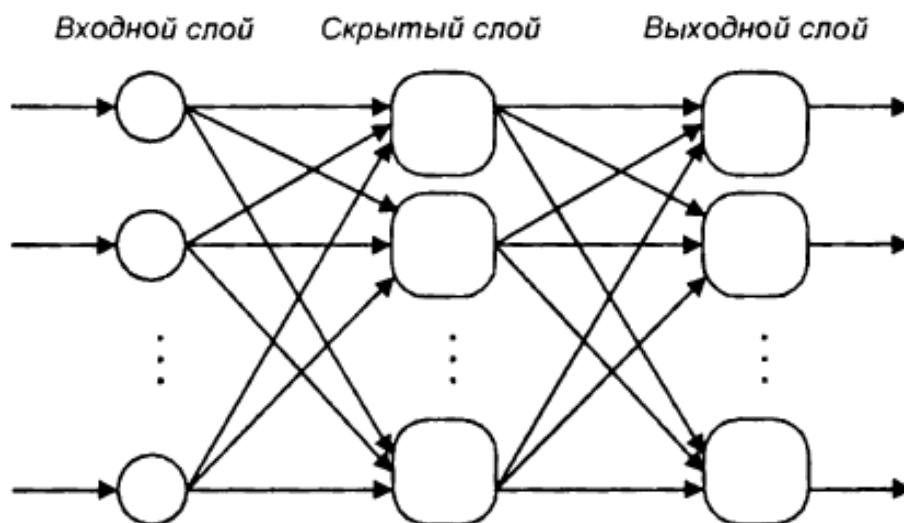


Рис. 2.4. Многослойная сеть прямого распространения

б) *слоисто-полносвязные*, состоят из слоев, каждый из которых представляет собой полносвязную сеть, а сигналы передаются как от слоя к слою, так и внутри слоя; в каждом слое цикл работы распадается на три части:

- 1) прием сигналов предыдущего слоя;
- 2) обмен сигналами внутри слоя;
- 3) выработка выходного сигнала и передача следующему слою.

в) *полносвязанно-слоистые*, по своей структуре аналогичны слоисто-полносвязным, но функционирующие по-другому, в них не разделяются фазы внутри слоя, на каждом такте нейроны всех слоев принимают сигналы от нейронов как своего слоя, так и последующих.

Подобные нейросети хорошо подходят для обработки данных, имеющих контекст, когда последующий элемент данных зависит от предыдущих — например, временных рядов, аудиозаписей, текстов на естественном языке.

В качестве примеров сетей с обратными связями (рекуррентных) на рис. 13.6, 13.7 приводятся архитектуры Элмана и Жордана. Весьма важной специфической рекуррентной архитектурой является LSTM-сеть.

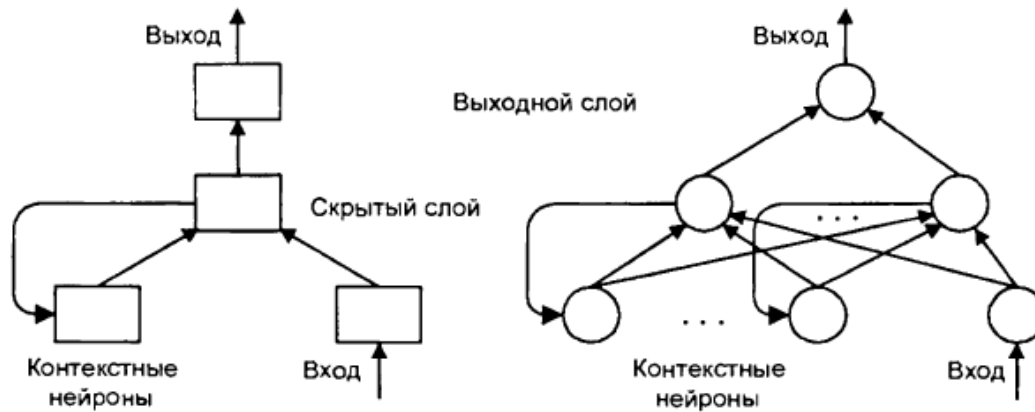


Рис. 2.5. Частично-рекуррентная нейронная сеть Элмана

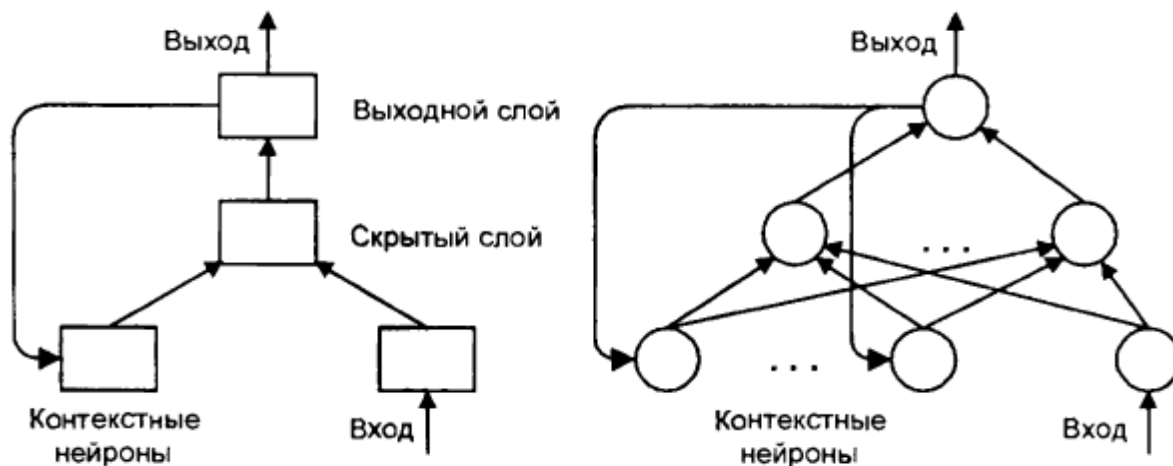


Рис. 2.6. Частично-рекуррентная нейронная сеть Жордана

Задание читателю. Определить и прокомментировать различия рекуррентных нейронных сетей Элмана и Жордана.

Замечание. Рекуррентные нейронные сети могут описывать конечный автомат, при этом не теряя преимуществ ИНС.

Сети могут быть *гомогенными*, если состоят из нейронов с одинаковыми функциями активации, и *гетерогенными*, если нейроны различны между собой.

Еще одна классификация делит нейронные сети на *синхронные* и *асинхронные*. Ход времени в нейронных сетях задается выполнением алгоритма определенных действий над нейронами. В синхронных в каждый момент времени лишь один нейрон изменяет состояние, в асинхронных сетях состояние меняется сразу у целой группы нейронов, как правило, у слоя.

Приведем, наконец, и следующую классификацию.

Однородная нейронная сеть – сеть, в которой все нейроны выполняют одинаковые функции.

Неоднородная сеть – сеть, в которой нейроны выполняют разные функции.

Теоремы Колмогорова – Арнольда и Хехт-Нильсена

Выдающимися отечественными математиками Арнольдом и Колмогоровым, решавшими задачу построения многомерного отображения $X \rightarrow Y$ с помощью математических операций над не более чем двумя переменными, был получен ряд важных теоретических результатов [37]:

1. Теорема о возможности представления непрерывных функций нескольких аргументов суперпозициями непрерывных функций меньшего числа переменных.

2. Теорема о представлении любой непрерывной функции трех аргументов в виде суммы функций не более двух переменных.

3. Теорема о представлении непрерывных функций нескольких переменных в виде суперпозиции непрерывных функций одного переменного и сложения.

Замечание. Буквально в процессе написания этой книги была анонсирована «революционная» архитектура искусственных нейронных сетей — KAN (Kolmogorov Arnold Network, сеть Колмогорова-Арнольда, где подбираются не веса нейронов и функции активации, а функции на «ребрах» сети, или связях между нейронами. Эта сеть, по утверждению ее создателей, значительно более эффективно решает ряд задач, например, в области математики, нежели многослойные нейросети с привычной архитектурой, даже самые лучшие.

Хехт-Нильсен в 1987 году применил подход Колмогорова-Арнольда к нейронным сетям. Его теорема доказывает представимость функций нескольких переменных достаточно общего вида с помощью двухслойной нейронной сети с прямыми полными связями и n нейронами входного слоя, $(2n+1)$ нейронами скрытого

слоя с заранее известными ограниченными функциями активации (например, сигмоидальными) и m нейронами выходного слоя с неизвестными функциями активации.

Теорема Хехт-Нильсена фактически в неконструктивной форме доказывает разрешимость задачи представления произвольной функции искусственной нейронной сетью с нахождением для каждого вида задач минимально необходимого для решения числа нейронов.

Следствие 1

Любая функция нескольких переменных представима с помощью нейронной сети фиксированной размерности. Неизвестными остаются параметры сигмоидальных функций и вид функции активации нейронов выходного слоя.

На практике в нейронных сетях как для первого, так и для второго слоя используют сигмоидальные передаточные функции, параметры которых настраиваются индивидуально для каждого нейрона в ходе обучения сети.

Следствие 2

Для любого множества пар (x, y) существует двухслойная однородная нейронная сеть первого порядка с последовательными связями и с конечным числом нейронов, реализующая отображение $x \rightarrow y$, причем нейроны в этой двухслойной сети должны иметь сигмоидальные передаточные функции.

Наконец, наиболее важное следствие может быть названо «теоремой о полноте нейронных сетей»: Любая непрерывная функция на замкнутом ограниченном множестве может быть равномерно приближена функциями, вычисляемыми нейронными сетями, если функция активации нейрона непрерывна и дважды непрерывно дифференцируема.

Таким образом, нейронные сети являются универсальными структурами, поддерживающими любой вычислительный алгоритм [31, 37].

Выбор архитектуры ИНС осуществляется в зависимости от решаемой задачи. Для определенных классов задач уже известны оптимальные конфигурации. Если задача не сводится к одной из известных разновидностей, приходится решать проблему синтеза архитектуры.

Замечание. Интеллектуально емкую задачу подбора архитектуры искусственной нейронной сети сейчас предлагается решать с помощью ИИ, основанного, в свою очередь, на ИНС. Получается, нейросеть строит нейросеть. Что Вы можете сказать по этому поводу?

Обучение нейронной сети

Читатель уже понял, что программирование нейронных сетей не производится, алгоритм работы сети в виде подробных пошаговых инструкций для решения конкретной задачи явным образом не задается.

Результат вычисления нейронной сети зависит от текущих сформированных значений весов нейронов. Веса же получают свои значения (настраиваются) в результате процесса обучения.

Соответственно, для ИНС процесс их обучения имеет определяющее значение. От его качества напрямую зависят получаемые полезные возможности — или бесполезность/неприменимость нейронной сети для решения данной задачи, если обучение проведено некачественно/на неадекватных примерах.

Процесс обучения нейронной сети состоит из двух этапов:

1) Сначала весам на входах нейронов присваиваются начальные (случайные) значения, как правило — небольшие.

2) Происходит поэтапная корректировка (подстройка) весов в ходе обучения, итеративно улучшающая функционал качества.

Один полный проход по данным обучающей выборки применительно к ИНС принято называть *эпохой*.

Обучение нейросети с учителем

Обучение с учителем подразумевает наличие некоего внешнего звена, предоставляющего нейронной сети помимо входных сигналов «правильные» выходные образы. Иными словами, на предварительном этапе решения задачи необходимо наличие эксперта.

Замечание. Говорят уже о новой профессии эпохи массового внедрения ИНС – «разметчика» данных. При этом речь идет именно о человеке, выполняющем вышеуказанные функции.

При обучении с учителем набор исходных данных делят на две части: собственно обучающую выборку и тестовые данные. Обучающие данные подаются сети для обучения, а проверочные используются для расчета ошибки сети (проверочные данные для обучения сети не применяются). Если на проверочных данных от эпохи к эпохе ошибка уменьшается, то сеть действительно выполняет свою задачу. Обучение прекращают волевым решением учителя, когда, по его мнению, ошибка сети на тестовых данных становится приемлемо низкой.

Замечание. Число эпох для качественного обучения нейросети обычно бывает довольно большим и измеряется тысячами и десятками тысяч. Соответственно, нужно достаточно большое число прецедентов для обучения. Это необходимо учитывать при намерении практического использования нейросетей.

Если ошибка на обучающих данных продолжает уменьшаться, а ошибка на тестовых данных увеличивается, значит, сеть перестала фактически работать и просто «запоминает» обучающие данные. Это явление называется *переобучением* сети. В процессе обучения могут

проявиться другие проблемы, такие как паралич или попадание сети в локальный минимум поверхности ошибок.

Замечание. При использовании «интеллектуальных» систем на основе нейросетей следует быть осторожным. Даже в случае успешного, на первый взгляд, обучения сеть не всегда обучается именно тому, чего от неё хотел создатель. Известен случай, когда сеть обучалась распознаванию изображений танков по фотографиям, однако позднее выяснилось, что все изображения были получены на одинаковом фоне. В результате сеть «научилась» распознавать этот тип ландшафта, вместо того чтобы «научиться» распознавать танки. Таким образом, сеть «поняла» не то, что было нужным, а то, что оказалось проще обобщить.

В многослойных ИНС оптимальные выходные значения всех слоев, кроме выходного, как правило, неизвестны. Трехслойный персептрон уже невозможно обучить, руководствуясь лишь величинами ошибок на выходах сети. Как поступить? Один из вариантов – подбор наборов «правильных» выходных сигналов для каждого из слоев сети, что весьма трудоемко и не всегда осуществимо. Второй вариант – динамическая подстройка весов синапсов, в ходе которой выбираются, как правило, самые слабые связи и изменяются на малую величину в сторону уменьшения или увеличения и сохраняются лишь изменения, кои привели к уменьшению ошибки на выходе всей сети. Данный способ требует громоздких рутинных вычислений. И, наконец, наиболее приемлемый на практике вариант – распространение по специальному алгоритму «сигналов ошибки» от выходных слоев ИНС к входным, в обратном направлении прохождению самих обрабатываемых сетью данных.

Весьма популярны среди методов обучения нейросетей с учителем:

Методы локальной оптимизации с вычислением частных производных первого порядка, в том числе градиентный метод (наискорейшего спуска), оптимизация с использованием антиградиента, метод сопряженных градиентов.

Методы локальной оптимизации с вычислением частных производных первого и второго порядка, в том числе метод Ньютона, методы, базирующиеся на разреженных матрицах Гессе, метод Гаусса – Ньютона; метод Левенберга – Маркардта, стохастические методы (например, поиски случайного направления, метод Монте-Карло, метод «имитации отжига»).

Однако, названным методам присущи и определенные недостатки. Методы локальной оптимизации могут находить лишь локальные экстремумы, в связи с этим дополнительно требуются соответствующие усилия.

Стохастический алгоритм требует весьма большого числа шагов обучения, что делает невозможным его практическое использование для обучения нейронных сетей больших размерностей. Экспоненциальный рост вычислений в алгоритмах глобальной оптимизации также делает невозможным их использование в этом случае.

Одним из практически применяемых является метод *обратного распространения ошибки* [11, 16], являющийся итеративным градиентным алгоритмом обучения, минимизирующим среднеквадратичное отклонение значений выходных сигналов многослойных нейронных сетей.

Минимизируемой целевой функцией ошибки нейронной сети при этом является величина

$$E(w) = \frac{1}{2} \sum_{j,k} (y_{j,k}^{(Q)} - d_{j,k})^2,$$

в которой $y_{j,k}^{(Q)}$ – текущее состояние нейрона j выходного слоя ИНС при подаче на ее вход k -ого образа, $d_{j,k}$ – необходимое выходное состояние этого нейрона.

Суммирование ведется по всем нейронам выходного слоя и по всем подаваемым на вход сети образам. Минимизация методом градиентного спуска обеспечивает подстройку весовых коэффициентов по формуле

$$\Delta w_{ij}^{(q)} = -\eta \frac{\partial E}{\partial w_{ij}},$$

где w_{ij} – вес синаптической связи, соединяющей i -ый нейрон слоя $(q-1)$ с j -ым нейроном слоя q , η – коэффициент скорости обучения, $0 < \eta < 1$.

Полностью алгоритм обучения многослойной ИНС с обратным распространением ошибки выглядит следующим образом (предварительно веса иницируются случайными значениями):

Шаг 1. Подать на входы сети один из возможных образов и в режиме обычного функционирования нейронной сети, когда сигналы распространяются от входов к выходам, рассчитать результат.

Шаг 2. Рассчитать ошибку для выходного слоя по формуле

$$\delta_j^{(Q)} = (y_j^{(Q)} - d_j) \frac{dy_j}{ds_j},$$

где s_j – взвешенная сумма входных сигналов нейрона j – аргумент активационной функции.

Рассчитать по нижеследующей формуле изменения слоев для слоя Q .

$$\Delta w_{ij}^{(q)} = -\eta \delta_j^{(q)} y_i^{(q-1)}$$

(иногда для придания процессу коррекции весов некой инерционности, сглаживающей резкие скачки по поверхности целевой функции, формула дополняется значением изменения веса на предыдущей итерации):

$$\Delta w_{ij}^{(q)}(t) = -\eta (\mu \Delta w_{ij}^{(q)}(t-1) + (1-\mu) \delta_j^{(q)} y_i^{(q-1)}),$$

где μ – коэффициент инерционности, t – номер итерации.

Шаг 3. Рассчитать по итерационной формуле

$$\delta_j^{(q)} = \left[\sum_r \delta_r^{(q+1)} w_{jr}^{(q+1)} \right] \frac{dy_j}{ds_j}.$$

и вышеприведенным формулам для $\delta w_{ij}^{(Q)}$ значения ошибки и коррекции весов для всех остальных слоев.

Шаг 4. Подстроить все веса в нейронной сети на данной итерации:

$$w_{ij}^{(q)}(t) = w_{ij}^{(q)}(t-1) + \Delta w_{ij}^{(q)}(t).$$

Шаг 5. Если ошибка ИНС все еще существенна, перейти на первый шаг. В противном случае – завершение алгоритма.

Поскольку создание искусственного интеллекта движется по пути воспроизведения природных прообразов, ученые не прекращают спор на тему, можно ли считать алгоритмы обучения с учителем натуральными или же нет. Например, обучение человеческого мозга, на первый взгляд, происходит без учителя: на зрительные, слуховые, тактильные и прочие рецепторы поступает информация извне, и внутри нервной системы происходит некая самоорганизация. Однако нельзя отрицать и того, что в жизни человека немало учителей – и в буквальном, и в переносном смысле.

Главная черта, делающая этот метод привлекательным, – его самостоятельность. В ходе обучения в сети без внешнего контроля происходит подстройка весов синапсов. Некоторые алгоритмы, правда, изменяют и структуру сети – число нейронов и их взаимосвязи, но подобные преобразования правильнее назвать самоорганизацией.

Рассмотрим два наиболее известных метода обучения ИНС без учителя: метод Хебба и метод Кохонена (самоорганизующиеся карты Кохонена) [37].

Сигнальный метод Хебба заключается в следующем. При обучении по данному методу усиливаются связи между возбужденными нейронами. Веса меняются по следующей формуле:

$$w_{ij}(t) = w_{ij}(t-1) + \alpha \cdot y_i^{(n-1)} \cdot y_j^{(n)},$$

где $y_i^{(n-1)}$ – выходное значение нейрона i слоя $(n-1)$, $y_j^{(n)}$ – выходное значение нейрона j слоя n ; $w_{ij}(t)$ и $w_{ij}(t-1)$ – весовой коэффициент синапса, соединяющего эти нейроны, на итерациях t и $t-1$ соответственно; α – коэффициент скорости обучения. Здесь и далее для общности под n подразумевается произвольный слой сети.

Существует и метод, получивший название «дифференциальный метод обучения Хебба», когда сильнее всего усиливаются связи синапсов, соединяющих нейроны, сигналы на выходах коих на предыдущем шаге наиболее динамично изменились в сторону увеличения. Описывается это нижеследующей формулой:

$$w_{ij}(t) = w_{ij}(t-1) + \alpha \cdot [y_i^{(n-1)}(t) - y_i^{(n-1)}(t-1)] \cdot [y_j^{(n)}(t) - y_j^{(n)}(t-1)].$$

Алгоритм обучения с применением вышеприведенных формул будет выглядеть следующим образом:

1. На стадии инициализации всем весовым коэффициентам присваиваются небольшие случайные значения.

2. На входы сети подается входной образ, и сигналы возбуждения распространяются по всем слоям согласно принципам классических ИНС, т.е. для каждого нейрона рассчитывается взвешенная сумма его входов, к которой затем применяется активационная (передаточная) функция нейрона, в результате чего получается его выходное значение $y_i^{(n)}$, $i=0...M_i-1$, где M_i – число нейронов в слое i ; $n=0...N-1$, а N – суммарное число слоев в сети.

3. На основании полученных выходных значений нейронов по одной из вышеприведенных формул производится изменение весовых коэффициентов.

4. Цикл с шага 2, пока выходные значения сети не стабилизируются с заданной точностью. Применение этого нового способа определения завершения обучения, отличного от использовавшегося для метода обратного распространения ошибки, обусловлено тем, что подстраиваемые значения синапсов фактически ничем не ограничены.

Затем поочередно предъявляются все образы из входного набора.

Следует отметить, что вид откликов на каждый класс входных образов здесь заранее не будет известен, представляя собой произвольное сочетание состояний нейронов выходного слоя, обусловленное случайным распределением весов на стадии инициализации.

Вместе с тем сеть способна *обобщать* схожие образы, относя их к одному классу. Тестирование обученной сети позволяет определить топологию классов в выходном слое. Для приведения откликов обученной сети к удобному представлению можно дополнить сеть одним слоем, который, например, по алгоритму обучения однослойного персептрона необходимо заставить отображать выходные реакции сети в требуемые образы.

Другой метод обучения без учителя – дифференциальный метод Кохонена, основанный на том, что алгоритм минимизирует разницу между входными сигналами нейрона, поступающими с выходов

нейронов предыдущего слоя, и весовыми коэффициентами его синапсов. Можно описать это формулой

$$w_{ij}(t) = w_{ij}(t-1) + \alpha \cdot [y_i^{(n-1)} - w_{ij}(t-1)] .$$

Полный алгоритм обучения имеет примерно такую же структуру, как в методах Хебба, но на шаге 3 из всего слоя выбирается нейрон, значения синапсов которого максимально подходят на входной образ, и подстройка весов проводится только для него.

Эта так называемая «аккредитация» нейрона может сопровождаться затормаживанием всех остальных нейронов слоя и введением выбранного нейрона в насыщение. Выбор аккредитуемого нейрона может осуществляться, например, расчетом скалярного произведения вектора весовых коэффициентов с вектором входных значений. Максимальное произведение задает «выигравший» нейрон.

Другой вариант – расчет расстояния между этими векторами в p -мерном пространстве, где p – размер векторов:

$$D_j = \sqrt{\sum_{i=0}^{p-1} (y_i^{(n-1)} - w_{ij})^2} ,$$

где x_i – i -тая компонента вектора входного образа или вектора весовых коэффициентов, а n – его размерность.

Подобный подход позволяет сократить длительность процесса обучения.

Пример создания нейросети на языке Питон

Материалы данного подраздела основаны на книге [14].

Начнем с простейшего. Создадим программную реализацию нейрона с сигмоидальной функцией активации на языке Питон в виде класса:

```
# Модель нейрона
import numpy as np

# функция активации:  $f(x) = 1 / (1 + e^{(-x)})$ 
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
```

```
# Создание класса нейрон
class Neuron:
    def __init__(self, w):
        self.w = w

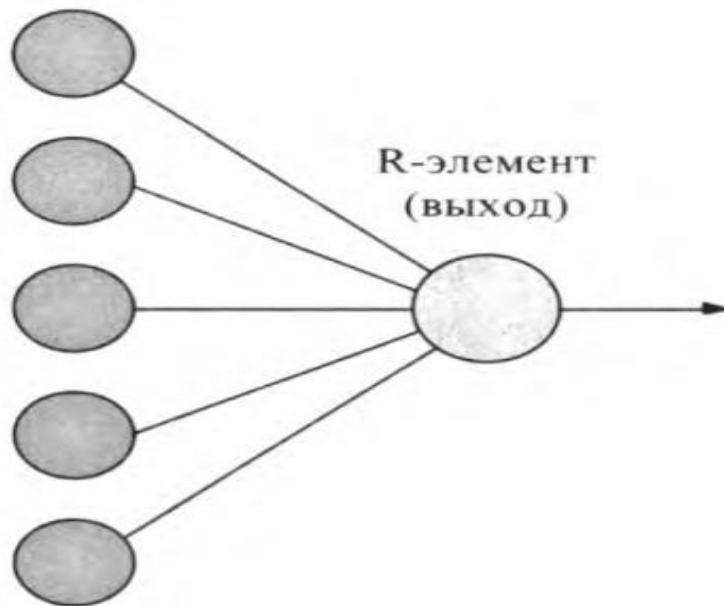
    def y(self, x): # Сумматор
        s = np.dot(self.w, x) # Суммируем входы
        return sigmoid(s) # функция активации

Xi = np.array([0, 0, 1, 1]) # Задание значений входам
Wi = np.array([5, 4, 3, 1]) # Веса входных сенсоров

n = Neuron(Wi) # Создание объекта из класса Neuron
print('Y= ', n.y(Xi)) # Обращение к нейрону
```

Создадим персептрон, см. рис. 2.7

S- или А-элементы
(входы)



R-элемент
(выход)

Рис
унок
2.7

Итак, мы
имеем
односл
ойный
персепт
рон.
Это
может

показаться странным, но даже в таком виде персептрон будет работать, решать задачи классификации.

Как учат детей распознавать цифры? Им показывают картинки с изображением цифр и говорят: «Это цифра 1. Это цифра 2» и т. д.

Аналогично поступим и с нашим персептроном — будем показывать ему изображения цифр и пояснять, какой цифре соответствует каждое изображение.

Как же передать смысл изображения нашему персептрону? Картинка состоит из пикселей. Значение каждого пикселя можно передавать одному сенсору персептрона. В этом случае сколько пикселей будет на картинке, столько нужно будет создать S-элементов для персептрона.

Для того чтобы упростить понимание процесса обучения персептрона, упростим и поставленную перед ним задачу. А именно: научим его распознавать только черно-белые цифры от 0 до 9; все цифры будут одного размера и вписаны черными квадратами в таблицу размером 3 квадрата в ширину и 5 квадратов в высоту; персептрон на должен распознать одну цифру.

Сформируем обучающую выборку, т.е. создадим набор картинок, которые мы будем демонстрировать персептрону на этапе обучения (рис. 2.8).

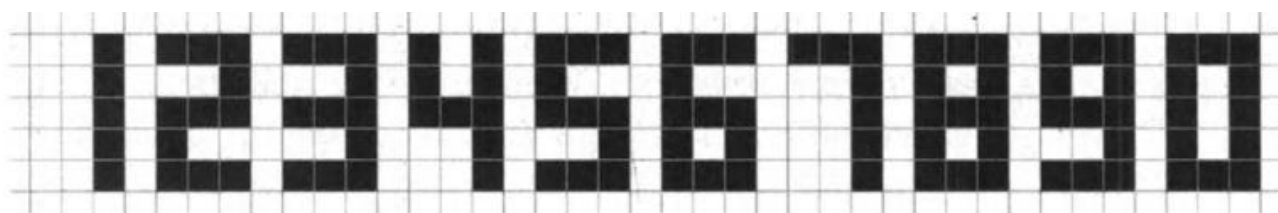


Рисунок 2.8.

Для решения нашей задачи потребуется 15 S-элементов, каждый из которых будет получать сигнал от одного квадрата таблицы размером 3x5. Черный цвет квадрата будет соответствовать возбуждению S-элемента. Белый цвет выхода — нахождению сенсора в состоянии покоя.

Для того чтобы работать с персептроном, мы должны подавать на его входы сигналы в виде чисел. Условимся, что черный цвет квадрата будет соответствовать возбуждению S-элемента (значение такого сигнала будет равно 1). Белый цвет квадрата будет соответствовать нахождению сенсора в состоянии покоя (значение такого сигнала будет равно 0).

В нашем примере каждая цифра представляет собой пятнадцать квадратиков, причем только двух возможных цветов. Как мы условились, за белый квадратик отвечает 0, а за черный квадратик —

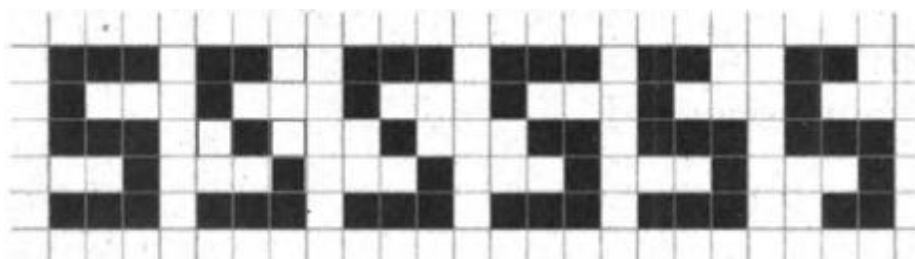
[illegible]

У персептрона данные от сенсоров поступают на сумматор. Для записи каждой цифры мы использовали таблицу из 5 строк и 3 столбцов. «Суммаризируем» строки, т.е. объединим все строки таблицы в одну длинную строку, содержащую сочетание нулей и единиц, которое будет соответствовать каждой цифре. Результат такой работы «сумматора» представлен в нижеследующей таблице:

Цифра	Сигналы от сенсоров	Значение R-элемента (посимвольное сложение)
1	001 + 001 + 001 + 001 + 001	001001001001001
2	111 + 001 + 111 + 100 + 111	111001111100111
3	111 + 001 + 111 + 001 + 111	111001111001111
4	101 + 101 + 111 + 001 + 001	101101111001001
5	111 + 100 + 111 + 001 + 111	111100111001111
6	111 + 100 + 111 + 101 + 111	111100111101111
7	111 + 001 + 001 + 001 + 001	111001001001001
8	111 + 101 + 111 + 101 + 111	111101111101111
9	111 + 101 + 111 + 001 + 111	111101111001111
0	111 + 101 + 101 + 101 + 111	111101101101111

Однако нужно иметь в виду, что мы сильно упростили условия задачи: на вход персептрону даем цифры одинакового размера (таблицы размером 3 x 5), и все клетки этих таблиц заполнены в

соответствии с образцами. Такой персептрон будет успешно работать лишь в том случае, если ему станут демонстрировать изображения цифр, которые идеально соответствуют обучающей выборке. Но ведь в реальной жизни цифры имеют разный размер, написаны разным шрифтом, и еще в большей степени различается одна и та же цифра, написанная разными людьми от руки. На рис. 2.10 представлены



разные варианты изображения цифры 5.

Рисунок 2.10

Даже в рамках одного размера цифра 5 имеет небольшие различия в написании. Человек без труда проигнорирует эти отличия и правильно определит значение этой цифры. Но если подать эти изображения на вход нашего упрощенного персептрона, то он не найдет правильного ответа, поскольку в его памяти отсутствуют символьные строки, соответствующие такой комбинации черных и белых точек. Да, наш упрощенный персептрон чему-то научился, но этих уроков оказалось недостаточно. Нужно продолжить обучение. В частности, в упрощенной модели персептрона мы задали такие условия, при которых значения весов всех связей одинаковы и равны 1. А вот если мы теперь научим персептрон самостоятельно подбирать веса связей для различных комбинаций значений сенсоров, то фактически научим его различать разный стиль написания той же цифры 5, и не только этой цифры.

Уже было упомянуто в предыдущих разделах, что процесс обучения заключается в подборе весов в цепочке связей между сенсорными элементами и сумматором. Мы знаем, что важность тем или иным входам (в нашем случае S-элементам) придают веса, связывающие их с выходным элементом. Чем сильнее вес какой-то связи, тем сильнее этот сенсор влияет на конечный результат. Но как менять вес, в какую сторону, на какую величину? Если мы станем

случайным образом изменять веса и смотреть на результаты, то процесс обучения может затянуться на годы. Вот тут мы подошли к главному — мы должны научить нейронную сеть обучаться самостоятельно. Иными словами, мы должны создать «программу-учителя», которая будет «давать уроки» нейронной сети.

Алгоритм, коий мы используем, нам уже знаком, в 1949 году его предложил канадский физиолог Дональд Хебб (рис. 4.30). В этом алгоритме использованы так называемые правила Хебба . Исследуя работу мозга, физиолог сформулировал следующий постулат: *если аксон клетки A находится достаточно близко, чтобы возбуждать клетку B, и неоднократно или постоянно принимает участие в ее возбуждении, то наблюдается некоторый процесс роста или метаболических изменений в одной или обеих клетках, ведущий к увеличению эффективности A как одной из клеток, возбуждающих B.*

На основе этого постулата были сформулированы два правила:

- Правило 1. Если сигнал персептрона неверен и равен 0, то необходимо увеличить веса тех входов, на которые была подана единица.
- Правило 2. Если сигнал персептрона неверен и равен 1, то необходимо уменьшить веса тех входов, на которые была подана единица.

По сути, на этих двух правилах и основан алгоритм обучения персептронов для решения простейших задач классификации, когда входы могут быть равны только 0 или 1. Попробуем написать простенькую программу, которая будет обучать нейронную сеть распознавать цифры с использованием этих правил. Распишем для начала, какие шаги нужно сделать, чтобы нейронная сеть научилась правильно распознавать, например, цифру 5:

1. Если нейронная сеть правильно распознала или отвергла цифру 5, то мы ничего не предпринимаем (она справилась с задачей).

2. Если нейронная сеть ошиблась и неверно распознала предъявленную цифру (например, цифру 3 приняла за 5, — *первый тип ошибки*), то мы уменьшаем веса тех связей, через которые прошел положительный сигнал. Другими словами, нужно уменьшить

влияние тех сенсоров, которые возбудились и привели к неправильному выводу.

3. Если нейронная сеть ошиблась и не распознала предъявленную цифру (например, цифру 5 приняла за другую цифру, — *второй тип ошибки*), то мы должны увеличить все веса, через которые прошел положительный сигнал. Другими словами, нужно увеличить влияние тех сенсоров, которые возбудились и привели к неправильному выводу.

Алгоритм самообучения нейронной сети, на первый взгляд, противоречит логике обработки ошибок. Все привыкли, что ошибка — это некий единственный элемент, который может иметь только два состояния: 1 — ошибка есть, 0 — ошибки нет. Но здесь нужно учитывать два вида ошибок, т.е. с точки зрения ошибок нейронная сеть может быть в четырех состояниях: первый тип ошибки есть/нет, второй тип ошибки есть/нет.

Составим алгоритм работы такой программы на примере обучения распознавания цифры 5:

1. Формируем обучающую выборку (набор идеальных сигналов сенсоров) для всех цифр от 0 до 9.

2. Получаем двумерный массив размером 10×15 . Здесь 10 — количество цифр (от 0 до 9), 15 — количество сенсоров, характеризующих каждую цифру.

3. Обнуляем веса связей для всех 15 сенсоров, которые характеризуют цифры. В предыдущем разделе мы определили, что каждая цифра представляет собой таблицу из пяти строк и трех столбцов, в которой каждая ячейка имеет черный или белый цвет (рис. 4.27). Создаем такую же ячейку, в которой будем хранить веса сенсорных элементов. Поскольку наша сеть еще не училась, то она не знает значения этих весов. Поэтому в каждую ячейку положим вес, равный 0.

5. Задаем количество циклов обучения n . Известно, что чем больше уроков посетил ученик, тем лучше он усвоил учебный материал. Похоже и с нейронной сетью — ей нужно преподать достаточно много уроков (в машинном обучении используется довольно странное слово «эпох»).

6. Организуем цикл, состоящий из p уроков.

7. В каждом цикле будем с помощью генератора случайных чисел получать цифру от 0 до 9 и пытаться угадать — пятерка это или нет. Персептрон вернет ответ, два значения: да — угадал (True), нет — не угадал (False).

8. Далее идет разветвление программы: если угаданная цифра - не пятёрка, идем в блок проверки наличия ошибки первого типа, если пятёрка, то идем в блок проверки наличия ошибки второго типа.

9. Проверка наличия ошибки первого типа. Если персептрон ошибся — например, на цифру 3, выданную генератором чисел, указал, что это цифра 5, то фиксируется ошибка первого типа. В этом случае вычитаем единицу из всех связей, связанных с возбуждившимися сенсорами - уменьшаем влияние тех сенсоров, которые возбудились и привели к неправильному выводу (как бы подсказываем ученику, что в этом случае нужно давать отрицательное решение). Если персептрон не ошибся, то происходит возврат к началу цикла.

10. Проверка наличия ошибки второго типа. Если персептрон ошибся— например, на цифру 5, выданную генератором чисел, сказал, что это другая цифра, то фиксируется ошибка второго типа. В этом случае прибавляем единицу ко всем связям, связанным с возбуждившимися сенсорами. Другими словами, мы увеличиваем влияние тех сенсоров, которые возбудились и привели к неправильному выводу. То есть мы как бы подсказываем ученику, что в этом случае нужно давать положительное решение. Если персептрон не ошибся, то происходит возврат к началу цикла.

В конце всех эпох мы сможем увидеть, сколько раз сеть правильно угадывала или ошибалась. Сколько эпох необходимо — отдельный вопрос.

Напишем программу для обучения персептрона на языке Питон.

```
# «Ручная» реализация персептрона
import random
```

```
# Обучающая выборка (идеальное изображение цифр от 0 до 9)
num0 = list('111101101101111')
```

```

num1 = list('001001001001001')
num2 = list('111001111100111')
num3 = list('111001111001111')
num4 = list('101101111001001')
num5 = list('111100111001111')
num6 = list('111100111101111')
num7 = list('111001001001001')
num8 = list('111101111101111')
num9 = list('111101111001111')

# Список всех цифр от 0 до 9 -> единый массив
nums = [num0,num1,num2, num3, num4,num5, num6, num7, num8, num9]

tema = 5 # какой цифре обучаем
n_sensor = 15 # количество сенсоров
weights = [0 for i in range(n_sensor)] # Обнуление весов

# Функция определения, является ли полученное изображение 5
# возвращает Да, если признано, что это 5.
def perceptron(Sensor):
    b = 7 # Порог функции активации
    s = 0 # Начальное значение суммы
    # цикл суммирования сигналов от сенсоров
    for i in range(n_sensor):
        s += int(Sensor[i]) * weights[i]
        if s >= b:
            return True # Сумма превысила порог
        else:
            return False # Сумма меньше порога

# Уменьшение значений весов
# Если сеть ошиблась и ложно выдала Да
def decrease(number):
    for i in range(n_sensor):
        if int(number[i]) == 1: # Если вход возбужден
            weights[i] -= 1 # Уменьшаем связанный с входом вес

# Увеличение значений весов
# Если сеть ошиблась и выдала Нет при поданной на вход цифре 5
def increase(number):
    for i in range(n_sensor):
        if int(number[i]) == 1: # Если вход возбужден
            weights[i] += 1 # Увеличиваем связанный вес

# Тренировка сети
n = 100 # количество эпох
for i in range(n):
    j = random.randint(0, 9) # Генерируем случайное число j
    r = perceptron(nums[j]) # Результат (ответ - Да или НЕТ)

    if j != tema: # Если генератор выдал случайное число j != 5

```

```

    if r: # Если сумматор сказал ДА, а j это не пятерка
        decrease(nums[j]) # Ошибка первого типа

else: # Если генератор выдал случайное число j равное 5
    if not r: # Если сумматор сказал False, а на самом деле j=5
        increase(nums[tema]) # Ошибка второго типа, увеличиваем веса

print(j)

print(weights) # Вывод значений весов

```

Для цифры 7 получим совершенно иные веса важности каждого сенсора. Таким образом, мы можем последовательно подобрать веса сенсорных связей для любой цифры.

Итак, мы обучили сеть распознаванию двух цифр. Можно перейти аналогично к обучению распознаванию остальных.

В данном разделе мы потренировались в «ручном» описании нейрона и нейронной сети на языке Питон. Однако, при решении практических задач в этом нет необходимости. Существует большое число доступных библиотек машинного обучения и реализации разнообразных нейросетей, кои мы можем использовать в Питоне — это прежде всего Keras и PyTorch, но и многие другие, обладающие большими возможностями для решения специфичных задач, например, в области компьютерного зрения.

В следующих разделах книги для построения нейросетей будут применяться библиотеки.

Контрольные вопросы и упражнения главы

1. Кратко опишите историю искусственных нейросетей.
2. Сколько нейронов насчитывает мозг человека?
3. Опишите особенности биологического нейрона.
4. Дайте определение формального (искусственного) нейрона.
5. Обязательна ли аппаратная реализация нейросетей? Выскажите Ваше мнение по поводу программной эмуляции на ЭВМ «фоннеймановской» архитектуры и проблемы данного подхода.
6. Опишите преимущества нейропроцессоров перед нейропакетами.
7. Перечислите основные применяемые виды пороговых функций.
8. Какие бывают основные архитектуры нейросетей?
9. Что такое обучение нейросети с учителем?
10. Как проводят обучение нейросети без учителя?
11. На произвольном языке программирования реализовать и обучить нейросеть для распознавания изображений цифр.
12. На произвольном языке программирования написать программу, моделирующую простейший персептрон.
13. Прокомментировать различия сетей Элмана и Жордана.
14. В чем состоят особенности нейросетей с обратными связями?
15. Сформулируйте теорему Колмогорова – Арнольда.
16. Сформулируйте теорему Хехт-Нильсена и приведите следствия из нее.
17. Сформулируйте теорему о сходимости персептрона.
18. Опишите алгоритм обучения нейросети с обратным распространением ошибки.
19. Опишите дифференциальный метод Кохонена.
20. Что такое сеть Хопфилда?
21. Сформулируйте принципы алгоритма обучения Хебба.

3. ПРИМЕНЕНИЕ СОВРЕМЕННЫХ НЕЙРОСЕТЕЙ

Сверточные сети

Понять по картинке, кто изображен — собака или кошка, долгое время было элементарной задачей для человека, начиная с детсадовского возраста, и почти неподъемной для машины. Если вы себе представите изображения собак, то они могут быть в самых разных позах, при разном освещении, на разном фоне. Их очень сложно классифицировать, непосредственно опираясь на пиксели. А последовательно получая иерархию признаков новых пространств, мы это сделать можем.

В области компьютерного зрения, в распознавании изображений до нейронных сетей исторически был принят следующий подход. Например, нам нужно между определять, к какому из классов принадлежит объект — дом, птица, человек, и т.п.

Люди долго думали, какие именно признаки можно использовать, чтобы отличать эти изображения. Например, изображение дома легко выделить, если присутствует много четких геометрических линий, которые пересекаются. У птиц, например, бывает очень яркий окрас, поэтому если у нас есть сочетание зеленых, красных цветов в изображении — то это похоже на птицу. Простейший подход заключался в том, чтобы придумать как можно больше подобных признаков, а дальше использовать их в достаточно простом линейном классификаторе.

Были и более сложные методы, но, тем не менее, они используют признаки, придуманные человеком. С нейронными сетями можно подготовить подходящую обучающую выборку и не придумывать никакие признаки, а просто подать ИНС на вход изображения - и она сама обучится, сама выделит признаки, важные для классификации данных изображений.

Скачок в решении задач в данной предметной области связан с использованием особой архитектуры, так называемых свёрточных (англ. convolutional) нейросетей. Данные сети были предложены

Яном Лекуном в 1988 году, когда он успешно использовал их для распознавания сканов рукописных цифр.

В распознавании изображений все кардинально изменилось буквально за предыдущее десятилетие. Уровень современных успехов сверточных сетей можно понять по следующему примеру. В Интернете сейчас доступны различные наборы данных, используемых для обучения и проверки качества работы различных алгоритмов распознавания. Есть, в том числе, база изображений ImageNet. В ней насчитывается около миллиона различных изображений, размеченных на тысячу классов.

В 2012 г. Алекс Крижевски и Джефф Хинтон представили на международном конкурсе распознавания изображений из ImageNet свою сверточную нейросеть. По сравнению с аналогичными моделями она совершала почти наполовину меньше ошибок при распознавании изображений: их количество снизилось с 26 до 15%.

Сейчас точность стала еще выше. Например, при распознавании лиц в толпе показатель составляет 99,8% [19].

У сверточных нейросетей выделяют два основных типа слоев — сверточные и подвыборки (англ. pooling).

Сверточный слой - ключевой. Здесь нейросеть выделяет важную информацию на фрагменте изображения. Сверточный слой может выделять разные признаки изображения: контур, текстуру, цвета, и пр. Сверточные слои используют фильтрацию с помощью так называемого ядра свертки. Это заранее заданные матрицы небольшого размера, часто три на три пикселя. Как осуществляется свертка? В центре обрабатываемого фрагмента выделяется «место приложения» свертки. Затем мы «взвешиваем» пиксели в данном квадрате - умножаем их яркость на вес соответствующих элементов ядра, и получаем некоторое значение. Из них формируется выходная матрица слоя, или карта признаков, которая по сути содержит информацию, какие признаки присутствуют на изображении. Вышесказанное можно наблюдать на рисунке 3:1.

Свертка изображения

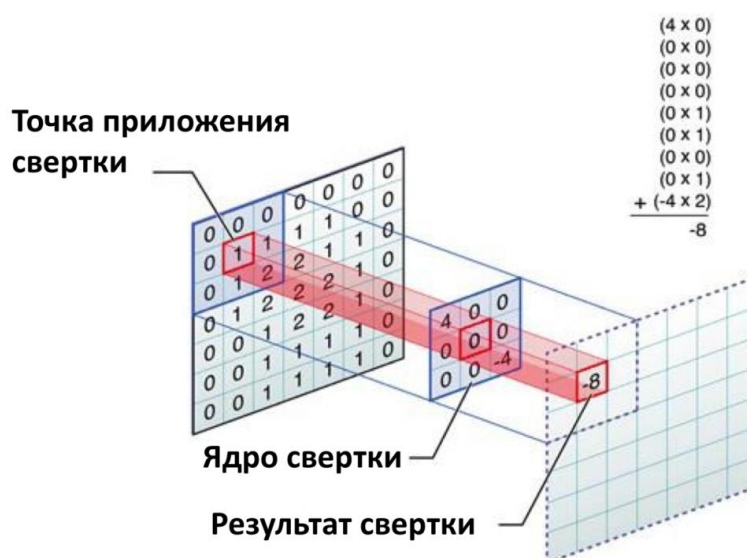


Рисунок 3.1.

Фильтр проходит по изображению, подобно сканеру.

После свертчного идет **слой подвыборки**, позволяющий уменьшить размер карты признаков. Он выбирает самые важные данные и убирает лишнее.

Затем можно снова применить свертчный слой и повторить процесс несколько раз. Так получается постепенно выделять все более сложные признаки изображения. Например, сначала нейросеть найдет контуры цветка, а потом распознает его форму и оттенки лепестков.

На первых слоях оказывается, что сеть выделяет очень простые признаки. Например, переходы градиентов или какие-то линии под разным углом. «Выделяет признаки» означает, что есть нейрон, возбуждающийся, если подобный фрагмент изображения присутствует в окне вокруг ядра свертки. Эти признаки не очень интересные. Но переходя глубже, мы видим, что сеть начинает выделять более сложные признаки, например, как круговые элементы и даже круговые элементы вместе с некими полосками. И чем дальше мы идем по сети, чем дальше от входа, тем более сложными становятся признаки.

Например, некий нейрон уже может реагировать на морду собаки - он возбуждается, когда находит в части изображения собачью морду. Другие нейроны могут выявлять на изображении пивную банку, часы, или еще что-либо.

Фактически «признаки», вырабатываемые сложной сетью, малопонятны и трудны для интерпретации настолько, что в практических системах не особенно рекомендуется пытаться понять содержание этих признаков или пытаться их «подправить». Вместо этого рекомендуется совершенствовать архитектуру сети, чтобы получить лучшие результаты.

Игнорирование ИНС каких-либо существенных характеристик изображений может говорить о том, что либо не хватает данных для обучения, либо структура сети обладает недостатками, и она не может выработать эффективные признаки для данных изображений.

У сверточной архитектуры есть и дополнительные преимущества. Во-первых, меньше весов можно быстрее настроить. Далее, ИНС менее подвержена переобучению. На выходе нейросети обычно используются полносвязные слои, которые используют найденные признаки для классификации или регрессии.

Интересно, что работа сверточной сети напоминает функционирование зрительной коры мозга. В ней присутствуют небольшие участки клеток, чувствительных к конкретным областям поля зрения. Ученые-биологи Хьюбел и Визель в 1962 году открыли, что отдельные мозговые нервные клетки активировались при визуальном восприятии границ определенной ориентации. Например, некоторые нейроны «воспринимали» вертикальные границы, а некоторые — горизонтальные или диагональные. Хьюбел и Визель выяснили, что эти нейроны сосредоточены в стержневой архитектуре и вместе формируют визуальное восприятие.

Обучение сверточных моделей концептуально не отличается от обучения многослойных полносвязных нейросетей. Рассмотрим общую схему:

1.«Движение вперед», или forward

На этом этапе входные векторизованные данные проходят через ряд слоев в сети. Каждый слой применяет свои веса и функции активации. Это нужно для того, чтобы преобразовать данные в более абстрактные представления, близкие к решению задачи. В результате формируется выход сети, то есть конечные предсказания модели.

2. Расчет функции потерь. Для разных типов задач используют разные функции потерь.

3.«Движение назад» (backpropagation). Этот шаг включает корректировку весов сети с использованием метода оптимизации, называемого стохастическим градиентным спуском. Это метод поиска минимума функции потерь. Основная задача — вычислить градиенты — направления и величины изменений для минимизации ошибки функции потерь по отношению к весам и обновить веса.

Обучением сети на практике, как правило, занимаются инженеры по компьютерному зрению, чтобы получать результаты для решения конкретных задач. Часто специалисты используют *предобученные модели*, что помогает значительно сократить время и ресурсы.

Под предобученной моделью понимается нейросеть, которая уже прошла множество эпох обучения на достаточно больших и универсальных наборах данных, в случае сверточных сетей — на библиотеках общих изображений.

Предобученные модели, адаптированные под специфические данные, часто демонстрируют лучшую производительность.

Сферы, где сейчас чаще всего применяют сверточные сети:

1. Медицина. Сверточные нейронные сети анализируют изображения для обнаружения патологий и постановки точного диагноза. Например, они анализируют рентгеновские снимки, результаты МРТ, КТ, УЗИ.

2. Робототехника, транспорт. В беспилотных системах сверточные нейросети обрабатывают визуальные данные.

Обычно это изображение видеопотока. Модели распознают дорожные знаки, полосы движения и другие объекты, чтобы сделать вождение безопасным.

3. Системы распознавания лиц. Сверточные нейронные сети используются для аутентификации в банке, в системах видеонаблюдения и других подобных сферах.

У сверточных нейросетей есть несколько наиболее вероятных направлений будущего развития:

1. Интеграция с другими типами нейросетей. Например, уже сейчас активно применяется комбинация сверточных нейросетей с трансформерами — моделями, изначально созданными для обработки текстов на естественном языке, которые в настоящее время хорошо справляются с переводами, генерацией текстов и отвечают на вопросы. Интеграция помогает обрабатывать многомодальные данные, то есть информацию в виде разных типов данных, например звука, видео, изображений, текста.

2. Оптимизация. Будут разрабатываться более технологичные и эффективные архитектуры и алгоритмы. Они помогут ускорить обучение нейросетей и точность их работы.

3. Расширение сфер использования. Сверточные нейронные сети показывают хорошие результаты не только при работе с изображениями — они эффективны и для других типов неструктурированных данных. Поэтому областей их применения будет становиться больше.

Замечание. Есть сферы, где сверточные нейронные сети применять бесполезно. Так, они хуже решают задачи по обработке последовательных данных или временных рядов.

Перечислим наиболее известные нейросети, имеющие сверточную архитектуру:

➤ ResNet. Сверточная нейросеть, основанная на концепции остаточных соединений — skip connections. Благодаря этому, можно обучать очень глубокие сети с десятками слоев.

➤ VGGNet. Модель с небольшими сверточными фильтрами. Входит в пятёрку лучших по точности при тестировании на наборе данных ImageNet.

➤ EfficientNet. Большая модель, у которой много параметров. Часто демонстрирует хорошие метрики качества, особенно если использовать предобученные веса.

Уровень успехов сверточных сетей можно понять по следующему примеру. В Интернете сейчас доступны различные наборы данных, используемых для обучения и проверки качества работы различных алгоритмов распознавания.

ImageNet

Хаски



Сибирский хаски

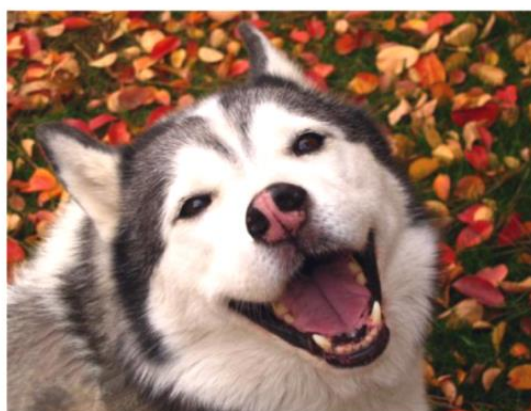
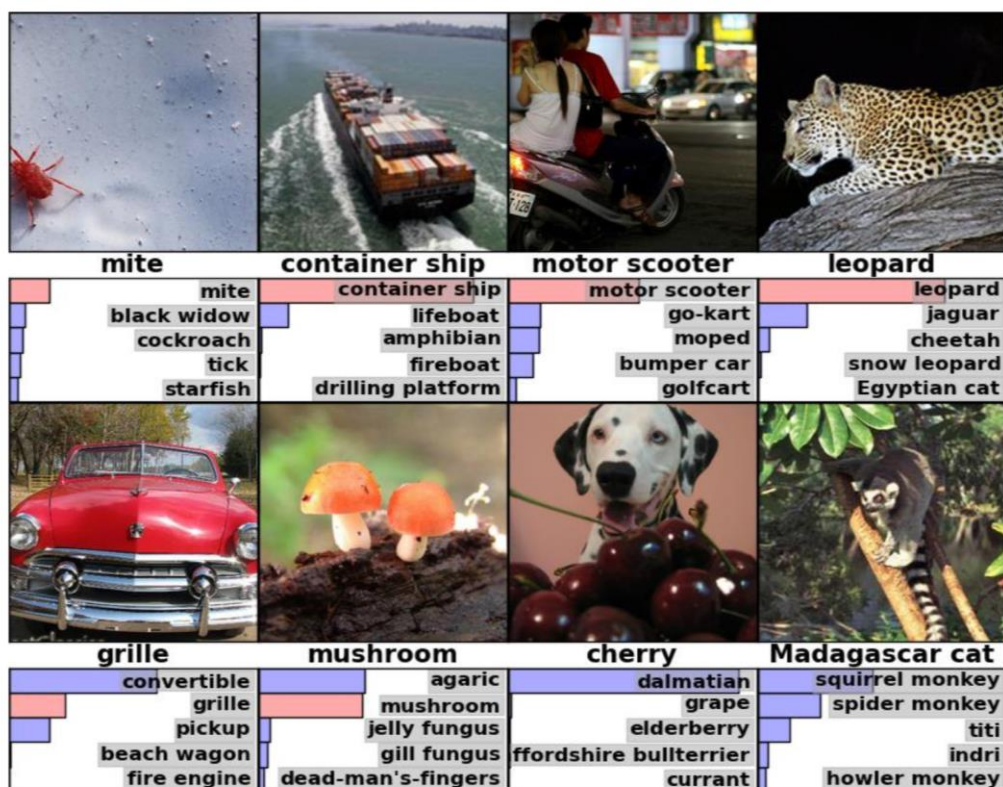


Рис. 3.2. Хаски и сибирский хаски в ImageNet

Есть, в том числе, по современным меркам не очень большая база изображений `Image.net`. В ней насчитывается несколько больше миллиона различных изображений, кои размечены на тысячу классов. Данная база не столь примитивна. Например, в ней собаки пород хаски и сибирский хаски в ней – два разных класса (рис. 3.2). На самом деле в базе больше 300 пород собак разных классов и пара десятков только видов терьеров.

Если показать обычному человеку, не обладающему специфическими знаниями в области кинологии, два изображения, он вряд ли поймет, где хаски, а где сибирский хаски.



Krizhevsky A., Sutskever I., Hinton G.E. <http://www.cs.toronto.edu/~fritz/absps/imagenet.pdf>

Рис. 3.3. Примеры изображений базы Image.net

К этой выборке изображений применили нейронную сеть, и оказалось, что она очень неплохо с ней справляется. Например, на рис. 3.3 показаны варианты изображений, причем указан правильный класс (по-английски) и столбиками – показания сети. Чем больше столбик, тем больше сеть уверена. Мы видим, что она правильно детектирует леопарда, детектирует скутер, несмотря на то, что на изображении помимо скутера еще есть люди. Сеть определяет, что скутер – доминирующий объект на изображении, и называет именно его.

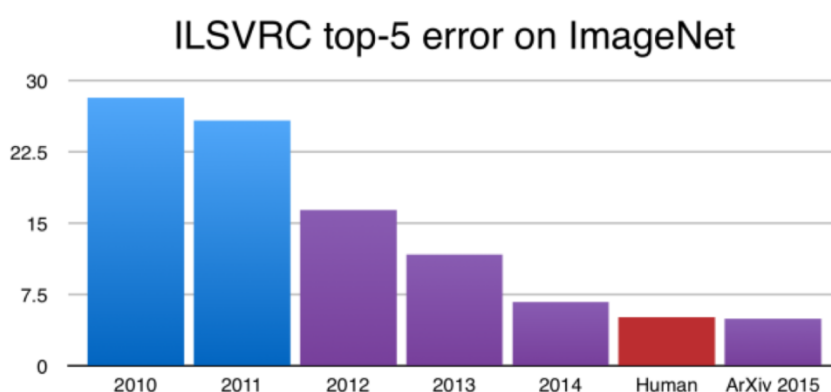
Сеть даже правильно детектирует клеща, несмотря на то, что все изображение состоит просто из однородного фона, и клещ находится в самом углу – «понимает», что клещ является основой этой сцены.

При этом анализ ошибок нейронной сети тоже очень интересен и позволяет выявить, например, ошибки в разметке обучающего множества. Ведь размечали люди, а люди ошибаются (что, кстати, представляет собой особую проблему!). Допустим, есть картинка с далматинцем и вишней, в отношении которой нейронная сеть с огромной уверенностью отвечает, что это далматинец. Мы смотрим

на это изображение и видим, что основной объект – далматинец. Но на переднем фоне здесь есть вишня, и человек, который размечал это изображение в процессе подготовки выборки, решил, что вишня – главная на этом изображении. И получается, что нейронная сеть «ошиблась» вслед за человеком. Почему ошиблась? Она же назвала далматинца. Неясно.

С другой стороны, например, здесь есть некое животное справа внизу (правильный ответ – «мадагаскарская кошка»). Сеть делает очень резонное предположение о том, что это может быть. Она говорит, что это какой-то вид обезьяны. Это – ошибка, но представляющаяся разумной. Человек аналогично вполне мог бы предположить, что это – вид обезьяны, очень малораспространенный и экзотический.

Прорыв в распознавании изображений



NVIDIA blog <http://devblogs.nvidia.com/parallelforall/mocha-jl-deep-learning-julia>

Рис. 3.4. Качество распознавания объектов ImageNet

Основной метрикой ошибки на этой базе является так называемая Топ-5 ошибка. Мы берем пять классификаций на выходе ЭВМ, в которых она наиболее уверена, и если правильный класс из разметки попал в эти пять предсказаний, то мы говорим, что сеть права.

Ошибкой считается, это когда у нас в первые пять предсказаний не попал правильный класс.

Мы видим, что до момента революции сверточных нейронных сетей ошибка уменьшалась, но медленно. Она была чуть ниже 30%.

В 2012 году, когда к этой задаче впервые применили сверточную нейронную сеть, оказалось, что мы можем снизить ошибку радикально. Все остальные методы распознавания были на это неспособны. Дальше по мере роста интереса к нейронным сетям оказалось, что мы можем еще снизить эту ошибку – вплоть до уровня, которого достигает человек на этой базе изображений.

Были проведены исследования по распознаванию подобного рода изображений, на подготовку которых ушло несколько дней, и по их результатам на этой выборке получилась ошибка примерно 4,5%. Нейронная сеть на момент весны или лета 2015 года этот рубеж побила, что всех поразило (рис. 3.4).

Сверточные сети – название целого класса ИНС. Глубина сети и особенности ее архитектуры и функции активации нейронов имеют значение. При этом чем больше мы применяем нелинейности, тем чаще получаем прорыв в качестве распознавания. Например, в 2013 году в конкурсе победила сеть, которая имела восемь или девять слоев последовательного преобразования изображения. А в 2014 году победила сеть, в которой имелось намного больше слоев и архитектура в целом была значительно сложнее.

Есть ли у сверточных ИНС скрытые возможности? Да. Например, так как мы с вами уже знаем, что на каждом слое мы получаем некоторые новые признаки, мы можем взять с какого-нибудь одного из последних слоев нейронной сети выходы и считать, что это – некоторые важные признаки изображения. Далее можно, например, искать похожие изображения на основе этих признаков. Например, есть картинка, где играют в баскетбол, NBA, и мы хотим найти похожие изображения. Используя описанный подход, мы действительно находим то, что необходимо – фотографии других игроков других команд. Но мы видим, что все изображения имеют отношение к игроку, который ведет мяч на площадке.

У нейросетей в обработке изображений есть много разнообразных применений. Некоторые из них весьма интересны.

Например, можно взять достаточно простую картинку (рис. 3.5) и научить сеть стилизации изображения под картины великих художников, таких как Пикассо, Кандинский, Ван Гог и пр. Мы хотим, чтобы, с одной стороны, выходное изображение было похоже на исходное, но, с другой стороны, чтобы в нем присутствовала явная манера этого художника.

Как мы видим, нейросеть достаточно интересным образом преобразует картинки. Читатель сам может легко ознакомиться с подобными примерами в Интернете.

Стилизация изображений



Gatys L.A. et al. <http://arxiv.org/pdf/1508.06576v1.pdf> , 2015

Рис. 3.5. Создание «шедевров» нейросетью

Рекуррентные нейронные сети

Давайте теперь разберемся, что из себя представляет рекуррентная нейронная сеть, или сеть с обратными связями.

Все архитектуры, которые мы разбирали до этого, имели и обрабатывали информацию лишь об одном примере и не существовало никакой связи между этими объектами. Информация об одном объекте никак не влияла на обработку другого.

При обработке текста, аудиосигналов и временных рядов данный принцип не очень приемлем. Тут важно видеть и учитывать всю предысторию. И чем длиннее эта предыстория, тем лучше.

Рекуррентная нейронная сеть на момент времени хранит всю предшествующую информацию о примерах, прошедших через нее. Сразу нужно оговориться, что это не абсолютная память, которая «помнит» все так, как оно было изначально. Чем длиннее была предыдущая последовательность, тем меньше остается информации о самых первых примерах последовательности. В разумных пределах рекуррентная нейронная сеть позволяет учитывать информацию о словах (других объектах), отстоящих друг от друга очень далеко.

Эффект «памяти» достигается за счет рекуррентных блоков (см. рисунок 1).

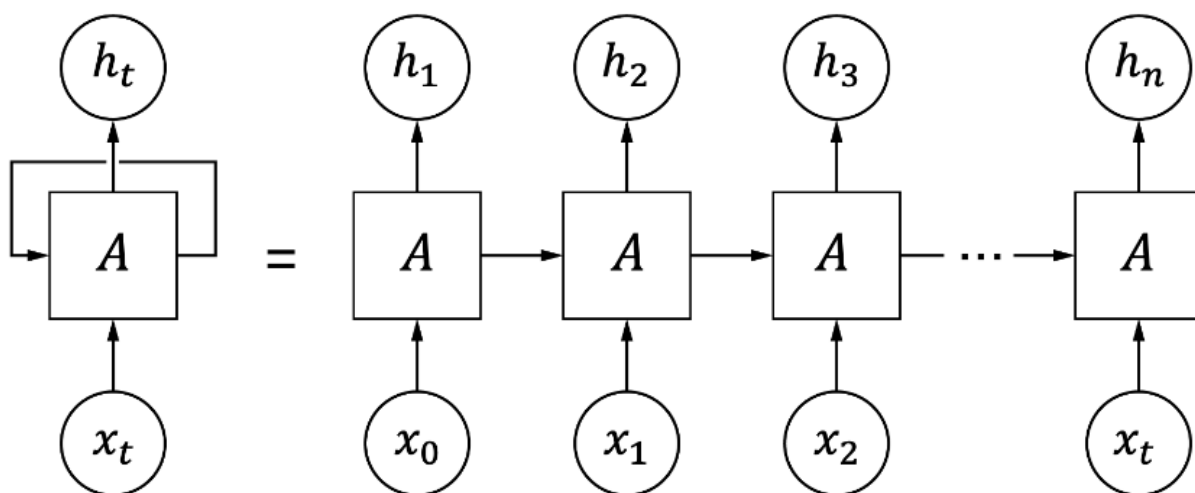


Рис. 1. Рекуррентный блок

Рекуррентный слой имеет обратную связь, он принимает не только входные данные, но и данные от самого себя, обработанные в предыдущий момент времени. По сути, один рекуррентный слой, если развернуть его во времени, – это серия обычных слоев, каждый из которых одновременно принимает на вход значение нового обучающего примера и результат обработки предыдущего слоя. Таким образом, на 1-й слой поступит векторное представление 1-го слова и результат обработки 0-го слова 0-м слоем, а на второй слой попадет векторное представление 2-го слова и результат обработки 0-го и 1-го слов соответствующими слоями. Только информация о 0-м слове пройдет уже через два слоя – 0-й и 1-й, а значит, исказится

больше, чем информация о 1-м слове. Значит, каждый следующий блок будет получать аккумулярованную информацию о всех предыдущих словах с начала последовательности, но информация будет тем более искаженной, чем дальше мы ушли от первого слова последовательности.

Из устройства рекуррентного слоя вытекает еще один дополнительный плюс. Так как слой принимает в один момент времени один элемент обучающей последовательности, то ему не важна длина входной последовательности. Раньше нам всегда нужно было четко задать размер входной последовательности. Полносвязный слой принимал вектор фиксированной длины, сверточные слои принимали матрицу или вектор фиксированного размера. Именно поэтому для одномерной сверточной сети мы подгоняли длину текстовой последовательности к какой-то фиксированной длине. Рекуррентная нейронная сеть позволяет нам не обращать внимания на разницу в длинах различных последовательностей, что является важным преимуществом. В реальности предложения могут составлять как одно слово, так и двадцать.

Но из-за такой последовательной во времени обработки возникают и минусы:

- так как перед обработкой N -го объекта нам нужно обработать все предыдущие, то теряется возможность обсчитывать объекты параллельно. Резко возрастает время обработки и обучения по сравнению со всеми другими моделями;
- из-за большого числа слоев, возникающих за счет рекурсии, мы очень быстро приходим к очень глубокой модели.

Возникает либо проблема взрывного градиента (если веса нейронов больше 1), либо проблема затухающего градиента (если веса нейрона меньше 1). То есть из-за большого количества слоев возникает множественное умножение весов. Если вы попытаете много раз умножать числа больше 1, то увидите, что накопленный результат экспоненциально растет, а если будете умножать числа меньше 1, то увидите, что накопленный результат экспоненциально убывает.

Итак, рекуррентная нейронная сеть имеет несколько серьезных проблем. В частности, мы видим, что ее возможность выявлять зависимости любой длины чисто теоретическая. На практике из-за невозможности бесконечного увеличения количества слоев мы довольно сильно ограничиваем длину «памяти» нейронной сети. Но в связи с тем, что она очень хорошо подходит для задач машинного перевода и генерации текстовых последовательностей, исследователями были созданы улучшенные модификации, которые пытаются решить эту проблему, и небезуспешно. Поэтому рекуррентные нейронные сети в классическом их представлении практически не используются в реальных задачах.

Следующая нейросеть носит парадоксальное название: «Долгая краткосрочная память» (англ. Long short-term memory; LSTM). Она была предложена в 1997 году Шмидхубером и Хохрайтером и относится к классу сетей с обратными связями, или рекуррентных ИНС.

Люди не перезапускают мыслительный процесс с нуля в каждый момент времени. Читая статью, вы понимаете смысл каждого слова на основе значений предыдущих слов. Мысли имеют свойство «накапливаться», влияя друг на друга. Этот принцип используется в сетях LSTM. Нейронные сети без обратных связей не могут этого сделать, и это – серьезнейший недостаток.

Представьте, что вы хотите в реальном времени классифицировать события, происходящие в кинофильме. Неясно, как обычная нейронная сеть может использовать знания о предыдущих событиях, изучая последующие.

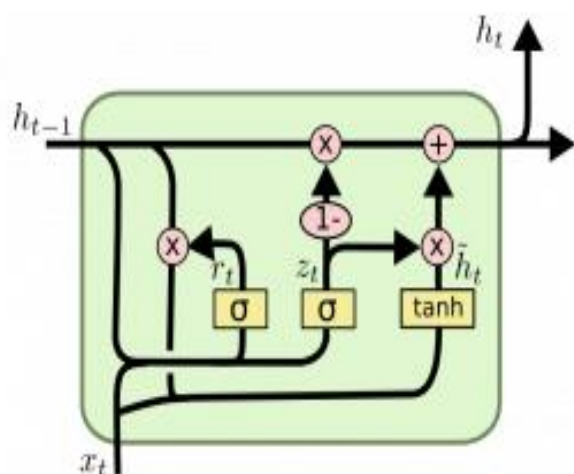
Рекуррентная нейронная сеть используют полученную ранее информацию для решения последующих задач, например, уже полученные фрагменты для анализа последующих.

Иногда для решения задачи требуется просмотреть только непосредственно предшествующую информацию. Например, можно построить лингвистическую модель, которая пытается предсказать следующее слово, основываясь на предыдущих. Если мы пытаемся предсказать последнее слово в словосочетании «облака в небе», нам не нужен дополнительный контекст – очевидно, что следующее слово будет «небе».

Но иногда требуется больше контекста. Рассмотрим попытку предсказать последнее слово в тексте «Я вырос во Франции ... Я свободно говорю на французском». Из предыдущих слов понятно, что следующим словом, вероятно, будет название языка, но если мы хотим назвать правильный язык, нужно учесть упоминание Франции. Разрыв между необходимой для учета информацией и точкой, в которой она нужна, может быть весьма значительным.

К сожалению, по мере увеличения этого разрыва «обычные» рекуррентные ИНС «теряют нить» связи.

LSTM (long short-term memory, дословно (долгая краткосрочная память) – тип рекуррентной нейронной сети, способный обучаться долгосрочным зависимостям. LSTM-сеть – это искусственная нейронная сеть, содержащая так называемые LSTM-модули. LSTM-модуль – это рекуррентный модуль сети, способный запоминать значения как на короткие, так и на длинные промежутки времени.



$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

Рис. 3.7. Устройство LSTM-модуля

Как показано на рис. 13.12, где справа представлены также формулы, по которым обрабатывается информация, LSTM-блоки содержат три или четыре «вентиля», которые используются для контроля потоков информации на входах и на выходах памяти данных блоков. Эти вентили реализованы в виде гиперболического тангенса для вычисления значения в диапазоне $[0;1]$. Умножение на это значение используется для частичного допуска или запрещения потока информации внутрь и наружу памяти. Например, «входной вентиль» контролирует меру вхождения нового значения в память, а

«вентиль забывания» контролирует меру сохранения значения в памяти. «Выходной вентиль» контролирует меру того, в какой степени значение, находящееся в памяти, используется при расчёте выходной функции активации для блока.

Имеются успешные примеры применения LSTM в робототехнике для анализа временных рядов, для распознавания речи, для распознавания человеческой активности, для генерации музыкальных композиций, в ритмическом обучении, в задачах распознавания рукописного ввода, в задаче выявления гомологичных белков.

При всех своих преимуществах ячейки LSTM являются достаточно сложными в расчетах. Поэтому были предприняты попытки создать архитектуру, обладающую всеми преимуществами LSTM, но несколько более простую с точки зрения вычислительных мощностей.

Это сети на основе GRU. *GRU (gated recurrent unit)* – закрытый рекуррентный блок (см. рисунок 3.8).

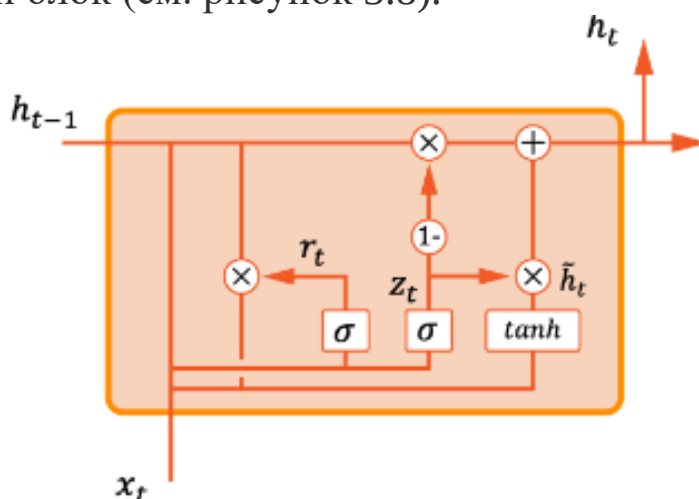


Рис. 3.8. Устройство ячейки GRU

Архитектура GRU весьма похожа по своей сути на LSTM, но в ней меньше вычислений. У каждой ячейки существует два входа: вектор текущего слова x_t и состояние прошлой ячейки h_{t-1} , и один выход — новое состояние h_t . Преобразования данных происходят по следующей схеме:

1. Вентиль обновления r_t заменяет вентиль «забвения» и вентиль входных данных LSTM, он образуется применением слоя с сигмоидальной функцией активации, к вектору входного слоя.

2. Затем r_t умножается на вектор входного слова x_t , и всё это конкатенируется с выходным состоянием прошлой ячейки h_{t-1} и входным вектором.

3. Выходной вентиль z_t получается применением слоя с сигмоидальной функцией активации к входному вектору слова x_t .

4. Промежуточное текущее состояние получается при применении слоя с функцией активации гиперболический тангенс к вектору входных данных.

5. Перемножаем результаты пункта 2 и $(-1) \times z_t$ (выходной вентиль).

6. Перемножаем z_t на промежуточное состояние ячейки h_t .

7. Складываем результаты операций пунктов 5 и 6. Получаем выходное состояние ячейки, которое отправляется на выход, плюс передается следующей ячейке h_t .

Данный подход работает быстрее LSTM, поэтому применяется достаточно часто.

Архитектура «трансформер»

Как мы увидели в предыдущем разделе, LSTM по-разному модифицировали, чтобы лучше отличать важный контекст от неважного. Даже придумали архитектуру GRU. В GRU тоже есть функции, определяющие, какую часть кодированного контекста передавать по цепочке дальше. Однако, все равно при этом остаются проблемы.

Проблема 1: рекуррентная сеть работает медленно

У всех нейросетей, что передают данные с шага на шаг, есть фундаментальные проблемы. Первая — если данные передаются по цепочке, значит, следующие вычисления не начнутся, пока не закончатся прошлые. Причина — следующий шаг принимает результат предыдущего как переменную.

Ждать, пока закончится старая операция, чтобы начать новую — долго. Компьютерное «железо», особенно графические процессоры, хорошо разбивает крупные счетные задачи на мелкие параллельные,

и выполняет их вместе быстрее, чем по очереди. Иными словами, мы не можем загрузить компьютер на полную мощность.

В 2017 проблему решили исследователи из Google Brain и Google Research. В статье «Attention is all you need» («Внимание — все, что вам нужно»), они описали новый тип нейросетей — «трансформер».

Трансформер — не рекуррентная нейросеть. Он не передает весь контекст по цепочке и не выбирает из него нужные части. Зато при каждом новом предсказании трансформер фокусирует внимание только на тех словах, кои считает важными.

Замечание: механизм внимание по сути — умножение вектора слова на числа. Если слово важное — множители будут большими и вектор слова «выделится» среди соседей.

Изначально эта архитектура была предложена для решения задачи машинного перевода. При машинном переводе сначала нужно «декодировать» оригинальный текст, затем — провести генерацию перевода.

Ключевая идея здесь это механизм «само-внимания» (self-attention), позволяющая учитывать контекст, что, безусловно, весьма важно при переводе с одного языка на другой. Механизм внимания, поддерживается слоем сети, который позволяет каждому входному вектору слова взаимодействовать с векторами других слов

Трансформер — архитектура, основанная на сочетании так называемых кодировщиков (*encoder*) и декодирующих (*decoder*) частях. Кодировщик может состоять как из рекуррентных, так и из сверточных слоев, декодер чаще всего состоит из первых.

При машинном переводе декодер начинает слово за словом генерирует выход, при этом он использует уже сгенерированное до того начало текста. После этого он забирает информацию из кодировщика. Выход с последнего декодера отправляется в последний линейный слой с применением функции активации *softmax*.

В целом архитектура достаточно непростая, представление о ней можно почерпнуть по нижеследующему рисунку.

Весьма известная архитектура для решения задач обработки естественного языка - BERT (Bidirectional Encoder Representations from Transformers). Она основана на идее трансформера и идее обучения с подкреплением. Обучаясь, BERT решает задачу предсказания части «замаскированных» слов, что означает, что мы запрещаем смотреть «вперёд» по тексту. BERT по сути произвел революцию в сфере обработки ЕЯ. Ученые до этого, как правило, создавали под каждую конкретную задачу свою нейросеть.

BERT имеет несколько кодирующих слоев (12 или 24), построенных по архитектуре «трансформер». BERT умеет предсказывать следующее слово в тексте — по сути, это давно известный «сервис T9» на смартфонах, но «на стероидах»!

В случае BERT применяется предварительное обучение нейросети. Что это такое?

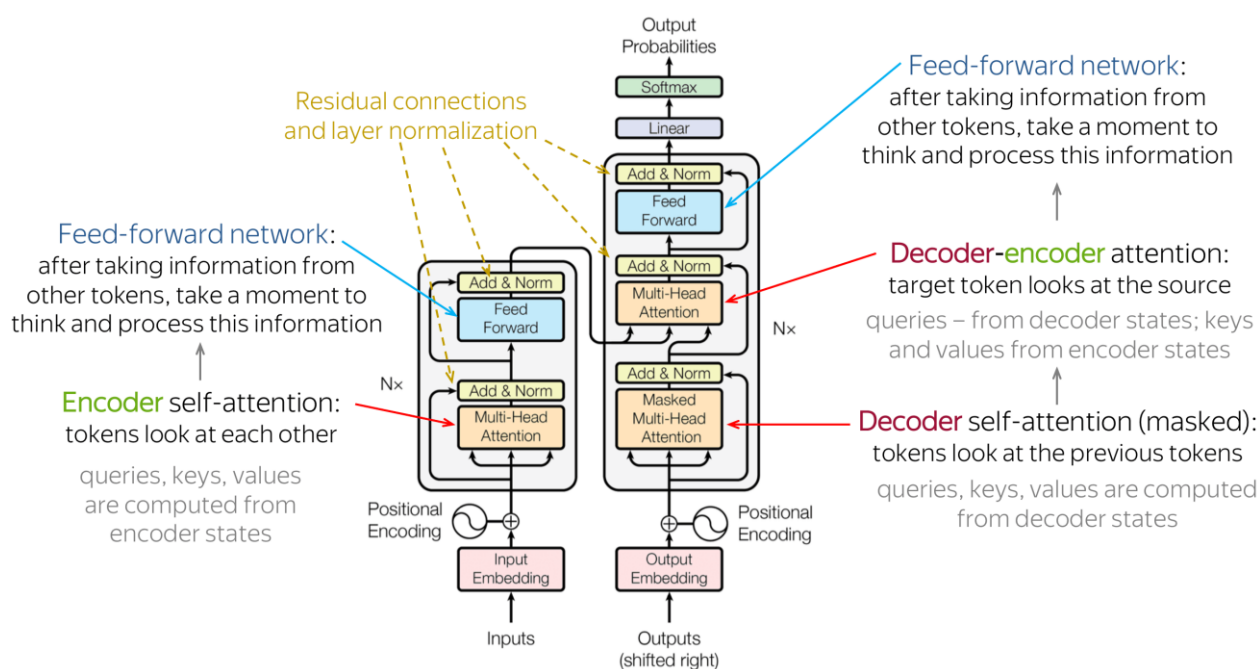


Рис. 3.9. Архитектура «трансформер»

В современных нейросетях, использующих архитектуру «трансформер», часто применяется парадигма дообучения (*англ. fine-tuning*) предварительно обученной (*pretrained*) модели.

Замечание. Знаменитая аббревиатура GPT расшифровывается как Generative Pretrained Transformer).

Предварительное обучение — весьма затратный и долгий процесс, хорошая новость при этом состоит в том, что его достаточно выполнить один раз. Бывает, что богатая корпорация, затратив долгое время, обучает нейросеть, а затем выкладывает ее веса в широкий доступ, что дает возможность ее использовать путем дообучения на своих данных, что вполне по силам и небольшой компании, и даже отдельно взятому пользователю.

Оригинальный BERT обучался на английских текстах, но в настоящее время есть варианты для многих языков. Вариант, обученный на русских корпусах текстов, построен SberDevices, и доступен для общего пользования.

Успехи обучения с подкреплением

Представим себе, что есть мышка и лабиринт. В определенных местах этого лабиринта находятся сыр, вода, морковка, но есть и пустые тупики. Каждый из перечисленных предметов обладает некоторой полезностью для нашей мышки. Например, мышка любит сыр, меньше любит морковку и воду. Если мышь придет к выходу из лабиринта, не найдя ничего, она будет очень расстроена. Пусть у нас в этом простом лабиринте есть всего три состояния, в которых может находиться мышка: S_1 , S_2 , S_3 . В состоянии S_1 она может сделать выбор — пойти налево или пойти направо. И в состоянии S_2 и S_3 она тоже может сделать выбор — пойти налево или пойти направо.

Мы можем задать такое чудесное правило, которое говорит нам, что если мышка два раза пойдет налево, она получит некоторую ценность, равную четырем условным единицам. Если она пойдет направо, а потом уже неважно, налево или направо, после того как первый раз она сходила направо, получит ценность, равную двум.

Задача обучения с подкреплением в этом простейшем случае состоит в построении такой функции Q , которая для каждого состояния S в нашем лабиринте или в какой-то иной среде будет говорить: «Если ты совершишь это действие, – например, пойдешь налево, – сможешь получить такую-то награду».

Нам важно, чтобы в состоянии S_1 , когда мы еще не знаем, что будет впереди, эта функция говорила, что «Если пойдешь налево в этом состоянии – сможешь получить награду 4». Хотя, если мы из S_1 перейдем в S_2 , то можем получить и награду 0. Но максимальная награда, которую мы можем получить при правильной стратегии поведения, – это 4.

Представим себе, что для некоторого из состояний мы знаем для всех следующих состояний, в которые можно попасть из этого, оптимальную награду, т.е. мы можем найти оптимальную методику поведения. Тогда для любого текущего состояния мы легко можем сказать, какое действие нам надо совершать. Можем просто просмотреть все следующие состояния, сказать, в каком из следующих состояний мы приобретаем наибольшую награду – представим себе, что мы ее знаем, хотя это может быть не так на самом деле, – и сделать вывод, что нужно идти в то состояние, в котором ожидаемая награда больше.

Представим себе, что в каком-то состоянии среды, например, лабиринта мы находимся в определенном месте в этом состоянии и мы хотим найти эту функцию $Q(S, up)$, то есть найти, сколько мы сможем приобрести «полезности», если пойдём наверх. И, допустим, в текущий момент времени значение функции равнялось трем. А в состоянии, в которое мы переходим после того, как вышли изданного? Если мы возьмем $\max_{a'} Q(S', a')$, – максимум, сколько мы можем потенциально получить, например, 5, мы пытаемся сделать следующее. Берем разницу между $Q(S, up)=3$ и $\max_{a'} Q(S', a')=5$ и на следующей итерации увеличиваем ценность данного действия. Подобным образом итеративно сеть «обучается».

Итак, в обучении с подкреплением ключевая сущность - агент, взаимодействующий с средой, предпринимая действия. Окружающая

среда дает «награду» (reward) за эти действия — бывает и со знаком минус! - а агент продолжает их предпринимать.

Алгоритмы в таких задачах стремятся найти стратегию, сопоставляющую состояниям среды действия, одно из которых может выбрать агент в этих состояниях.

Среда обычно формулируется как марковский процесс принятия решений с конечным множеством состояний. Такая формулировка позволяет нам упростить сложную задачу из множества последовательных эпизодов к случаю, когда каждое новое принятие решения зависит только от текущего состояния. Таким образом, задача специалиста сводится к заданию среды, в коей будет соблюдаться свойство зависимости агента и среды только от текущего состояния (оно же «марковость»), и выбору подходящего алгоритма.

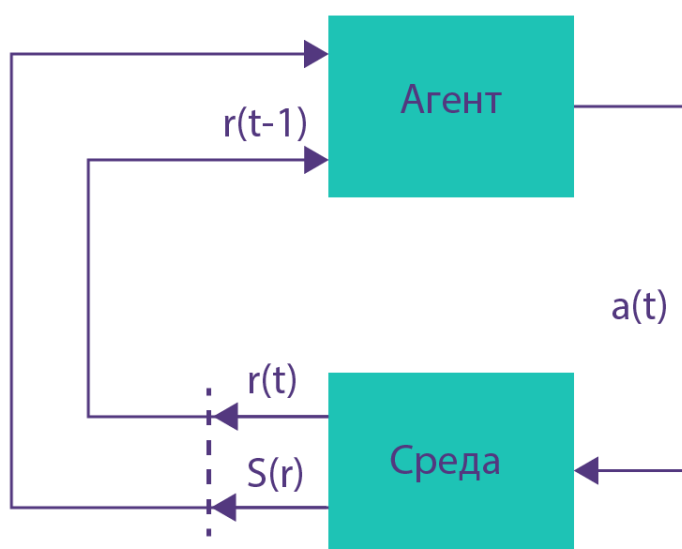


Рис. 3.10. Взаимодействие агента со средой

В каждый момент времени $\square$ агент характеризуется состоянием $s_t \in S$ и множеством возможных действий $A(s_t)$. Выбирая действие $a \in A(s_t)$, он переходит в состояние s_{t+1} и получает выигрыш r_t .

Основываясь на подобном взаимодействии со средой, агент должен выработать стратегию $\pi : S \rightarrow A$, которая по состоянию

среды выбирает действие. Обычно стратегия должна максимизировать величину суммы наград для нее за T следующих раундов $R = r_0 + r_1 + \dots + r_T$ или величину «дисконтированной» награды:

$$R = \sum_{t=0}^{\infty} \gamma^t r_t$$
 где $0 \leq \gamma \leq 1$ — дисконтирующий множитель для «предстоящего выигрыша». Его выбор позволяет балансировать между выгодой «в моменте» и стратегически.

Хороший пример обучения с подкреплением - задача о «многоруком бандите». У нас есть автомат, в коем можно выиграть деньги. На каждом шаге мы выбираем, за какую из N ручек автомата дернуть, то есть в данном случае множество возможных действий определяется номером $A = 1, 2, \dots, N$.

Выбор действия a_t на шаге t влечет награду $R(a_t)$, при этом величина $R(a)$ для любого действия $a \in A$ — случайная величина, распределение которой неизвестно. Задача агента, таким образом, — понять, за какие ручки дергать выгоднее всего, чтобы в среднем выигрыш был выше.

Состояние среды у нас от шага к шагу не меняется, а значит, множество состояний S тривиально, ни на что не влияет, поэтому его можно проигнорировать.

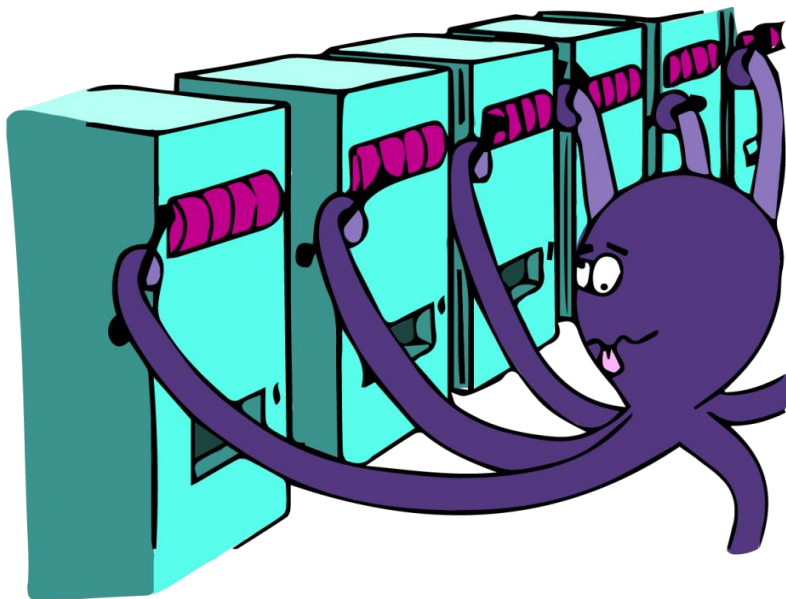


Рис. 3.11. Задача о «многоруком бандите»

Для простоты будем полагать, что каждому действию соответствует некоторое распределение, которое не меняется со временем. Если бы мы знали эти распределения, то очевидная стратегия заключалась бы в том, чтобы подсчитать математическое ожидание (среднее) для каждого, выбрать действие с максимальным математическим ожиданием и теперь совершать это действие на каждом шаге.

Проблема в том, что эти характеристики неизвестны, а агенту необходимо выбрать стратегию для максимизации выигрыша. Этот пример показателен для понимания проблемы исследования среды и эксплуатации стратегии (exploration-exploitation problem). Суть в нахождении стратегии, которая выдерживает компромисс между выбором действий, максимизирующих мгновенную награду, и выбором новых или менее очевидных шагов (которые потенциально могут принести награду еще большего размера).

После определения функции выигрыша R , нужно определить алгоритм, который будет использоваться для нахождения стратегии, дающей наилучший результат.

«Наивный» подход здесь подразумевает следующие шаги:

- Опробовать все возможные стратегии
- Выбрать стратегию с наибольшим ожидаемым выигрышем

Первая проблема такого подхода заключается в том, что количество доступных стратегий может быть очень велико или бесконечно. Вторая проблема возникает, если выигрыши стохастические (случайные): чтобы точно оценить выигрыш от каждой стратегии, потребуется многократно применить каждую из них. Этих проблем можно избежать, если допустить некоторую структуризацию и, возможно, позволить результатам, полученным от одной стратегии, влиять на оценку для другой. Двумя основными подходами для реализации этих мыслей являются оценка функций полезности и прямая оптимизация стратегий.

Подход с использованием функции полезности использует множество оценок ожидаемого выигрыша только для одной стратегии π (либо текущей, либо оптимальной). При этом пытаются

оценить либо ожидаемый выигрыш, начиная с состояния s , при дальнейшем следовании стратегии π ,

$$V(s) = \mathbb{E}_\pi[R|s],$$

либо ожидаемый выигрыш при принятии решения a в состоянии s и последующем соблюдении π ,

$$Q(s, a) = \mathbb{E}_\pi[R|s, a].$$

Если для выбора оптимальной стратегии используется функция полезности Q , то оптимальные действия всегда можно выбрать как действия, максимизирующие полезность.

Имея фиксированную стратегию π , оценить выигрыш при $\gamma = 1$ можно, просто усреднив непосредственные выигрыши. Наиболее очевидный способ оценки при $\gamma \in (0, 1)$ — усреднить суммарный выигрыш после каждого состояния. Но для этого требуется, чтобы марковский процесс принятия решений достиг терминального состояния (завершился).

Поэтому построение искомой оценки при $\gamma \in (0, 1)$ не очевидно. Однако можно заметить, что R образует рекурсивное уравнение Беллмана:

$$\mathbb{E}[R|s_t] = r_t + \gamma \mathbb{E}[R|s_{t+1}].$$

Подставляя имеющиеся оценки V и применяя метод градиентного спуска с квадратичной функцией ошибок, мы приходим к алгоритму обучения с временными воздействиями (temporal difference learning). В простейшем случае и состояния, и действия дискретны, и мы получаем табличные оценки для каждого состояния. Во многих реальных задачах пространство состояний огромно, даже в шахматах их больше, чем атомов во Вселенной.

Как мы уже обсудили, обучение с подкреплением закономерно использовать в ситуациях, когда алгоритм активно взаимодействует со средой, опираясь при этом на собственные действия. Игры — совершенно закономерный пример использования обучения с подкреплением: агентом здесь выступает игрок, средой — информация о состоянии игрового процесса. Например, для шахмат средой выступает положение фигур на шахматной доске, для компьютерной игры — изображение игрового поля, количество здоровья игрока и так далее. На данный момент уже разработаны

алгоритмы на основе RL, которые превосходят человека во многих играх: как настольных, так и компьютерных.

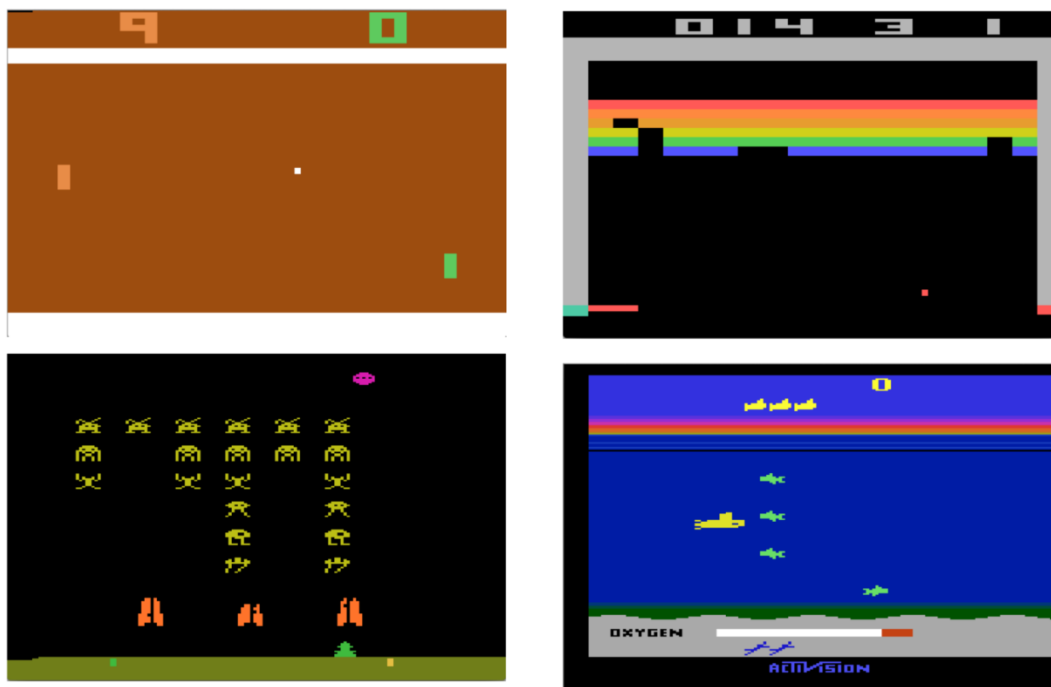
Существуют компьютерные игры-аркады, бывшие в свое время весьма популярными, например, на ЭВМ фирмы [Atari](#) (рис. 3.12). Одна из самых известных игр на рис. 3.14 представлена слева внизу. Она называется «космические захватчики». У нас внизу космический корабль с пушкой, а сверху надвигается группа инопланетных захватчиков, и мы должны всех желтых захватчиков уничтожить и, таким образом, выиграть.

Есть «Пинг-понг», когда нам надо пытаться не упустить мяч у своей половины и сделать так, чтобы соперник упустил мяч на своей половине,

и

т.п.

Atari 2600 games



Mnih V. et al. Human-level control through deep reinforcement learning. *Nature*, 2015

Рис. 3.12. Игры компьютера Атари

Atari 2600 games



Рис. 3.14. Игра «космические захватчики»

Оказывается, мы можем опять взять сверточные нейронные сети, подать на вход изображение игры и на выходе попробовать получить не класс этого изображения, а действие, необходимое в данной игровой ситуации, как нам лучше поступить: уйти налево, выстрелить, ничего не делать и так далее (иными словами, «на какую клавишу нажать»).

Если мы построим достаточно большую сеть, запустим ее в этой среде – а надо понимать, что у нас есть счет, и этот счет как раз и является «наградой», мы получим систему, которая может даже «играть» лучше человека.

На данный момент нейросети играют во многие игры на уровне, значительно превышающем лучших людей-чемпионов, которые играют в эту игру. Но на самом деле, чтобы добиться такого прорыва, потребовалось достаточно много работы. Это действительно впечатляюще.

До применения нейронных сетей для решения этой задачи использовали множество методов, которые создавались специально под каждую из этих игр, и они хорошо работали. Они тоже иногда

обыгрывали человека, и т.д. Но, как и с областью распознавания изображений, людям приходилось очень много работать, чтобы придумать те или иные выигрышные стратегии в игре.

Брали каждую игру в отдельности, например, Space Invaders или «Пинг-понг», долго думали, как нужно двигаться, придумывали какие-то признаки, например, если мяч летит под заданным углом и с заданной скоростью, мы должны развить необходимую скорость, чтобы отбить его на противоположной стороне – и чтобы он еще отскочил неудобно для противника. И так далее.

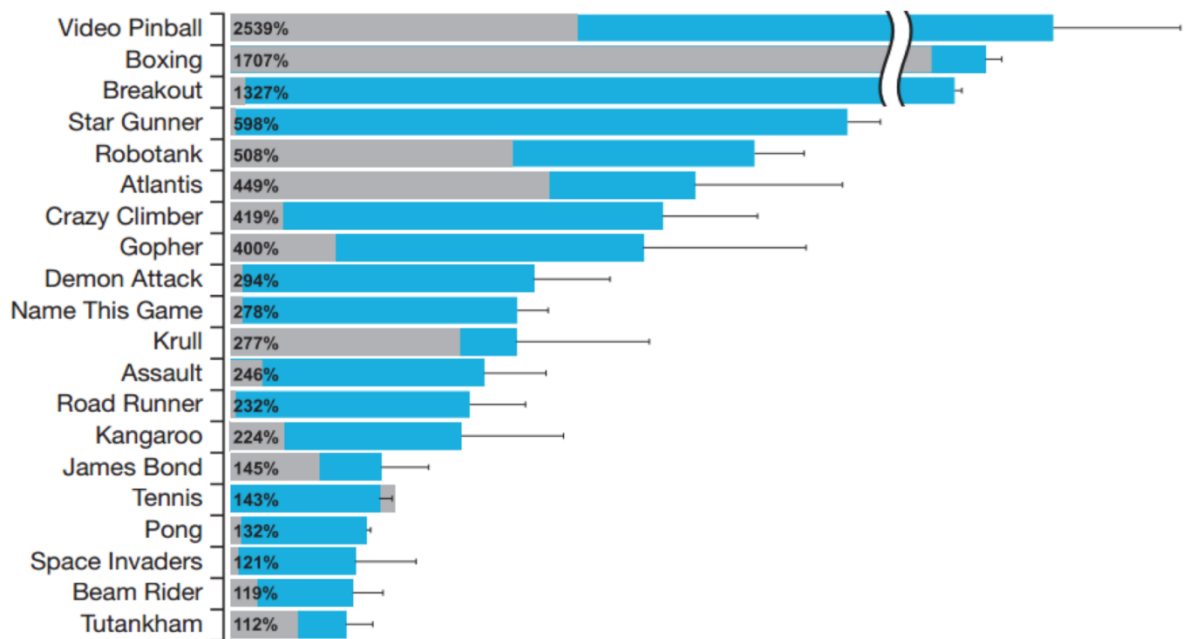
А сейчас взяли одну и ту же архитектуру нейронной сети, применили ее к разным играм, и на всех этих играх эта одна архитектура, правда с разными весами, играла лучше большинства методов, которые придумывались на протяжении 10–20 лет.

На рис. 14.15 представлены удивительные достижения нейросетей в данной задаче. Здесь синее – какое количество очко набирает нейронная сеть, серое – это лучшие результаты ЭВМ до нейронных сетей, а проценты – на сколько процентов нейронная сеть играет лучше, нежели человек. Можем видеть, например, что в «Пинбол» нейронная сеть играет **на 2500 % лучше**. В игру «Бокс», в самые разные другие игры – гонки, теннис, Space Invaders нейронная сеть играет лучше человека.

Надо понимать, что Space Invaders – достаточно сложная игра, но и в ней удалось обойти человека.

Сейчас на большинстве игр Atari сеть выигрывает у человека. Конечно, есть очень сложные для нейросети игры, где нужно обладать памятью. В данной ситуации можно применить рекуррентные сети, и тогда результаты значительно улучшатся.

Atari 2600 games



Mnih V. et al. Human-level control through deep reinforcement learning. *Nature*, 2015

Рис. 3.15. Сравнение качества игры человека и нейросети

Ещё одна область применения обучения с подкреплением — рекомендательные системы. Цель рекомендательных систем — советовать пользователям что-то такое, что они с достаточно высокой вероятностью купят, или такое, что их заинтересует и увлечет. Количество возможных продуктов, которое может порекомендовать система, весьма велико. Поэтому нужен алгоритм, кой не только подберет то, что сильнее всего понравится пользователю, но и обеспечит то, что данный выбор приведет в долгосрочной перспективе к улучшению показателей бизнеса, важных для компании. Многие платформы, такие как YouTube, Netflix, Alibaba, активно применяют RL в своих рекомендациях.

Управление беспилотными транспортными средствами — естественная сфера применения изучаемого подхода. В наши дни обучение с подкреплением используется в Tesla и других компаниях для планирования маршрутов беспилотных транспортных средств и для управления этими транспортными средствами. Один из примеров

— использование для планирования пути при парковке автомобиля. Количество маршрутов при этом — показатель, для которого характерен экспоненциальный рост. При этом традиционные методы планирования маршрутов - медленные и неэффективные, а подходы из обучения с подкреплением оказываются лучше и быстрее находят правильную стратегию.

Тенденции развития больших языковых моделей

Революционные изменения в сфере искусственного интеллекта, начиная с конца 2022 года, связаны прежде всего с большими языковыми моделями (LLM – Large Language Models).

Говоря о работе с естественным языком, можно выделить анализ смысла текстов и автоматическое реферирование, распознавание и синтез речи, речевых помощников, и пр. Однако, одним из самых ранних и классических направлений ИИ является машинный перевод.

Еще 7 января 1954 года в США был проведен знаменитый Джорджтаунский эксперимент. В Нью-Йорке в штаб-квартире корпорации IBM прошла демонстрация возможностей машинного перевода, в ходе коей был показан полностью автоматический перевод более чем 60 предложений с русского языка на английский (первые системы машинного перевода были ориентированы на пару «английский-русский», что неудивительно для эпохи противостояния двух сверхдержав) [11]. Эксперимент был задуман и подготовлен с целью привлечения внимания общества и правительственных структур. В основе машинного перевода лежала довольно простая система, основанная на использовании всего шести грамматических правил; словарь включал 250 записей. Система была специализированной: в качестве предметной области была выбрана органическая химия. Программа выполнялась на мейнфрейме IBM 701. В компьютер в торжественной обстановке на перфокартах вводились предложения вроде: «Обработка повышает качество нефти», «Командир получает сведения по телеграфу», — и машина

выводила их перевод. Демонстрация была широко освещена в СМИ и воспринята как успех. Она повлияла на решение правительств некоторых государств, в первую очередь США, направить инвестиции в область вычислительной лингвистики.

В том же году первый эксперимент по машинному переводу был произведён в СССР. Организаторы Джорджтаунского эксперимента уверяли, что в течение пяти лет проблема машинного перевода будет решена. Однако в действительности всё оказалось сложнее, и в 1966 году, на основании отчёта комитета ALPAC, — который подытожил, что более чем 10 лет исследований не дали законченного результата, — финансирование было значительно урезано.

Терри Виноград [21] приводит примеры фраз: «Человек стоял за забором в зеленой шляпе» - возникает вопрос для машины, человек в шляпе, или забор в шляпе? Человек-переводчик за счет понимания, что заборы не носят шляп, осуществит правильный перевод. Для машины это может оказаться проблемой.

Как понимать совет «не покупайте этот лук», зависит от того, вопрос задан в овощном магазине или в магазине спорттоваров. «Джек видел их семью своими глазами» - у Джека семь (числительное) глаз (Джек — мутант после атомной войны или раненый паук?), или Джек видел нескольких родственников?

В этой связи встал ребром вопрос, как достичь эффективного и точного машинного перевода.

Анализируя данную проблематику, можно обратить внимание на глубокую связь языка и мышления. Алан Тьюринг, выдающийся английский математик, один из основоположников информатики, в честь которого названа и высшая премия в данной сфере науки, создатель первых ЭВМ в Великобритании, даже предложил в качестве критерия «сильного», не уступающего человеческому, искусственного интеллекта, знаменитую «игру в имитацию». По мнению британского ученого, машина, способная неотличимо от человека вести разговор на произвольные темы, способна мыслить.

Выдающиеся философы прошлого постоянно подчеркивали неделимую связь языка с процессами мышления. Виднейший

философ XVIII в. Иммануил Кант писал: «Мыслить – это значит говорить с самим собой, значит внутренне (через репродуктивное воображение) слышать самого себя» [8]. Многие исследователи уверены, что язык с самого начала выполняет по существу ничем не заменимую функцию в обобщающей работе мышления и именно с помощью языка человек оказался в состоянии перейти от познания единичных предметов и явлений к их обобщенному отражению в форме понятий.

Многие полиглоты отмечали, что в зависимости от того, на каком языке они думают в данный момент, они формулируют идеи по-разному, идут к выводам несколько разными путями и даже получают различные умозаключения. Дуглас Хофштадтер написал: «Я обнаружил, что, когда я «думаю по-французски» мне в голову приходят совсем иные мысли, чем когда я «думаю по-английски»! Мне захотелось понять, что же главнее, язык или мысли?» [9]. Карлу V Габсбургу, императору Священной Римской империи, приписывают высказывание: «Если бы я хотел говорить с мужчинами, я говорил бы по-французски, если бы я хотел говорить с женщинами, я использовал бы итальянский, если бы я хотел говорить с моей лошадью, я бы говорил на немецком, если бы я хотел говорить с Богом, я бы говорил по-испански».

Физики, математики, химики, ботаники и даже астрологи для описания своих задач и методов их решения используют специфичные языки, удобные в каждом конкретном случае. Жан-Луи Лорьер отмечает [8], что привычный всем со школы современный язык математики, кажущийся столь логичным и естественным, родился в муках на протяжении тысячелетий. В современном виде он существует совсем недавно, причем в некоторых разделах науки (скажем, в математической логике, где можно для обозначения одного и того же действия встретить запись и $\rightarrow$, и $\square$, и $\square$) до сих пор не существует единой общепринятой системы обозначений! Вычисления в Древнем Египте или Вавилоне были гораздо более громоздкими и трудоемкими просто в силу используемых форм

записи. Для примера Лорьер приводит два математических выражения — в записи Франсуа Виета (1540–1603):

$$\left\{ \begin{array}{l} H \text{ in } B \\ -F \text{ in } D \\ \hline -F + D \end{array} \right\} \text{aequebitur } A$$

и принятой сейчас:

$$\frac{ay - by}{b + y} = x.$$

Понятие языка неразрывно связано с понятием символа. Значительная часть языков,, в том числе языков программирования, используют в качестве своих базовых составляющих *цепочки символов* (идущих последовательно один за другим знаков). Что же это — *знак*? Рассмотрим в качестве примера следующий знак

Р

Что это? Русская буква «эр»? Латинская буква «пэ»? Греческая буква «ро»? Или обозначение давления (если, например, символ встретился в записи физического закона)? Возможно, обозначение фосфора при записи химической реакции? Автомобилисты могут вспомнить о столь дефицитной в больших городах стоянке. Мы начинаем чувствовать нечто важное, глубокое, характеризующее символ и язык вообще. А именно, формальный (синтаксический), содержательный (семантический) и прагматический аспекты этого феномена.

Внешний вид символов и способы их сочетания при записи послания образуют первый — формальный — уровень языка. Но есть еще и значение символа. Говорят, что знак обозначает *денотат*.

А есть еще и смысл записанного на языке послания для получателя!

Лучше понять это позволит следующий пример. Предположим, вы — зеленый человечек, брат по разуму из системы Эпсилон Эридана. Высадившись на Земле, вы находите листок бумаги — записку на русском языке: «Даша любит Петю». Что можно извлечь из этой записки? Безусловно, вы понимаете, что планета населена разумными существами, обладающими письменностью. Далее можете отметить наличие в записке линейной последовательности символов разного размера, разделенных пробелами. Некоторые символы повторяются, другие — нет. Видимо, это все. Проведен анализ послания с *синтаксической*, или *формальной*, точки зрения. Далее. Предположим, что записку находит кто-то из читающих по-русски. Какие он сделает выводы? Существует некая особа женского пола по имени Даша, и она равнодушна к мужчине или мальчику по имени Петя. Это уже понимание значения символов, или *семантический* анализ. После этого он без особых эмоций отложит записку в сторону или выбросит.

И лишь один-единственный Ваня, найдя эту записку в определенном месте, плача, рвет на себе волосы. А вот это уже — *прагматический* аспект, или *смысл* послания для получателя.

Иными словами, для перевода необходим учет контекста. В какой ситуации появилась фраза, к чему семантически относится высказывание. По большому счету, правильный перевод требует понимания. Необходима не пословная замена по словарю входящих во фразу составляющих, а понимание.

Проблемой первых систем машинного перевода было отсутствие так называемых «корпусов» - больших массивов одинаковых текстов на нескольких различных языках. В настоящее время эта проблема отпала. В Интернете накоплены огромные наборы текстов различного назначения (научных, разговорных, литературных, и пр., это можно считать неким вариантом «больших данных» - Big Data для текстов).

Несмотря на это, машинный перевод текста с одного естественного языка на другой до сих пор сталкивается с

проблемами. Сергей Марков, известный исследователь и популяризатор ИИ в России, приводит следующие примеры [22]. Фраза «Our cat gave birth to three kittens – two whites and one black» была переведена российской системой ПРОМТ в 2007 году как «Наш кот родил трех котят – двух белых и одного афроамериканца».

Понять качество работы систем машинного перевода поможет нижеследующее.

Пример - перевод стихов Афанасия Фета:

=====

*Мама! глянь-ка из окошка —
Знать, вчера недаром кошка
Умывала нос:
Грязи нет, весь двор одело,
Посветлело, побелело —
Видно, есть мороз.*

*Не колючий, светло-синий
По ветвям развешан иней —
Погляди хоть ты!
Словно кто-то тороватый
Свежей, белой, пухлой ватой
Все убрал кусты.*

*Уж теперь не будет спору:
За салазки, да и в гору
Весело бежать!
Правда, мама? Не откажешь,
А сама, наверно, скажешь:
«Ну, скорей гулять!»*

Перевод Яндекса на английский:

=====

*Mom! look out the window -
Know yesterday no wonder the cat
Washing her nose:
Dirt there is no, the entire yard clothed,
Brightened, whitened —
There's frost.*

*Not prickly, light blue
Frost hung on the branches —
Look at you!
As if someone torovatyy
Fresh, white, plump cotton
All removed the bushes.*

*Now there will be no dispute:
For the sled, and uphill
Fun to run!
Really, mom? Not refuse,
And itself, perhaps, do you say:
"Well, to the curb!»*

Обратный перевод Яндексa

=====

*Мама! выгляни в окно —
Знаете, вчера не даром кошка
Промывание носа:
Грязи нет, весь двор одет,
Посветлело, побелело —
Там же мороз.*

*Не колючий, светло-голубой
Мороз повис на ветвях —*

*Посмотри на себя!
Как будто кто то тороватый
Свежий, белый, пухлый хлопок
Все убрали кусты.*

*Теперь не будет никаких споров:
Для саней, и в гору
Весело бегать!
Правда, мам? Не отказываться,
А сама, пожалуй, скажешь:
"Ну, к обочине!»*

Перевод Google на английский:

=====

*Mama! take a look from the window -
Know yesterday not for nothing that a cat
Washed my nose:
There is no dirt, the whole yard is dressed
Brightened, whitened -
It can be seen there is frost.*

*Not scratchy, light blue
Frost is hung on the branches -
Look at least you!
It's like someone is weedy
Fresh, white, puffy cotton
All cleaned the bushes.*

*Now there will be no dispute:
For the slide, and uphill
Fun to run!
Really mom? You will not refuse*

*And you'll probably say it yourself:
"Well, hurry up for a walk!"*

Обратный перевод Google'ом:

=====

*Мама! посмотри из окна -
Знай вчера не зря, что кот
Помыл нос
Там нет грязи, весь двор одет
Осветлил, отбелил -
Видно, что мороз.*

*Не колючий, голубой
Мороз повешен на ветках -
Посмотри хоть на себя!
Это как кто-то слабый
Свежий, белый, пухлый хлопок
Все убрано в кустах.*

*Теперь не будет споров
Для горки и в гору
Весело бегать!
Действительно мама? Вы не откажетесь
И вы, вероятно, скажете это сами:
"Ну, поторопись на прогулку!"*

Языковое моделирование — задача, вставшая перед исследователями, занимавшимися машинным переводом.

...

Одной из широко известных задач, решавшихся в рамках языкового моделирования, стал t9, получивший своё название от английских слов "text on 9 buttons". В своё время мобильная связь нашла своего массового потребителя, причем одной из

востребованных услуг, наряду с голосовыми вызовами, быстро оказалась система коротких сообщений — SMS, Short Message System. Однако, телефоны того времени имели ограниченное число кнопок, в первую очередь предназначенных для набора номера. Набирать сообщение по одной букве, когда необходимо несколько раз нажимать одну и ту же кнопку — а иначе было никак! - было утомительным занятием, сопряженным ещё и с большим числом опечаток. Умные головы догадались предсказывать по контексту набираемого послания, какое слово наиболее вероятно пойдет дальше, и появились программы, подсказывающие при наборе сообщения несколько наиболее вероятных последующих слов.

Подобные программы появились и быстро стали популярными.

Как ни удивительно, но современные системы ИИ наподобие ChatGPT есть «t9 на стероидах»!

Языковая модель предсказывает вероятность появления следующего слова в последовательности.

Первые языковые модели вовсе не были нейросетевыми, а базировались на математической статистике. Правда, качество их работы было невысоким.

Но в 2017 году появились трансформеры, и произвели в данном направлении настоящую революцию. Ключевым в архитектуре данной искусственной нейронной сети стал механизм «внимания». Данный механизм напоминает метод восприятие текста человеком, когда в первую очередь для понимания важны лишь некоторые «ключевые» слова. В архитектуре «трансформер» сочетают «кодировщики» и «декодеры».

На базе данной архитектуры были построены множество больших языковых моделей.

BERT, или Bidirectional Encoder Representations from Transformers использовал предварительно обученные – англ. Pretrained модели, кои затем для применения в некой специализированной области несложно и относительно недорого можно дообучить (англ. Fine-tuning). Дообучение BERT показало отличные результаты при решении огромного спектра задач, где ранее часто для каждой

конкретной проблемы создавали свою архитектуру нейросети. При этом авторы выложили свою модель BERT в публичный доступ.

Следующей вехой стало появление модели GPT3 у компании OpenAI в 2020 году. GPT расшифровывается как Generative Pretrained Transformer. Прорывом стал масштаб. У модели было 175 миллиардов параметров, а обучали её на корпусах текстов суммарным размером 570 гигабайт. Для лучшего понимания отметим, что все тома «Войны и мира» Льва Толстого в простом текстовом формате укладываются менее чем в 1 (!) мегабайт, Шекспир за всю жизнь написал текста менее чем на 10 мегабайт. Количество перешло в качество.

GPT3 была успешно применена для решения многих задач вообще без дополнительного обучения! Удачной практикой стали "few shot" примеры, когда человек предоставляет языковой модели несколько примеров, в каком виде нужны ему ответы, и система его успешно понимает.

На базе английской GPT3 была в России создана и RuGPT.

Следующей ступенькой от OpenAI стал ChatGPT, построенный на базе GPT3, но дообученный в «инструктивной» манере, с примерами задач генерации текстов, востребованных человеком в типовых задачах. Дообучение по специальной «инструктивной» схеме позволило получить новое качество. Модель научилась хорошо реагировать на вопросы человека и вести эффективный диалог.

ChatGPT набрал 100 миллионов пользователей в Интернете менее чем за два месяца после выпуска для публичного доступа 30 ноября 2022 года.

Популярные модели Qwen и DeepSeek (Китай), Claude и LLAMA (США), Mistral (Франция), наши YandexGPT и GigaChat являются аналогами ChatGPT. Внутри базовой архитектурой является трансформер, дополнительно обученный.

DeepSeek R1 в январе-феврале 2025 года произвела фурор в первую очередь благодаря дополнительной возможности *рассуждений* (англ. reasoning), или построения логических цепочек, что заметно повышает интеллектуальный потенциал больших

языковых моделей. Весьма любопытно в данной связи, что многие ученые считают, что будущее ИИ связано с гибридизацией логического и нейросетевого подходов к искусственному интеллекту [].

На данном же этапе китайцы эффективно применили обучение с подкреплением, англ. Reinforcement Learning.

Иными словами, можно сказать, что современные качественные скачки в развитии языковых моделей выглядят на начало 2025 г. следующим образом:

BERT → GPT3 → ChatGPT → DeepSeek R1

Большим успехом стала интеграция языковых моделей с генеративными моделями для построения изображений (Midjourney, DALL-E, Stable Diffusion, Кандинский, Шедеврум), что дало возможность «читать» надписи внутри изображений и наоборот.

Поскольку изначально было замечено, что ChatGPT склонен к «фантазиям» или «галлюцинациям», или выдаче непроверенной, недостоверной или устаревшей информации, моделям стали предоставлять доступ либо к специализированным базам знаний (подход RAG – Retrieval-Augmented Generation), либо вообще выход в Интернет. В итоге, текст, построенный моделью, снабжает нас достоверной и свежей информацией, причем с указанием ссылок на источники, что позволяет данные легко проверить.

Следующий шаг — создание на базе больших языковых моделей так называемых «LLM-агентов», или виртуальных помощников.

Агенты могут быть физически воплощенными — роботами, техническими устройствами, снабженными датчиками и исполнительными механизмами и подобным путем взаимодействующими со средой, или просто виртуальными, когда ИИ-модель на компьютере способна запускать другие приложения, выходить в Интернет, обмениваться сообщениями с другими агентами, и т. д.

Например, если мы попросим подобного виртуального ассистента посчитать что-то, для него естественным будет запустить программу калькулятор для данной цели. Можно попросить в будущем заказать гостиницу с указанием своих предпочтений, забронировать билет на самолет, и пр.

Сценарии взаимодействия с внешней средой подобных агентов могут быть гибкими, что приближает будущее, кое становится на наших глазах настоящим.

Некоторые роботы от Boston Dynamics и других поставщиков уже интегрированы с большими языковыми системами.

Агенты, взаимодействующие между собой, смогут строить сообщества агентов, с «социализацией» некоторого рода.

Использование библиотек языка Питон для создания ИНС

Применение библиотеки Keras

Установка и настройка. Изначально Keras вырос как удобная надстройка над Theano, открытой библиотекой, основным разработчиком коей является группа машинного обучения в Монреальском университете. Отсюда его греческое имя — кέρας, что значит "рог". С тех пор утекло много воды, и Keras стал сначала поддерживать Tensorflow, а потом и вовсе стал его частью.

Keras устанавливается как обычный пакет Питона:

```
pip install keras
```

Keras предлагает последовательный и простой API — интерфейс прикладного программирования, он минимизирует количество действий пользователя, необходимых в большинстве случаев, и обеспечивает четкую и эффективную обратную связь.

Основная структура данных Keras - это **модель**, способ организации слоев искусственной нейронной сети. Шаг за шагом последовательно пользователь указывает, какие слои должны быть в конструируемой им сети.

В Keras доступны два основных типа моделей: последовательная модель Sequential и общий класс Model. Простейшим типом модели является Sequential, которая представляет собой линейную совокупность слоев. Для более сложных архитектур необходимо использовать функциональный API Keras, который позволяет создавать произвольный набор слоев.

Эти модели имеют ряд общих свойств и общих методов:

- model.layers - список слоев
- model.inputs - список входных тензоров
- model.outputs - список выходных тензоров модели.
- model.summary() - печатает сводку о модели.
- model.get_config() - возвращает словарь - конфигурацию модели.

- `model.get_weights()` - возвращает список весов модели.
- `model.set_weights(weights)` - устанавливает значения весов
- `model.to_json()` - возвращает представление модели как JSON.

Приведем пример использования метода `model.to_json()`:

```
from keras.models import model_from_json
json_string = model.to_json()
model = model.from_json(json_string)
```

Создать последовательную модель полносвязной нейросети можно передав список экземпляров слоя в конструктор класса `Sequential`, например (`Dense` – полносвязный слой):

```
from keras.models import Sequential
from keras.layers import Dense, Activation
model = Sequential([Dense(32, input_shape=(784,)), Activation('relu'),
Dense(10), Activation('softmax'),])
```

Но можно ещё просто добавлять слои с помощью метода `add()`:

```
model = Sequential()
model.add(Dense(32, input_dim=784))
model.add(Activation('relu'))
```

Модель должна знать размерность входного массива данных. Поэтому первый слой в `Sequential` модели (и только первый, потому что следующие слои могут получать автоматически эту информацию с предыдущего слоя) должен получать информацию про размерность входного массива.

Все слои Keras имеют ряд общих методов:

- `layer.get_weights()`- возвращает веса слоя в виде списка массивов.
- `layer.set_weights(weights)`- устанавливает веса слоя из списка массивов (с теми же формами, что и выход `get_weights()`).
- `layer.get_config()` - возвращает конфигурацию слоя.

Слой создается использованием следующего метода:

```
keras.layers.Dense(units, activation=None, use_bias=True, kernel_initializer='glorot_uniform', bias_initializer='zeros', kernel_regularizer=None, bias_regularizer=None, activity_regularizer=None, kernel_constraint=None, bias_constraint=None)
```

Слой может быть восстановлен из его конфигурации:

```
layer = Dense(32)
config = layer.get_config()
reconstructed_layer = Dense.from_config(config)
```

Метод извлечения слоя сети на основе его имени или индекса возвращает экземпляр слоя:

```
get_layer(self, name=None, index=None)
```

Основные аргументы: `name`: - строка, имя слоя; `index` - число, индекс слоя.

Чтобы указать функцию активации, можно использовать следующий метод:

```
keras.layers.Activation(activation)
```

В качестве аргумента `activation` необходимо указать имя используемой функции активации.

Для преобразования формата выхода к определенной форме можно использовать следующий метод:

```
keras.layers.Reshape(target_shape)
```

В качестве аргумента `target_shape` указывается кортеж целых чисел.

Рассмотрим пример:

```
model.add(Reshape((3, 4), input_shape=(12,)))
# размерность массива выходного слоя: model.output_shape ==
(None, 3, 4)
model.add(Reshape((6, 2)))
# размерность массива выходного слоя: model.output_shape ==
(None, 6, 2)
```

Кроме полносвязной сети, Keras позволяет создавать и более сложные архитектуры. Например, добавить слой свёртки. Особенностью свёрточного слоя является сравнительно небольшое число параметров.

Метод `Conv1D` создает сверточное ядро, по одному пространственному (или временному) измерению:

```
keras.layers.Conv1D(filters, kernel_size, strides=1, padding='valid', data_format='channels_last', dilation_rate=1, activa-
```

```
tion=None, use_bias=True, kernel_initializer='glorot_uniform', bi-
as_initializer='zeros', kernel_regularizer=None, bias_regularizer=None
, activity_regularizer=None, kernel_constraint=None, bi-
as_constraint=None)
```

Основные аргументы: `filters` - размерность выходного пространства (количество выходных фильтров в свертке); `kernel_size` - размер окна свертки; `strides`: величина шага свертки; `activation` - функция активации слоя. Если этот параметр не указан, то активация не применяется (используется «линейная» функция активации $a(x) = x$).

Conv2D – двумерный сверточный слой (свертка изображений:

```
keras.layers.Conv2D(filters, kernel_size, strides=(1, 1), pad-
ding='valid', data_format=None, dilation_rate=(1, 1), activa-
tion=None, use_bias=True, kernel_initializer='glorot_uniform', bi-
as_initializer='zeros', kernel_regularizer=None, bias_regularizer=None, activi-
ty_regularizer=None, kernel_constraint=None, bias_constraint=None)
```

Слой пулинга представляет собой нелинейное уплотнение карты признаков, при этом группа пикселей (обычно размера 2×2) уплотняется до одного пикселя, проходя нелинейное преобразование. Преобразования затрагивают непересекающиеся прямоугольники или квадраты, каждый из которых ужимается в один пиксель, при этом выбирается пиксель, имеющий максимальное значение.

Операция пулинга позволяет существенно уменьшить объём изображения. К тому же фильтрация уже ненужных деталей помогает не переобучаться. Слой пулинга, как правило, вставляется после слоя свёртки перед слоем следующей свёртки.

```
keras.layers.MaxPooling1D(pool_size=2, strides=None, padding='vali-
d')
```

Основные аргументы: `pool_size` - число, размер максимальных окон объединения; `strides` - параметр, с помощью которого можно уменьшить масштаб. Например, значение 2 уменьшит вдвое.

MaxPooling2D используется для двумерного пулинга:

```
keras.layers.MaxPooling2D(pool_size=(2, 2), strides=None, pad-
ding='valid', data_format=None)
```

Как мы уже знаем, переобучение (*overfitting*) — одна из проблем глубоких нейронных сетей, состоящая в том, что модель хорошо распознает только примеры из обучающей выборки, адаптируясь к обучающим примерам, вместо того чтобы учиться классифицировать

примеры, не участвовавшие в обучении - теряя способность к обобщению. Наиболее эффективным решением проблемы переобучения является метод (Dropout):

```
keras.layers.Dropout(rate, noise_shape=None, seed=None)
```

Сети получают здесь с помощью исключения из сети (dropping out) нейронов с вероятностью rate, таким образом, вероятность того, что нейрон останется в сети, составляет $1 - \text{rate}$. “Исключение” нейрона означает, что при любых входных данных или параметрах он возвращает 0.

Модель после определения архитектуры необходимо скомпилировать:

```
compile(self, optimizer, loss=None, metrics=None,
loss_weights=None, sample_weight_mode=None, weighted_metrics=None,
target_tensors=None)
```

Метод имеет три аргумента:

- Оптимизатор. Это может быть строковый идентификатор существующего оптимизатора (например, rmsprop или adagrad) или экземпляр класса Optimizer.
- Функция вычисления ошибок. Её модель попытается свести к минимуму. Может быть строковым идентификатором существующей функции ошибок (например, categorical_crossentropy или mse), или функцией.
- Список метрик, может быть строковым идентификатором существующей метрики или специальной метрической функцией.

Сеть для задачи классификации по нескольким классам:

```
model.compile(optimizer='rmsprop', loss='categorical_crossentropy', metrics=['accuracy'])
```

Для задачи бинарной классификации:

```
model.compile(optimizer='rmsprop', loss='binary_crossentropy', metrics=['accuracy'])
```

Для задачи регрессии со среднеквадратичной ошибкой:

```
model.compile(optimizer='rmsprop', loss='mse')
```


Пример создания и использования собственной метрики:

```
import keras.backend as K
def mean_pred(y_true, y_pred):
    return K.mean(y_pred)

model.compile(optimizer='rmsprop', loss='binary_crossentropy',
              metrics=['accuracy', mean_pred])
```

Обучение модели для определенного количества эпох осуществляется с помощью метода `fit`:

```
fit(self, x=None, y=None, batch_size=None, epochs=1, verbose=1,
    callbacks=None, validation_split=0.0, validation_data=None,
    shuffle=True, class_weight=None, sample_weight=None, initial_epoch=0,
    steps_per_epoch=None, validation_steps=None)
```

Основные аргументы этого метода: `x` - массив данных обучения; `y` - массив целевых выходных данных (если модель имеет один вывод) или список массивов (если модель имеет несколько выходов); `batch_size` - количество подвыборок при обновлении градиента; `epochs` - количество эпох обучения модели; `validation_split` - число между 0 и 1, доля данных валидации; `initial_epoch`: эпоха, с которой начать обучение (для возобновления предыдущего цикла обучения).

Метод для получения результатов сети - прогнозов:

```
predict(self, x, batch_size=None, verbose=0, steps=None)
```

Основные аргументы метода: `x` - входные данные, в виде массива (или список массивов NumPy, если модель имеет несколько входов); `steps` - количество шагов до раунда прогнозирования.

Готовый метод для оценки качества модели возвращает значения ошибок и показателей для модели в тестовом режиме:

```
evaluate(self, x=None, y=None, batch_size=None, verbose=1)
```

Рассмотрим некоторые примеры построения сетей в Keras.

Сначала модель для бинарной классификации:

```
model = Sequential()
model.add(Dense(32, activation='relu', input_dim=100))
model.add(Dense(1, activation='sigmoid'))
model.compile(optimizer='rmsprop', loss='binary_crossentropy',
              metrics=['accuracy'])
# Создаем фиктивные данные
import numpy as np
```

```

data = np.random.random((1000, 100))
labels = np.random.randint(2, size=(1000, 1))
# Обучаем модель в 10 эпох по 32 примера в пакете
model.fit(data, labels, epochs=10, batch_size=32)

```

Далее приведем пример построения многослойного перцептрона для классификации по нескольким классам:

```

# необходимые библиотеки
import keras
from keras.models import Sequential
from keras.layers import Dense, Dropout, Activation
from keras.optimizers import SGD
import numpy as np

x_train = np.random.random((1000, 20))
y_train = keras.utils.to_categorical(np.random.randint(10,
size=(1000, 1)), num_classes=10)

x_test = np.random.random((100, 20))
y_test = keras.utils.to_categorical(np.random.randint(10,
size=(100, 1)), num_classes=10)

# создаем сеть с последовательными слоями
model = Sequential()

# сначала - полносвязный слой
model.add(Dense(64, activation='relu', input_dim=20))

# следующий слой - Dropout
model.add(Dropout(0.5))
# слой с функцией активации ReLU
model.add(Dense(64, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(10, activation='softmax'))
sgd = SGD(lr=0.01, decay=1e-6, momentum=0.9, nesterov=True)

# компилируем модель
model.compile(loss='categorical_crossentropy', optimizer=sgd,
metrics=['accuracy'])

model.fit(x_train, y_train, epochs=20, batch_size=128)
score = model.evaluate(x_test, y_test, batch_size=128)

```

Keras, помимо методов для построения, обучения и применения нейросетей, дает возможность использовать и некоторые стандартные наборы данных, включая популярные CIFAR10 и MNIST.

MNIST - набор данных, разработанный Яном Лекуном с коллегами для оценки моделей машинного обучения на задаче классификации рукописных цифр. Набор данных был построен из ряда отсканированных наборов документов, доступных в Национальном институте стандартов и технологий (NIST). Изображения цифр были взяты из множества отсканированных документов, нормированных по размеру и по центру. Это делает его отличным набором данных для оценки моделей, позволяя разработчику сосредоточиться на механизме обучения с очень небольшой очисткой данных или необходимой подготовкой.

Каждое изображение представляет собой квадрат размером 28 на 28 пикселей (всего 784 пикселя). В этом наборе 60 000 изображений используются для обучения модели, и для ее тестирования используется отдельный набор из 10 000 изображений. Это задача распознавания 10 цифр (от 0 до 9) или классификация на 10 классов.

Библиотека глубокого обучения Keras предоставляет удобный метод `mnist.load_data()` для загрузки набора данных MNIST. Набор данных загружается автоматически при первом вызове этой функции и сохраняется в вашем домашнем каталоге в `~ / .keras / datasets / mnist.pkl.gz` в виде файла 15 мегабайт, что очень удобно для разработки и тестирования моделей глубокого обучения. Чтобы продемонстрировать, насколько легко загружать набор данных MNIST, мы сначала напишем небольшой скрипт для загрузки и визуализации первых четырех изображений в наборе учебных материалов:

```
from keras.datasets import mnist
import matplotlib.pyplot as plt
# load (downloaded if needed) the MNIST dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()
# plot 4 images as gray scale
plt.subplot(221)
plt.imshow(X_train[0], cmap=plt.get_cmap('gray'))
plt.subplot(222)
plt.imshow(X_train[1], cmap=plt.get_cmap('gray'))
plt.subplot(223)
plt.imshow(X_train[2], cmap=plt.get_cmap('gray'))
plt.subplot(224)
plt.imshow(X_train[3], cmap=plt.get_cmap('gray'))
# show the plot
plt.show()
```

Чтобы понять действительно ли нам нужна для распознавания сложная модель, такая как сверточная нейронная сеть, попробуем использовать очень простую модель нейронной сети с одним скрытым слоем.

Начнем с импорта классов и функций, которые нам понадобятся.

```
import numpy
from keras.datasets import mnist
from keras.models import Sequential
from keras.layers import Dense
from keras.layers import Dropout
from keras.utils import np_utils
```

Набор данных структурирован как трехмерный массив. Чтобы подготовить данные, сперва мы представим изображения в виде одномерных массивов (так как считаем каждый пиксель отдельным входным признаком). В этом случае изображения размером 28×28 будут преобразованы в массивы, содержащие 784 элементов.

Мы можем сделать это преобразование, используя функцию `reshape()` библиотеки NumPy. Для уменьшения потребления оперативной памяти преобразуем точность значений пикселей в 32.

```
(X_train, y_train), (X_test, y_test) = mnist.load_data()
num_pixels = X_train.shape[1] * X_train.shape[2]
X_train = X_train.reshape(X_train.shape[0],
num_pixels).astype('float32')
X_test = X_test.reshape(X_test.shape[0],
num_pixels).astype('float32')
```

Значения пикселей заданы в оттенках серого со значениями от 0 до 255. Для эффективного обучения нейронных сетей практически всегда рекомендуется выполнять некоторое масштабирование входных значений. Мы можем нормализовать значения пикселей в диапазоне 0 и 1, разделив каждое значение на максимальные значения 255:

```
X_train = X_train / 255
X_test = X_test / 255
```

Выход представляет собой целое число от 0 до 9. Хорошей практикой является использование кодирования значений класса преобразованием вектора целых чисел класса в двоичную матрицу.

Мы можем легко сделать это, используя встроенную вспомогательную функцию `np_utils.to_categorical()` в Keras:

```
y_train = np_utils.to_categorical(y_train)
y_test = np_utils.to_categorical(y_test)
num_classes = y_test.shape[1]
```

Создадим нашу базовую модель однослойной нейронной сети:

```
def baseline_model():
    model = Sequential()
    model.add(Dense(num_pixels, input_dim=num_pixels,
        kernel_initializer='normal', activation='relu'))
    model.add(Dense(num_classes, kernel_initializer='normal',
        activation='softmax'))
    model.compile(loss='categorical_crossentropy', optimizer='adam',
        metrics=['accuracy'])
    return model
```

Прокомментируем вышеприведенный фрагмент программы. Модель представляет собой простую нейронную сеть с одним скрытым слоем с таким же количеством нейронов, что и количество входов (784). В скрытом слое используем полулинейную функцию активации `relu`.

На выходном слое используется функция активации `softmax` для преобразования выходов в вероятностные значения, позволяя выбрать один класс из 10 в качестве выходного значения модели.

Теперь нам осталось только определить функцию потерь, алгоритм оптимизации и метрики, которые мы будем собирать. В задачах с вероятностной классификацией, в качестве функции потерь лучше всего использовать не квадратичную ошибку, а кросс-энтропию. Потери будут меньше для вероятностных задач (например, с логистической/`softmax` функцией для выходного слоя), в основном из-за того, что данная функция предназначена для максимизации уверенности модели в правильном определении класса, и ее не заботит распределение вероятностей попадания образца в другие классы.

Используемый алгоритм оптимизации будет модификацией алгоритма градиентного спуска, отличие будет лишь в том, как выбирается скорость обучения. В нашем случае мы будем использовать оптимизатор Adam, который обычно показывает хорошую производительность.

Так как наши классы сбалансированы (количество рукописных цифр, принадлежащих каждому классу, одинаково), подходящей метрикой будет точность (accuracy) — доля входных данных, отнесенных к правильному классу.

Наконец, мы можем обучить модель и оценить её качество:

```
model = baseline_model()
model.fit(X_train, y_train, validation_data=(X_test, y_test),
epochs=10,
batch_size=200, verbose=2)
scores = model.evaluate(X_test, y_test, verbose=0)
print("Baseline Error: %.2f%%" % (100-scores[1]*100))
```

Модель подходит 10 эпох обучения, при каждом обновлении весов используется 200 изображений. Тестовые данные которые используются в качестве набора данных валидации, позволяют вам видеть качество распознавания модели по мере ее обучения. Значение `verbose = 2` используется для уменьшения вывода на одну строку для каждой учебной эпохи. Далее тестовый набор данных используется для оценки модели и выдается сообщение о размере ошибки.

Результат:

```
Train on 60000 samples, validate on 10000 samples
Epoch 1/10 - 21s - loss: 0.2781 - acc: 0.9213 - val_loss: 0.1443
- val_acc: 0.9585
Epoch 2/10 - 21s - loss: 0.1100 - acc: 0.9686 - val_loss: 0.0943
- val_acc: 0.9709
Epoch 3/10 - 18s - loss: 0.0709 - acc: 0.9798 - val_loss: 0.0809
- val_acc: 0.9739
Epoch 4/10 - 18s - loss: 0.0511 - acc: 0.9855 - val_loss: 0.0679
- val_acc: 0.9781
Epoch 5/10 - 18s - loss: 0.0361 - acc: 0.9898 - val_loss: 0.0650
- val_acc: 0.9801
64
Epoch 6/10 - 18s - loss: 0.0265 - acc: 0.9936 - val_loss: 0.0640
- val_acc: 0.9790
Epoch 7/10 - 18s - loss: 0.0191 - acc: 0.9953 - val_loss: 0.0624
- val_acc: 0.9810
```

```
Epoch 8/10 - 18s - loss: 0.0145 - acc: 0.9965 - val_loss: 0.0592
- val_acc: 0.9822
Epoch 9/10 - 18s - loss: 0.0109 - acc: 0.9977 - val_loss: 0.0554
- val_acc: 0.9827
Epoch 10/10 - 18s - loss: 0.0079 - acc: 0.9986 - val_loss:
0.0596 - val_acc: 0.9814
Baseline Error: 1.86%
```

Как видно, наша модель достигает точности приблизительно 98.14% и ошибки 1.86% на тестовом наборе данных, это вполне достойно для такой простой модели.

Использование библиотеки PyTorch

PyTorch — это фреймворк для глубокого обучения, разработанный Facebook's AI Research Lab (FAIR). Он используется как для исследований (большинство ученых в этой области применяют именно его), так и для разработки приложений.

Преимущества PyTorch:

- Динамическое построение графов вычислений: PyTorch использует динамическое построение графов, что упрощает отладку и делает программу более интуитивно понятной.
- Интуитивный синтаксис.
- PyTorch имеет большое и активное сообщество пользователей.

Недостатки PyTorch:

- Меньшая экосистема: Хотя PyTorch быстро развивается, его экосистема менее обширна по сравнению с Keras/TensorFlow.
- Отсутствие встроенной поддержки распределенных вычислений: PyTorch имеет менее развитую поддержку распределенных вычислений по сравнению с TensorFlow.

Рассмотрим работу с уже знакомым нам набором данных MNIST с использованием библиотеки PyTorch.

Сначала нам нужно настроить наше окружение в Google Colab и установить необходимые библиотеки:

```
#@title Установка и импорт библиотек
!pip install torchviz
!pip install torch torchvision # Устанавливаем PyTorch и
библиотеку torchvision для работы с изображениями
```

```

import torch # Импортируем основную библиотеку PyTorch
import torchvision # Импортируем библиотеку torchvision
# Импортируем необходимые модули для визуализации
import torchviz
from torch.autograd import Variable
from torch.utils.data import Dataset, DataLoader
import torch.nn as nn # Импортируем модуль для создания
нейронных сетей
import torch.nn.functional as F # Импортируем модуль для
функций активации и других операций
import torch.optim as optim # Импортируем модуль для
оптимизаторов
import numpy as np # Импортируем библиотеку NumPy
import matplotlib.pyplot as plt
from IPython.display import clear_output
from torchviz import make_dot
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
from IPython.display import Image

clear_output() # очистка вывода

# Определяем трансформации для предобработки данных
transform = transforms.Compose([
    transforms.ToTensor(), # Преобразуем изображения в тензоры.
    Это необходимо для работы с PyTorch, так как все данные должны быть
    в формате тензоров.
    transforms.Normalize((0.5,), (0.5,)) # Нормализуем данные.
    Преобразуем диапазон значений пикселей с [0, 1] в [-1, 1] с
    использованием среднего значения 0.5 и стандартного отклонения 0.5.
])

# Загружаем тренировочный и тестовый наборы данных MNIST
train_dataset = datasets.MNIST(root='./data', train=True, down-
load=True, transform=transform)

test_dataset = datasets.MNIST(root='./data', train=False, down-
load=True, transform=transform)

# Создаем загрузчики данных
train_loader = DataLoader(train_dataset, batch_size=64, shuf-
file=True)

test_loader = DataLoader(test_dataset, batch_size=64, shuf-
file=True)

```

Теперь мы определим простую нейронную сеть, используя `nn.Module`. Наша сеть будет состоять из нескольких слоев, включая

сверточные и полносвязные слои. Как и в случае использования Keras, мы последовательно — но в данном случае более детально — прописываем, какие слои в сети идут один за другим.

```
class SimpleNN(nn.Module):
    """
    Класс, определяющий простую полносвязную нейронную сеть,
    наследуется от nn.Module.

    Атрибуты:
    fc1 - первый полносвязный слой
    fc2 - второй полносвязный слой
    fc3 - третий полносвязный слой
    relu - функция активации ReLU
    """
    def __init__(self): # это метод-конструктор класса. Он
        вызывается автоматически при создании нового экземпляра класса -
        self.
        super(SimpleNN, self).__init__() # используется для вызова
            методов базового класса, первый аргумент указывает на
            текущий класс (SimpleNN), а второй аргумент (self) — это
            экземпляр текущего класса
        self.fc1 = nn.Linear(28 * 28, 128) # Первый полносвязный
            слой с входом 28*28 и выходом 128 нейронов
        self.fc2 = nn.Linear(128, 64) # Второй полносвязный слой с
            входом 128 и выходом 64 нейрона
        self.fc3 = nn.Linear(64, 10) # Третий полносвязный слой с
            входом 64 и выходом 10 нейронов (количество классов)
        self.relu = nn.ReLU() # Функция активации ReLU, обнуляет
            выходные значения активации нейронов, если они
            отрицательные

    def forward(self, x):
        """
        Метод, определяющий прямое распространение (forward pass)
        через нейронную сеть.

        Параметры:
        x - вход

        Возвращает:
        Результат прохождения через все слои сети
        """
        x = x.view(-1, 28 * 28) # Преобразуем входные данные
            (изображение) в вектор. Параметр -1 позволяет PyTorch
            автоматически определять размер первой размерности
            (например, размер пакета).
        x = self.relu(self.fc1(x)) # Применяем ReLU к первому слою
        x = self.relu(self.fc2(x)) # Применяем ReLU ко второму слою
```

```
x = self.fc3(x) # Применяем третий полносвязный слой
return x
```

Созданная структура нейросети имеет три полносвязных слоя и функцию активации Relu. Как мы уже отмечали, PyTorch позволяет изменять структуру и параметры сети в процессе обучения. Это - одно из его главных преимуществ.

Далее инициализируем нашу модель, определим функцию потерь и выберем оптимизатор для обучения.

```
# Инициализируем модель (создаем экземпляр класса SimpleNN)
model = SimpleNN()

# Определяем функцию потерь
criterion = nn.CrossEntropyLoss() # измеряет, насколько хорошо
работает модель, путем сравнения предсказанных выходов модели с
истинными метками

# Выбираем оптимизатор
optimizer = optim.Adam(model.parameters(), lr=0.001) #
используется для обновления весов модели на основе градиентов,
вычисленных во время обратного распространения
```

На следующем шаге мы обучим нашу нейронную сеть на тренировочном наборе данных. Мы будем выполнять несколько эпох обучения, в каждой из которых будем проходить через все тренировочные данные.

Обратите внимание, обучение занимает некоторое, заметное время. Наиболее эффективно использование графических ускорителей, причем PyTorch, в отличие от Keras, предоставляет гибкие возможности параллельного, например, использования двух видеокарт.

```
def train_model_with_gradients(model, train_loader, criterion,
optimizer, num_epochs=10):
    """
    Функция для обучения модели и сохранения градиентов на каждом
    шаге.

    Входные параметры:
    model - модель нейронной сети (экземпляр nn.Module)
    train_loader - загрузчик тренировочных данных (DataLoader)
    criterion - функция потерь (nn.CrossEntropyLoss)
    optimizer - оптимизатор (optim.Adam)
```

num_epochs - количество эпох обучения (по умолчанию 10)

Возвращаемые значения:

gradients - список градиентов для каждого слоя на каждом шаге

layer_names - список имен слоев модели

"""

```
gradients = [] # Создаем список для хранения градиентов
# Сохраняем имена слоев для легенды графика
layer_names = [name for name, _ in model.named_parameters()]
for epoch in range(num_epochs): # Проходимся по количеству эпох
    model.train() # Устанавливаем модель в режим обучения
    running_loss = 0.0 # Инициализируем переменную потерь
    for images, labels in train_loader:
        optimizer.zero_grad() # Обнуляем градиенты перед каждым
            шагом обучения
        outputs = model(images) # Прямой проход
        loss = criterion(outputs, labels) # Вычисляем потери
        loss.backward() # Обратный проход: вычисляем градиенты
        # Сохраняем значения градиентов
        gradients.append([param.grad.abs().mean().item() for param
            in model.parameters() if param.grad is not None])

        optimizer.step() # Шаг оптимизации: обновляем параметры
            модели на основе вычисленных градиентов
        running_loss += loss.item() * images.size(0)
        # Вычисляем средние потери за эпоху
        epoch_loss = running_loss / len(train_loader.dataset)
    print(f'Epoch [{epoch+1}/{num_epochs}], Потери:
        {epoch_loss:.4f}')
    # Возвращаем сохраненные градиенты и имена слоев
    return gradients, layer_names
```

Обучаем модель

```
gradients, layer_names = train_model_with_gradients(model,
train_loader, criterion, optimizer, num_epochs=10) # Обучаем модель
и получаем градиенты и имена слоев
```

Вывод:

```
Epoch [1/10], Loss: 0.3920
Epoch [2/10], Loss: 0.1899
Epoch [3/10], Loss: 0.1373
Epoch [4/10], Loss: 0.1084
Epoch [5/10], Loss: 0.0881
Epoch [6/10], Loss: 0.0800
Epoch [7/10], Loss: 0.0700
Epoch [8/10], Loss: 0.0627
Epoch [9/10], Loss: 0.0575
Epoch [10/10], Loss: 0.0543
```

Видим, что после 10 эпох обучения достигнута неплохая точность с потерями около пяти процентов.

После обучения мы оценим нашу модель на тестовом наборе данных и визуализируем предсказания.

```
#@title Функция для оценки модели
def evaluate_model(model, test_loader):
    """
    Функция для оценки точности модели на тестовом наборе данных.

    Входные параметры:
    model - модель нейронной сети, кою нужно оценить
    test_loader - загрузчик проверочных данных

    Функция ничего не возвращает. Дает точность модели на прове
    рочных изображениях.
    """
    model.eval() # Устанавливаем модель в режим оценки
    correct = 0 # Счетчик правильных предсказаний
    total = 0 # Общее количество образцов

    with torch.no_grad(): # Отключаем вычисление градиентов
        # Проходимся по тестовым данным (изображения, истинные метки)
        for images, labels in test_loader:
            outputs = model(images) # Прямой проход
            # Находим класс с максимальным значением индекса для
            # каждого изображения в батче
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0) # Считаем общее количество образцов
            correct += (predicted == labels).sum().item()

    accuracy = 100 * correct / total # Находим точность
    print(f'Точность модели на тестовых изображениях:
          {accuracy:.2f}%') # Выводим точность

# Оцениваем модель
evaluate_model(model, test_loader)
```

Вывод:

Точность модели на тестовых изображениях: 97.12%

Что ж, это очень неплохой показатель!

Итак, мы успешно создали и обучили нейронную сеть на наборе данных MNIST. Мы прошли через все шаги, начиная с загрузки данных и заканчивая оценкой модели и визуализацией предсказаний.

Теперь у вас есть базовое понимание того, как работать с PyTorch и обучать с ним нейронную сеть.

Распознавание медицинских изображений

Дальше попробуем усложнить задачу. Продемонстрируем использование предобученной модели ResNet18 с PyTorch для классификации рентгеновских снимков легких. Пример взят, как и некоторые другие, из бесплатных материалов компании «Университет ИИ».

Набор данных содержит рентгеновские снимки легких с пневмонией и без. Изображения рассортированы по классам и организованы в соответствующие папки. Имеется 1341 снимка без патологии, и 3875 с пневмонией, всего 5216 изображения.

В нашем примере мы будем использовать предобученную ResNet18.

Сначала нам нужно настроить наше окружение в Google Colab и установить необходимые библиотеки:

```
# Импортируем необходимые библиотеки
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms, models
import matplotlib.pyplot as plt # Библиотека для визуализации
import numpy as np # Библиотека для работы с массивами

import gdown # Утилита для загрузки файлов из Google Drive
import os # Библиотека для работы с операционной системой
import shutil # Модуль для работы с файлами и директориями

#для разделения данных на обучающую и тестовую выборки
from sklearn.model_selection import train_test_split
# Модуль преобразований изображений
from torchvision import transforms
from PIL import Image # Библиотека для работы с изображениями
```

```

#На следующем шаге осуществляем загрузку и предобработку данных:
#@title Функция загрузки датасета и его и разделения на
тренировочную и тестовую выборки
# Функция для загрузки и разархивации датасета
def download_and_unzip_dataset(url, output_path, extract_to):
    gdown.download(url, output_path, quiet=True)
    !unzip -qo {output_path} -d {extract_to}
    print("Содержимое папки:", os.listdir(extract_to))

# Функция для разделения данных на тренировочные и тестовые
def split_dataset(data_dir, train_dir, test_dir, test_size=0.2):
    # Проверяем, что папки для разделения существуют, и создаем
    их, если они отсутствуют
    if not os.path.exists(train_dir):
        os.makedirs(train_dir)
    if not os.path.exists(test_dir):
        os.makedirs(test_dir)

    classes = os.listdir(data_dir)
    for cls in classes:
        class_dir = os.path.join(data_dir, cls)
        if os.path.isdir(class_dir):
            files = os.listdir(class_dir)
            if len(files) == 0:
                print(f"Категория {cls} пуста, пропускаем.")
                continue
            train_files, test_files = train_test_split(files,
                test_size=test_size, random_state=42)

# Создаем папки для каждой категории в обоих наборах
train_class_dir = os.path.join(train_dir, cls)
test_class_dir = os.path.join(test_dir, cls)
if not os.path.exists(train_class_dir):
    os.makedirs(train_class_dir)
if not os.path.exists(test_class_dir):
    os.makedirs(test_class_dir)

# Копируем файлы в нужные папки
for file in train_files:
    shutil.move(os.path.join(class_dir, file),
                os.path.join(train_class_dir, file))
for file in test_files:
    shutil.move(os.path.join(class_dir, file), os.path.join(test_class_dir, file))

# URL для загрузки набора данных
dataset_url =
'https://drive.google.com/uc?id=1iHp9lxKFcHt1fBDv66ShB46LVn-8zRjE'
output_path = 'pneumonia.zip'
extract_to = '/content/pneumonia/'

```

```

# Вызов функции для загрузки и разархивации датасета
download_and_unzip_dataset(dataset_url, output_path, extract_to)

# Пути к папкам для тренировочных и тестовых данных
train_dir = '/content/train'
test_dir = '/content/test'

# Определяем метки классов
classes = ['NORMAL', 'PNEUMONIA'] # Метки классов

# Вызов функции для разделения данных на тренировочные и тестовые
наборы
split_dataset(extract_to, train_dir, test_dir)

# Проверяем содержимое папок после разделения
print("Содержимое папки train:", os.listdir(train_dir))
print("Содержимое папки test:", os.listdir(test_dir))
# Определяем трансформации для предобработки данных
transform = transforms.Compose([
# Изменяем размер изображений до 224x224 пикселей
# Нормализуем данные (сред. и станд.отклонения дляImageNet)
transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224,
0.225])
])

# Загружаем тренировочный и тестовый наборы данных

# Тренировочный набор данных с применением трансформаций
train_dataset = datasets.ImageFolder(root=train_dir, trans-
form=transform)

# Тестовый набор данных с применением трансформаций
test_dataset = datasets.ImageFolder(root=test_dir, trans-
form=transform)

# Создаем загрузчики данных
train_loader = DataLoader(train_dataset, batch_size=32, shuf-
fle=True)

test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)

# Выводим информацию о размерах наборов данных
print(f'Размер тренировочного набора: {len(train_dataset)}
изображений')

print(f'Размер тестового набора: {len(test_dataset)} изображений')
# Вывод размера тестового набора данных

```

Вывод:

Размер тренировочного набора: 4172 изображений

Размер тестового набора: 1044 изображений

Как уже говорилось, будем использовать предобученную модель ResNet18 и настроим ее на следующем шаге для нашей задачи классификации:

```
# Загружаем предобученную модель ResNet18
model = models.resnet18(pretrained=True)

# Заменяем последний слой для классификации на 2 класса (NORMAL,
# PNEUMONIA)
# Получаем количество входных признаков выходного слоя
num_features = model.fc.in_features

# Заменяем выходной слой
model.fc = nn.Linear(num_features, 2)

# Перемещаем модель на устройство (GPU, если доступно)
device = torch.device('cuda' if torch.cuda.is_available() else
    'cpu')
# Определяем устройство для вычислений (GPU или CPU)
model = model.to(device)
```

Далее определим функцию потерь и выберем оптимизатор для обучения модели:

```
# Определяем функцию потерь и оптимизатор
# Используем функцию потерь CrossEntropyLoss для классификации
criterion = nn.CrossEntropyLoss()
# Используем оптимизатор Adam с скоростью обучения 0.001
optimizer = optim.Adam(model.fc.parameters(), lr=0.001)
```

Наконец мы можем перейти к обучению нейронной сети на тренировочном наборе данных:

```
# Функция для обучения модели
def train_model(model, train_loader, criterion, optimizer, device, num_epochs=5):
    for epoch in range(num_epochs):
        model.train() # Устанавливаем модель в режим обучения
        running_loss = 0.0
        for images, labels in train_loader:
            images, labels = images.to(device), labels.to(device)
```



```

optimizer.zero_grad() # Обнуляем градиенты предыдущих
                        итераций
outputs = model(images) # Прямой проход:
loss = criterion(outputs, labels) # Вычисляем потери
loss.backward() # Обратный проход: вычисляем градиенты
optimizer.step() # Обновляем параметры модели с учетом
                  вычисленных градиентов
running_loss += loss.item() * images.size(0)
epoch_loss = running_loss / len(train_loader.dataset)
print(f'Epoch [{epoch+1}/{num_epochs}], Loss: {epoch_loss:.4f}')

# Обучаем модель
train_model(
    model,          # Модель, которую мы обучаем (экземпляр nn.Module)
    train_loader,   # Загрузчик тренировочных данных (DataLoader)
    criterion,      # Функция потерь
    optimizer,      # Оптимизатор, используемый для обновления весов
    device,         # Устройство, на котором производится вычисление
    num_epochs=25   # Количество эпох для обучения (по умолчанию 25)
)

```

Обратим внимание, что процесс может занять до 10 минут и более. После обучения оценим нашу модель на тестовом наборе данных и визуализируем предсказания.

Для лучшего понимания, мы визуализируем несколько изображений из тестового набора данных вместе с их предсказанными и реальными метками, см. рисунок 3.16.

Создадим для этого специальную функцию:

```

#@title Функция для визуализации изображений
def visualize_images_from_folder(model,          folder_path,
class_names, folder_name, num_images=6):
    model.eval() # Устанавливаем модель в режим оценки

    # Создаем список изображений из папки
    image_paths = [os.path.join(folder_path, fname) for fname in
                    os.listdir(folder_path)
                    if fname.endswith(('.png', '.jpg', '.jpeg'))]
    if len(image_paths) == 0: # Проверяем, есть ли изображения
        print(f"Изображения не найдены в {folder_path}.")
        return

    # Применяем преобразования изображений
    transform = transforms.Compose([
        transforms.Resize((224, 224)), # Изменяем размер до 224x224
        transforms.ToTensor(), # Преобразуем изображение в тензор

```

```

transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229,
0.224, 0.225]) # Нормализуем ])

# Визуализируем
fig = plt.figure(figsize=(12, 6))
for i, img_path in enumerate(image_paths[:num_images]):
    image = Image.open(img_path) # Открываем изображение

    # Проверка на количество каналов и преобразование в
    # трехканальное изображение
    if image.mode != 'RGB':
        image = image.convert('RGB')
    image_tensor = transform(image).unsqueeze(0).to(device)
    with torch.no_grad(): # Отключаем вычисление градиентов
        outputs = model(image_tensor) # Пропускаем через модель
    # Получаем предсказание
    _, predicted = torch.max(outputs, 1)
    image = image.convert('RGB') # Конвертируем изображение в RGB
    ax = fig.add_subplot(2, num_images // 2, i + 1, xticks=[],
        yticks=[]) # Добавляем подграфик
    ax.imshow(image) # Отображаем
    ax.set_title(f'Pred: {class_names[predicted.item()]}, True:
        {folder_name}') # Устанавливаем заголовок
plt.show() # Отображаем фигуру
# Список названий классов для датасета пневмонии
class_names = ['NORMAL', 'PNEUMONIA']
# Пути к папкам
folders = {
    'NORMAL': '/content/test/NORMAL',
    'PNEUMONIA': '/content/test/PNEUMONIA'
}
# Визуализируем изображения из каждой папки
for folder_name, folder_path in folders.items():
    visualize_images_from_folder(
        model, # Модель, которую мы используем для предсказания
        folder_path, # Путь к папке с изображениями
        class_names, # Список названий классов для датасета
        folder_name # Название текущей папки (класс)
    )

```

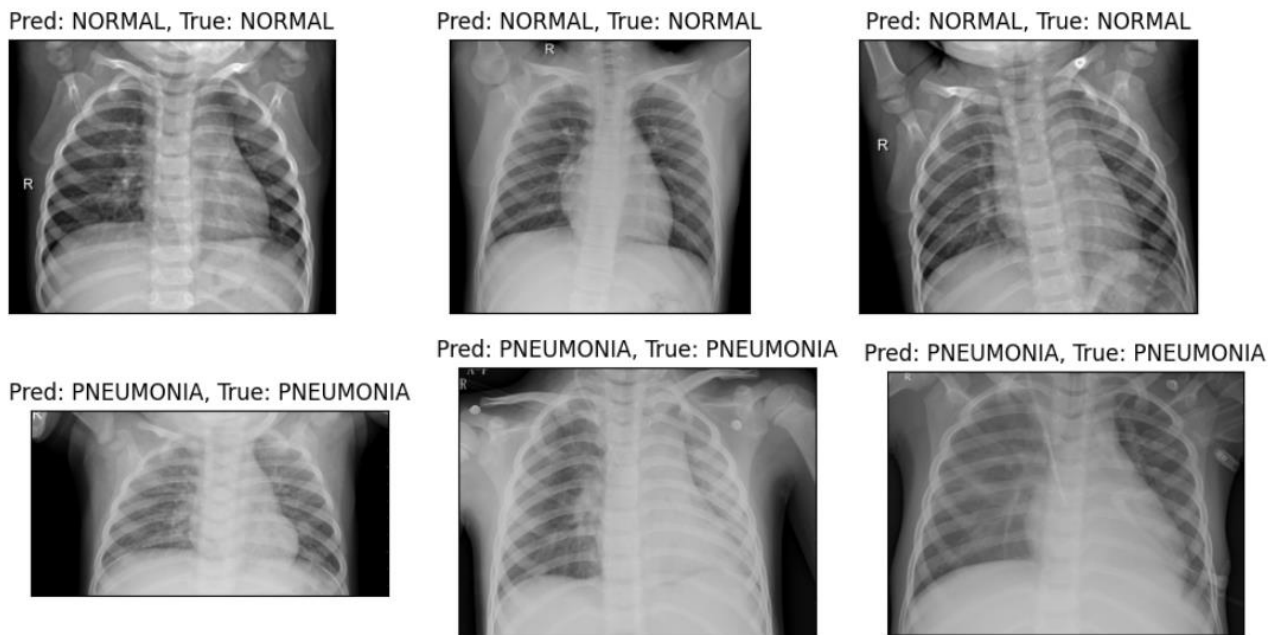


Рис. 3.16. Визуализация классификации рентгеновских снимков

Итак, мы успешно применили предобученную модель ResNet18 для классификации рентгеновских снимков легких. В процессе работы мы выполнили все ключевые этапы: от подготовки данных и их загрузки до настройки модели, ее обучения и оценки производительности.

Мы визуализировали предсказания модели, что позволило наглядно оценить результат. Данный процесс демонстрирует, как можно эффективно использовать предобученные модели в PyTorch для решения задач классификации изображений.

PyTorch для предсказания стоимости квартир

Далее рассмотрим ещё пример - используем нейросеть на PyTorch для предсказания стоимости квартир — при решении задачи регрессии. В нашем примере мы будем использовать нейронную сеть для предсказания стоимости квартиры на основе данных об однокомнатных квартирах стоимостью до 300 млн рублей.

Поля данных здесь следующие:

Цена: Цена квартиры в тысячах рублей (целевая переменная);

Дом: Тип дома (кирпичный, панельный и т.д.);
Балконы: Количество балконов;
Лоджии: Количество лоджий;
Станция метро: Название ближайшей станции метро;
Пешком или на транспорте: Указание на то, добираться ли до метро;
Время в пути: Время в пути до ближайшей станции метро (в минутах);
Площадь: Общая площадь квартиры в квадратных метрах;
Этаж: Этаж, на котором находится квартира;
Всего этажей в доме: Общее количество этажей в здании;
Санузел: Наличие и тип санузла (совмещенный, раздельный и т. д.);
Тип санузла: Тип санузла (например, совмещенный, раздельный);
Примечание: Дополнительная информация о квартире (произвольный текст).

Часть параметров числовые, часть — категориальные. Примечание вообще может быть достаточно длинным текстом из нескольких предложений на естественном языке. Общее количество полей 12.

Сначала нам нужно настроить окружение и установить необходимые библиотеки:

Затем мы импортируем все необходимые библиотеки для работы с PyTorch и загрузки данных:

```
# Устанавливаем библиотеки PyTorch и torchvision
!pip install gdown torch torchvision scikit-learn
# Импортируем необходимые библиотеки
import pandas as pd # для работы с данными в формате DataFrame
import numpy as np # для работы с массивами
import gdown # для загрузки файлов из Google Drive
import warnings # для управления предупреждениями
from sklearn.preprocessing import OneHotEncoder, MinMaxScaler,
    StandardScaler # для предварительной обработки данных

from sklearn.model_selection import train_test_split
import torch # основной пакет PyTorch
import torch.nn as nn # для определения нейросетей в PyTorch
import torch.optim as optim # для оптимизаторов PyTorch

# для преобразования текстовых данных в числовые векторы
from sklearn.feature_extraction.text import CountVectorizer
import matplotlib.pyplot as plt # для построения графиков
import seaborn as sns # для построения сложных графиков
```

```

from IPython.display import clear_output # для очистки вывода
from sklearn.metrics import mean_absolute_error

#На следующем шаге необходимо осуществить загрузку и
#предобработку данных.
#@title Функции загрузки и предобработки данных
# Загрузка и распаковка данных
URL =
'https://storage.yandexcloud.net/terraai/sources/flats.zip'
download_filename = gdown.download(URL, None, quiet=True)
!unzip -q {download_filename} -d '/content/data'
!rm -rf {download_filename}

# Игнорирование предупреждений
warnings.filterwarnings("ignore", category=UserWarning)

#@title Загрузка данных
def load_data():
    df = pd.read_csv('/content/data/flats.csv')
    df.drop(columns=['Источник', 'Бонус агенту', 'Дата', 'Кол-во
дней в экспозиции'], inplace=True)
    df = df[df['Цена, тыс.руб.'] <= 300_000]
    df = df[df['Комнат'] == 1]
    df.drop(columns=['Комнат'], inplace=True)
    return df

# Очистка вывода
clear_output()

#@title Преобразование текста в векторы
def text_to_vector(df, column, vector_size):
    vectorizer = CountVectorizer(max_features=vector_size,
    stop_words='english', token_pattern=r'\b\w+\b')
    text_vectors = vectorizer.
er.fit_transform(df[column].astype('U')).toarray()
    text_vector_df = pd.DataFrame(text_vectors, columns=[f'{col
umn}_vec_{i}' for i in range(vector_size)])
    df = pd.concat([df.reset_index(drop=True), text_vector_df.re
set_index(drop=True)], axis=1)
    df.drop(columns=[column], inplace=True)
    return df

# Функция предобработки данных
def preprocess_data(df):
    # Обработка пропущенных значений
    df = df.dropna()
    # Список столбцов для ONE, исключаем про этажи
    categorical_columns = ['Тип дома', 'Балконы', 'Лоджии',
'Станция метро', 'Пешком или на транспорте', 'Санузел', 'Тип
санузла']

```

```

# Применение One Hot Encoding
df = pd.get_dummies(df, columns=categorical_columns,
drop_first=True, dtype=np.int8)

# Нормирование столбцов Время в пути, Площадь, Этаж и Всего
этажей в доме
scaler_time = StandardScaler()
scaler_area = StandardScaler()
scaler_floor = StandardScaler()
scaler_total_floors = StandardScaler()

df[['Время в пути']] = scaler_time.fit_transform(df[['Время в
пути']])
df[['Площадь']] = scaler_area.fit_transform(df[['Площадь']])
df[['Этаж']] = scaler_floor.fit_transform(df[['Этаж']])
df[['Всего этажей в доме']] = scaler_total_floors.fit_trans
form(df[['Всего этажей в доме']])

# Расчет средней стоимости до нормализации
mean_price = df['Цена, тыс.руб.'].mean()

return df, mean_price

#@title Нормирование целевых значений
def normalize_target(df):
    scaler_price = StandardScaler()
    df[['Цена, тыс.руб.']] = scaler_price.fit_transform(df[['Цена,
тыс.руб.']])
    return df, scaler_price

#@title Разделение данных
def split_data(df, test_size):
    # Разделение данных на обучающую и тестовую выборки
    X = df.drop('Цена, тыс.руб.', axis=1)
    y = df['Цена, тыс.руб.']
    X_train, X_temp, y_train, y_temp = train_test_split(X, y,
test_size=test_size, random_state=42)
    X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp,
test_size=0.5, random_state=42)
    return X_train, X_val, X_test, y_train, y_val, y_test

```

Обратите внимание, что для целей дальнейшего использования в модели мы провели замену текстовых полей числовыми векторами.

```

import pandas as pd # для работы с данными в формате DataFrame
import numpy as np # для работы с массивами
import gdown # для загрузки файлов из Google Drive
import warnings # для управления предупреждениями
# для предварительной обработки данных

```

```

from sklearn.preprocessing import OneHotEncoder, MinMaxScaler,
    StandardScaler
from sklearn.model_selection import train_test_split
import torch # основной пакет для работы с PyTorch
import torch.nn as nn # для определения нейронных в PyTorch
import torch.optim as optim # для оптимизаторов в PyTorch
# для преобразования текстовых данных в числовые векторы
from sklearn.feature_extraction.text import CountVectorizer
import matplotlib.pyplot as plt # для построения графиков
import seaborn as sns # для построения сложных графиков
from IPython.display import clear_output # для очистки вывода

# для вычисления метрики средней абсолютной ошибки
from sklearn.metrics import mean_absolute_error

# Загрузка данных
df = load_data()

# Предобработка категориальных и числовых данных, расчет средней
стоимости
df, mean_price = preprocess_data(df)

# Нормирование целевых значений
df, scaler_price = normalize_target(df)

# Преобразование текста в векторы
df = text_to_vector(df, 'Примечание', vector_size=1000)

# Разделение выборок на обучающую, валидационную и тестовую
X_train, X_val, X_test, y_train, y_val, y_test = split_data(df,
    test_size=0.2)

# Конвертация данных в numpy.ndarray
X_train = X_train.to_numpy()
X_val = X_val.to_numpy()
X_test = X_test.to_numpy()
y_train = y_train.to_numpy()
y_val = y_val.to_numpy()
y_test = y_test.to_numpy()

# Вывод форм выборок
print(f'Форма X_train: {X_train.shape}')
print(f'Форма X_val: {X_val.shape}')
print(f'Форма X_test: {X_test.shape}')
print(f'Форма y_train: {y_train.shape}')
print(f'Форма y_val: {y_val.shape}')
print(f'Форма y_test: {y_test.shape}')

```

На следующем шаге определим улучшенную нейронную сеть, используя `nn.Module`. Наша сеть будет состоять из нескольких полносвязных слоев с большим количеством нейронов, а также слоев Dropout для регуляризации. Мы также будем использовать функцию активации LeakyReLU для лучшей производительности.

```
# Определение модели
class ImprovedRegressionModel(nn.Module):
    def __init__(self, input_dim):
        super(ImprovedRegressionModel, self).__init__()
        # Первый полносвязный слой с 1024 нейронами
        self.layer1 = nn.Linear(input_dim, 1024)
        # Второй полносвязный слой с 512 нейронами
        self.layer2 = nn.Linear(1024, 512)
        # Третий полносвязный слой с 256 нейронами
        self.layer3 = nn.Linear(512, 256)
        # Четвертый полносвязный слой с 128 нейронами
        self.layer4 = nn.Linear(256, 128)
        # Пятый полносвязный слой с 64 нейронами
        self.layer5 = nn.Linear(128, 64)
        # Шестой полносвязный слой с 32 нейронами
        self.layer6 = nn.Linear(64, 32)
        # Выходной полносвязный слой с одним нейроном для регрессии
        self.layer7 = nn.Linear(32, 1)

        # Функция активации LeakyReLU с негативным наклоном 0.1
        self.leaky_relu = nn.LeakyReLU(0.1)
        self.dropout = nn.Dropout(0.3) # Dropout слой для регуляризации

    def forward(self, x):
        # Применяем первый полносвязный слой, LeakyReLU и Dropout
        x = self.leaky_relu(self.layer1(x))
        x = self.dropout(x) # Применяем Dropout
        # Применяем второй полносвязный слой, затем LeakyReLU и Dropout
        x = self.leaky_relu(self.layer2(x))
        x = self.dropout(x) # Применяем Dropout
        x = self.leaky_relu(self.layer3(x))
        x = self.dropout(x) # Применяем Dropout
        x = self.leaky_relu(self.layer4(x))
        x = self.dropout(x) # Применяем Dropout
        x = self.leaky_relu(self.layer5(x))
        x = self.dropout(x) # Применяем Dropout
        x = self.leaky_relu(self.layer6(x))
        x = self.layer7(x) # Применяем выходной полносвязный слой
        return x # Возвращаем выход модели
```

Инициализируем нашу модель, определим функцию потерь и выберем оптимизатор для обучения:


```

# Инициализируем модель
# Создаем экземпляр регрессионной модели, передавая количество
  входных признаков (input_dim)
model = ImprovedRegressionModel(X_train.shape[1])

# Определяем функцию потерь
criterion = nn.MSELoss() # Используем MSE - задача регрессии

# Выбираем оптимизатор
# Используем оптимизатор Adam с начальной скоростью 0.001
optimizer = optim.Adam(model.parameters(), lr=0.001)

```

Обучим нашу нейронную сеть на тренировочном наборе данных. Будем выполнять 200 эпох обучения, в каждой из которых будем проходить все тренировочные данные. Мы также будем использовать метод раннего прекращения (early stopping), чтобы остановить обучение, если качество модели на валидационном наборе данных перестанет улучшаться.

```

def train_model(model, criterion, optimizer, X_train, y_train,
X_val, y_val, epochs, patience=10):
    """
    Обучение модели.

    Параметры:
    model (nn.Module): Нейронная сеть для обучения.
    criterion (nn.Module): Функция потерь.
    optimizer (torch.optim.Optimizer): Оптимизатор.
    X_train (numpy.ndarray): Обучающие данные.
    y_train (numpy.ndarray): Метки обучающих данных.
    X_val (numpy.ndarray): Валидационные данные.
    y_val (numpy.ndarray): Метки валидационных данных.
    epochs (int): Количество эпох для обучения.
    patience (int): Количество эпох для раннего прекращения при
    отсутствии улучшений.

    Возвращает:
    model (nn.Module): Обученная модель.
    train_losses (list): Список значений функции потерь на
    обучающих данных.
    val_losses (list): Список значений функции потерь на
    валидационных данных.
    """
    train_losses = []
    val_losses = []

```

```

best_val_loss = float('inf')
patience_counter = 0 # Инициализация счетчика эпох

for epoch in range(epochs): # Цикл по эпохам
    model.train() # Устанавливаем модель в режим обучения
    optimizer.zero_grad() # Обнуляем градиенты
    outputs = model(torch.from_numpy(X_train).float())
    loss = criterion(outputs,
        torch.from_numpy(y_train).float().view(-1, 1))
    # Обратный проход (backward pass) и вычисление градиентов
    loss.backward()
    optimizer.step() # Обновление параметров модели
    # Сохраняем значение функции потерь на обучающих данных
    train_losses.append(loss.item())

    model.eval() # Устанавливаем модель в режим оценки
    with torch.no_grad(): # Отключаем вычисление градиентов
        # Прямой проход на валидационных данных
        val_outputs = model(torch.from_numpy(X_val).float())
        # Вычисляем значение функции потерь на валидационных данных
        val_loss = criterion(val_outputs,
            torch.from_numpy(y_val).float().view(-1, 1))
        val_losses.append(val_loss.item())

    print(f'Epoch {epoch+1}/{epochs}, Training Loss: {loss.item()},
        Validation Loss: {val_loss.item()}')

    # Проверка условия раннего прекращения обучения
    if val_loss < best_val_loss:
        # Обновляем лучшее значение функции потерь
        best_val_loss = val_loss
        patience_counter = 0 # Сбрасываем счетчик
    else:
        patience_counter += 1 # Увеличиваем счетчик
    # Если количество эпох без улучшений достигло порога, прекращаем
    if patience_counter >= patience:
        print("Раннее прекращение обучения")
        break

    # Возвращаем обученную модель, значения функции потерь на
    # обучающих и валидационных данных
    return model, train_losses, val_losses

# Обучение модели
model, train_losses, val_losses = train_model(
    model, # Нейронная сеть для обучения
    criterion, # Функция потерь
    optimizer, # Оптимизатор
    X_train, # Обучающая выборка
    y_train, # Целевая переменная для обучения
    X_val, # Валидационная выборка
    y_val, # Целевая переменная для валидации

```

```

    epochs=200, # Количество эпох для обучения увеличено до 200
    patience=20 # Используем раннее прекращение - до 20 эпох
)

```

После обучения мы оценим нашу модель на тестовом наборе данных, процесс может занять до 15-20 минут.

```

#@title Функции оценки работы модели
def evaluate_model(model, X_test, y_test, scaler_price,
mean_price):
    """
    Оценка работы модели.

    Параметры:
    model (nn.Module): Обученная нейронная сеть.
    X_test (numpy.ndarray): Тестовые данные.
    y_test (numpy.ndarray): Метки тестовых данных.
    scaler_price (sklearn.preprocessing.StandardScaler): Скейлер,
        использованный для нормализации цен.
    mean_price (float): Средняя цена на всей выборке данных.

    """
    model.eval() # Устанавливаем модель в режим оценки
    with torch.no_grad(): # Отключаем вычисление градиентов
        y_pred = model(torch.from_numpy(X_test).float()).numpy().flat
            ten() # Прямой проход через модель для тестовых данных

    # Обратное преобразование нормализованных тестовых меток
    y_test_actual = scaler_price.inverse_transform(y_test.reshape(-
        1, 1)).flatten()

    # Обратное преобразование предсказанных значений
    y_pred_actual = scaler_price.inverse_transform(y_pred.reshape(-
        1, 1)).flat
        ten()

    # Вычисляем среднюю абсолютную ошибку и преобразуем в млн руб
    mae = mean_absolute_error(y_test_actual, y_pred_actual) / 1000

    # Расчет средней стоимости на всей выборке
    mean_price_mln = mean_price / 1000 # Преобразуем в млн руб

    # Расчет процента ошибки
    percentage_error = (mae / mean_price_mln) * 100

    # Выводим результаты оценки
    print(f'Средняя абсолютная ошибка (MAE): {mae:.2f} млн руб.')
    print(f'Средняя стоимость на всей выборке: {mean_price_mln:.2f}
        млн руб.')
    print(f'Процент ошибки: {percentage_error:.2f}%')

```

```

# Вывод первых нескольких предсказанных и реальных значений
comparison = pd.DataFrame({'Предсказанное': y_pred_actual /
1000, 'Реальное': y_test_actual / 1000}).head(10) #
Преобразуем в млн руб
print("Сравнение предсказанных и реальных значений:")
print(comparison)

# Оценка работы модели
evaluate_model(
    model, # Нейронная сеть для оценки
    X_test, # Тестовые данные
    y_test, # Метки тестовых данных
    scaler_price,
    mean_price # Средняя цена на всей выборке
)

```

Вывод:

Средняя абсолютная ошибка (MAE): 0.97 млн руб.

Средняя стоимость на всей выборке: 8.91 млн руб.

Процентная ошибка: 10.85%

Сравнение предсказанных и реальных значений:

Предсказанное Реальное

0	7.882679	8.50000
1	27.187309	30.71280
2	3.911184	3.80000
3	5.892978	5.85000
4	6.768680	6.40000
5	6.734076	6.28000
6	8.220881	8.30000
7	8.065257	9.03508
8	7.013803	6.70000
9	8.639194	7.70000

Итак, мы успешно применили нейронную сеть для предсказания стоимости квартир на основе данных об однокомнатных квартирах стоимостью до 300 млн рублей. В процессе работы мы выполнили все ключевые этапы: от подготовки данных и их загрузки до настройки модели, ее обучения и оценки производительности.

Мы начали с очистки и предобработки данных, нормализовали числовые признаки и использовали кодирование категориальных переменных. Затем мы определили архитектуру нейронной сети, выбрали функцию потерь и оптимизатор. После этого мы обучили модель на тренировочных данных, используя метод раннего прекращения для предотвращения переобучения.

Наконец, мы оценили производительность модели на тестовых данных и вычислили среднюю абсолютную ошибку (MAE).

Все это показывает, как можно эффективно использовать нейронные сети с PyTorch для решения задач регрессии.

Современные достижения компьютерного зрения

К сожалению, англицизм Computer Vision сейчас вытесняет давно существовавший и более правильный русскоязычный термин «техническое зрение» - ведь мобильные роботы и иные технические устройства, предназначенные для решения самых разных задач и имеющие камеры и другие датчики, создавались уже давно. Увы, часто бывает, что даже значимые достижения прошлого забываются и нечто в науке и технике «переоткрывается» несколько раз. Автору приходилось даже встречать в материалах скороспелых «знатоков ИИ», которые даже претендуют на обучение других, утверждение, что «ИИ появился после 2000 года».

К наиболее распространенным задачам здесь относят, как мы уж знаем, следующие:

- распознавание изображений (классификация, когда на выходе — метка класса, например: «школьный автобус», «фура», «танк»), подклассом здесь идет задача бинарной классификации — удовлетворяет ли картинка заданным признакам (в каске рабочий или нет, есть ли в руках у посетителя оружие, и т. п.).
- сегментация — когда и на выходе изображение, в коем различные интересующие нас области выделены (например, обведены прямоугольной или многоугольной рамкой, или же закрашиваются).

- детектирование объектов — это нечто смешанное между распознаванием и сегментацией, например, нужно не просто определить каски на головах рабочих, но и отметить их на выходном изображении.

В настоящее время доступны (на сайте <http://roboflow.com> и пр.) для бесплатного использования достаточно эффективные модели для распознавания изображенного на картинке, сегментации, и пр. Данные модели предварительно обучены на больших наборах данных и демонстрируют хорошие результаты своей работы. На рисунке 3.17 представлен вариант обнаружения повреждений на изображениях автомобилей.

Ещё более сложной вариацией задачи детектирования объектов является отслеживание перемещения конкретных объектов (задача трекинга), применяемая для анализа последовательности кадров, видео. Классическими приложениями здесь являются трекинг посетителей, например, магазина, трекинг пешеходов и автомобилей на дороге, отслеживание спортсменов, например, перемещение игроков по полю, и даже умная ферма (отслеживание животных). Если мы говорим об автоматизации производства, мы должны учитывать перемещение сотрудников, погрузчиков и других вспомогательных технических средств, перемещение заготовок на конвейере, контроль выполнения технологических операций, и т.п.



Рис. 3.17. Выделение поврежденной области

Перечислим полезные и довольно эффективные предобученные (конечно, возможно и дообучение на своем наборе данных).

- Мы уже упоминали выше модель Yolo, которая весьма эффективна в решении задач распознавания и сегментации изображений, однако нужна лицензия поставщика Ultralytics.
- DINOv2 – находит и выделяет объекты на изображениях, плюсом является возможность по двумерной картинке определять глубину/расстояние, что важно для задач управления движущимися объектами.
- Grounded SAM – понимает и описывает, что изображено на картинке, ищет заданные словами объекты, решает задачу сегментации и накладывает маски, выделяя контур нужной области, подписывая, что это (см. рисунок 3.18); фактически это комбинация двух моделей компьютерного зрения.
- Florence-2 – работает с картинками и текстом совместно, эта мультимодель используется для анализа изображений

городских, пригородных и сельских ландшафтов, обеспечивая продвинутую обработку визуальной информации.

- LLaVA – анализирует видео и картинки, генерирует текстовое описание, что изображено на картинке (см. рисунок 3.19); является аналогом GPT4V, но может исполняться локально на своих видеокартах.

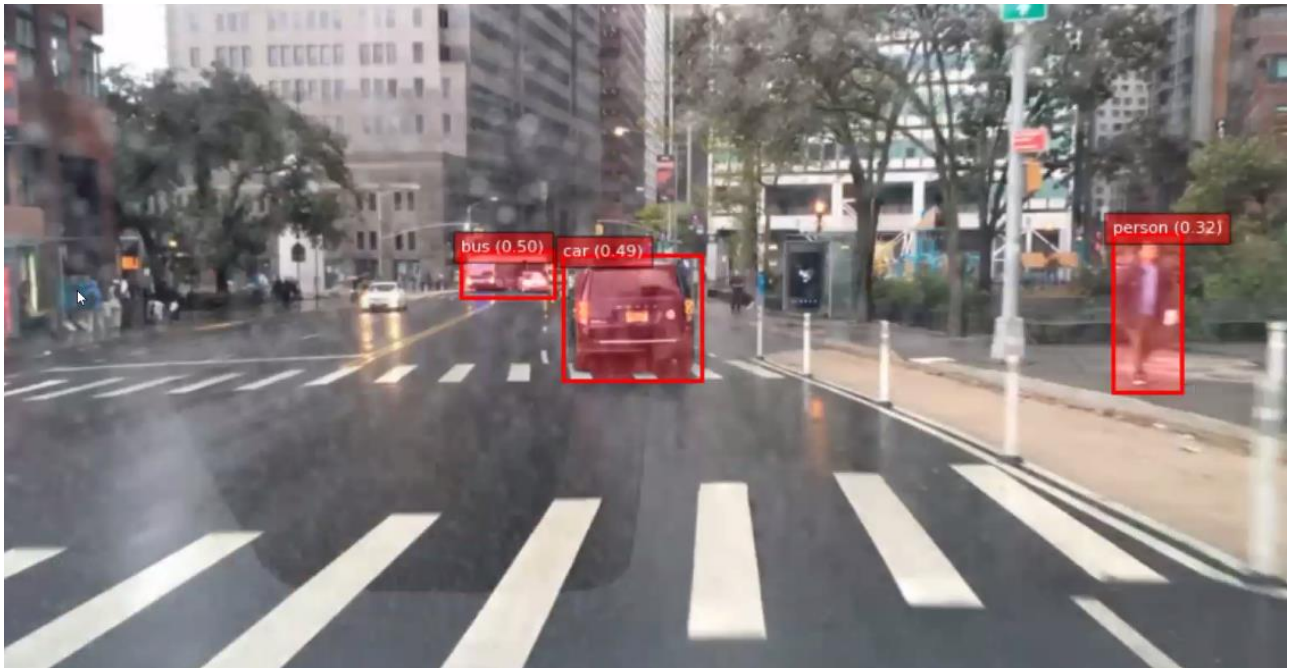


Рис. 3.18. Работа алгоритма определения объектов на улице



Запрос: Для чего нужны выделенные приборы?

Ответ LLaVA: Магистраль показывает кабину с различными приборами и элементами управления. Посвященные устройства, используемые в этой кабине, включают в себя:

1. Индикатор скорости воздушного потока (ASI): Этот прибор показывает скорость самолета относительно скорости воздуха. Обычно находится на левой стороне кабины.
2. Индикатор положения (AI): Этот прибор показывает угол тангажа и крена самолета относительно горизонта. Обычно располагается в центре кабины.

Рис. 3.19. Генерация текстового описания изображения

Нейросети для анализа естественного языка

Область обработки естественного языка (*natural language processing – NLP*) есть отдельная тема интеллектуального анализа данных и глубокого обучения в частности. Важным блоком работы с естественным языком является предварительная обработка текста.

Прежде чем приступать к построению алгоритмов, необходимо:

1. привести текст к единому регистру;
2. в зависимости от задачи провести очистку текста от лишних символов (знаки пунктуации, не очень значимые по смысловой нагрузке слова, *html/xml*-разметка и т. д.);
3. в зависимости от задачи провести *токенизацию* текста, то есть разбить монолитный блок текста на атомарные составные части: абзацы, предложения, слова, *n*-граммы символов;
4. провести разметку слов по частям речи (чтобы различать, например, слово «село» в предложениях «солнце село» и «маленькое село»);

5. привести слова в тексте к нормальной форме: *лемматизация* (приведение слова к словарной форме), *стемминг* (нахождение основы слова);
6. провести процесс преобразования текста в числовое представление – векторизацию.

Все пункты, кроме первого, являются отдельными и достаточно сложными задачами. Часто их реализация зависит от конкретной задачи и требует творческого подхода и долгого изучения конкретной ситуации. Некоторые этапы можно опускать, чтобы получить быстрое первоначальное приближение результата, но, как правило, нужно прорабатывать каждый пункт.

Обработка естественного языка по сути - пересечение сфер искусственного интеллекта и математической лингвистики. В рамках NLP выделяют две большие задачи: анализ и синтез. В рамках анализа мы пытаемся понять смысл текста, а в рамках синтеза генерируем свой текст.

Обработка естественного языка включает:

- распознавание речи (автоматическое преобразование речи в текстовые данные);
- реферирование и аннотирование текста;
- информационный поиск (поисковые системы *Google*, *Yandex*);
- классификация текста по темам (отнесение текста новостей к одной из N тем);
- анализ тональности текста (положительные/отрицательные отзывы);
- выделение именованных сущностей и фактов (извлечение из неструктурированного текста имен, или дат рождения, или марок автомобилей);
- чат-боты (чат-бот Сбербанка для клиентской поддержки, чат-бот Тинькофф для поиска авиабилетов);
- машинный перевод;
- генерация текста;
- синтез речи (голосовые помощники, голосовые автоответчики).

Отсюда вытекает, что мы практически каждый день пользуемся плодами разработок в сфере обработки естественного языка. Данная

сфера является действительно важной, практичной, актуальной и очень бурно развивается. ChatGPT – именно нейросеть для обработки текста — по сути, для предсказания следующего слова в тексте!

Какие же архитектуры глубоких нейронных сетей помогают решать весь этот огромный спектр задач? Давайте кратко познакомимся с ними:

1. сверточные одномерные нейронные сети (*CNN 1D*), работающие по принципу двумерных сверточных сетей, с которыми мы уже знакомы, но анализирующие плоские структуры текста;
2. рекуррентные нейронные сети (*RNN*), основанные на том, что «помнят» не только информацию о текущем примере, на котором обучаются, но и о предыдущих (что очень важно для текстовой информации, где важна сама последовательность слов, а не отдельные слова).
3. Трансформеры;

В свою очередь, как мы уже знаем, рекуррентные нейронные сети стали основой для множества современных архитектур, которые решают сложные реальные задачи: *LSTM (Long short-term memory)* – рекуррентные нейронные сети с долгой краткосрочной памятью; *GRU (Gated Recurrent Units)* – еще одна модификация рекуррентных нейронных сетей на основе механизма вентиляей; *ELMO* – нейронная сеть на основе *LSTM*, но она сочетает две параллельные *LSTM*, одна из которых идет слева направо, а вторая – справа налево. Это значит, что мы знаем контекст для каждого слова не только с начала предложения, но и с конца предложения; *BERT (Bidirectional Encoder Representations from Transformers)* – на данный момент самая передовая архитектура для решения задач обработки естественного языка. Она основана на идеях *ELMO* и *Transformer*, а также на идее обучения с подкреплением. Обучаясь, он решает задачу предсказания части замаскированных слов. Маскирование слов происходит случайным образом.

К сожалению, нейросети не могут изначально обрабатывать текстовую информацию. Для начала, её необходимо преобразовать в массивы (векторы) чисел.

Модели векторных представлений слов и документов можно разделить на три блока:

- частотный подход;
- тематическое моделирование;
- дистрибутивная семантика.

Частотный подход представляет метод *Bag of words* («мешок слов»). В нем выполняются следующие основные шаги:

1. Находим все уникальные слова в тексте.
2. Присваиваем каждому слову порядковый номер от 1 до N (N – число уникальных слов в документе).
3. Меняем в исходном документе все слова на их порядковый номер из шага п. 2.

К примеру, у нас есть два предложения, которые уже очищены от знаков пунктуации и токенизированы на отдельные слова:

```
[[«сегодня», «был», «чудесный», «день»], [«сегодня», «был», «мой», «день», «рождения»]]
```

Тогда можно создать вот такой словарь:

```
{
  «сегодня»: 1,
  «был»: 2,
  «чудесный»: 3,
  «день»: 4,
  «мой»: 5,
  «рождения»: 6
}
```

И весь текст примет вид:

```
[[1, 2, 3, 4],
 [1, 2, 5, 4, 6]]
```

Как мы видим, подход «мешок слов» реализовать довольно просто. Но также мы видим, что получившиеся последовательности имеют разную длину: 4 слова и 5 слов. Но нейронная сеть должна получать на вход последовательности одной длины.

Есть два подхода, решающие эту проблему:

- One-Hot Encoding;
- TF-IDF.

Оба эти подхода представляют текст в виде вектора, длина которого равна длине словаря.

One-Hot Encoding использует разреженный вектор, в котором стоят нули на позициях тех слов из словаря, которых нет в тексте, и единица на местах слов из словаря, которые есть в тексте.

```
[[1, 1, 1, 1, 0, 0],  
 [1, 1, 0, 1, 1, 1]].
```

Возвращаясь к нашему примеру:

Таким образом, достигается одинаковая длина входной последовательности, но теряется вся информация о частоте слова в документе и его расположении относительно других слов.

Проблему частотности решает метод построения векторного представления *TF-IDF* (*term frequency, inverse document frequency*). В данном методе *TF* (частота слова в конкретном документе) умножается на *IDF* – обратную частоту слова во всех документах.

Таким образом мы можем приблизить к нулю значение слов, которые часто встречаются во всех документах одновременно, то есть тех слов, которые никак не выделяют какой-то конкретный документ. Это могут быть различные предлоги. Они могут встречаться почти везде и их *IDF* будет близок к 0, а значит, и в векторе они будут почти нулевыми и не будут вносить различные шумы.

Одна группа используемых методов основана на семантическом моделировании текстов. Грубо говоря, данный подход пытается разделить тексты на разные кластеры, но в отличие от обычной кластеризации, данный подход производит не жесткое разделение на кластеры, а «мягкое». То есть каждое слово не четко относится к какому-то кластеру, а может относиться к нескольким кластерам с разной вероятностью. А еще правильнее будет сказать, что при таком

подходе каждое слово может определять несколько тем (топиков), но на отнесение текста к разным темам влияет различным образом. Например, слово «рецепт» может с большой вероятностью говорить о том, что текст относится к теме «Кулинария», но также может с некой меньшей долей вероятности отнести текст и к теме «Медицина».

К данной группе методов относятся:

➤вероятностный латентно-семантический анализ (PLSA, *англ. Probabilistic latent semantic analysis*). Основан на скрытом семантическом анализе и выявлении скрытых тем в корпусе текстов на основе сингулярного разложения;

➤латентное размещение Дирихле (*англ. Latent Dirichlet Allocation*). Основано на генеративной вероятностной модели, извлекающей скрытые темы из корпуса текстов.

Данные подходы нами разбираться не будут. Но если вы в дальнейшем столкнетесь с задачей тематической классификации текстов, то вспомните об этих алгоритмах. Они дают отличный результат даже без применения сложных алгоритмов. Часто достаточно применить поверх имеющегося векторного представления простейший алгоритм машинного обучения «Деревья решений» и уже можно получить хороший результат.

Подходы на основе *дистрибутивной семантики* делятся на использующие нейросетевой подход и не использующие его.

К нейросетевым можно отнести *word2vec* и *fasttext*. Алгоритм *GloVe* не использует нейросетевой подход. Алгоритмы данного типа решают задачу определения семантической близости слов на основе их распределения в большом корпусе слов.

Семантическое представление слова, или семантический вектор слова (*word embedding*) – это представление слова как точки в многомерном пространстве. Данный тип вектора еще называют *плотным*, так как он имеет очень компактное представление (для решения многих задач хватает вектора длиной 300 элементов).

При правильном построении алгоритма и достаточно большом корпусе слов для обучения семантически близкие слова должны

лежать близко в N -мерном пространстве, в котором представляется вектор слова. К примеру, слова «собака», «пес», «щенок», должны лежать ближе друг к другу, чем к слову «кот». Размерность пространства, в котором будет представлено слово, может быть различной и обычно ее подбирают под задачу.

Подобные представления (*Embedding*) могут строиться в процессе обучения нейронной сети для решения конкретной задачи, для этого применяется специальный слой нейросети *Embedding*.

Но были придуманы достаточно универсальные представления, которые можно применять в дальнейшем для разного типа задач. Данные представления специально обучаются на отдельной задаче – задаче моделирования языка.

В случае алгоритма *word2vec* моделирование языка может происходить двумя разными способами:

- CBOW – нейронная сеть пытается предсказать слово на основе слов, лежащих в его «окрестности», и таким образом обучается слой *Embedding*;

- SKIP-GRAM – нейронная сеть пытается предсказать контекстные слова по текущему слову и таким образом обучается слой *Embedding*

Оказалось, что представления слов, обученные при таком подходе, очень хорошо моделируют язык и помогают в дальнейшем решать различные задачи обработки естественного языка.

Алгоритм *fasttext* обучается в целом таким же образом и имеет две такие же версии, как и *word2vec*, но только обучение происходит не на словах, а на n -граммах символов из слов. Таким образом, частично решается проблема слов, которые отсутствовали при обучении алгоритма. *Word2vec* не сможет выдать представление незнакомого слова, а вот *fasttext* сможет, если слово состоит из знакомых ему n -грамм.

GloVe использует подобную идею совместно встречающихся слов, но обучает сразу на всем корпусе одну огромную матрицу частот слов, встречающихся совместно. По сути, сочетает в себе

особенности сингулярного разложения и методов `word2vec`. На многих задачах `word2vec` показывает себя лучше, чем GloVe.

Рассмотрим пример решения практической задачи обработки ЕЯ на Питоне.

Одномерные сверточные нейронные сети

Мы уже знакомы со сверточными нейронными сетями в разрезе решения задач классификации изображений. Они гораздо лучше полносвязных нейронных сетей анализируют пространственную информацию и не зависят от размера входного изображения, к тому же позволяют получать признаки разного уровня.

Но текст – не матрица, а вектор, и никакой пространственной информации в нем нет. Это верно, но также верно и то, что в тексте очень важна последовательность слов, а она является аналогом пространственного расположения пикселей, но только в одномерном пространстве. Также важно понимать, какие именно слова в тексте находятся рядом, так как их смысл может меняться от контекста. Это также аналог пространственной информации матрицы изображения.

Давайте разберемся. К примеру, у нас есть предложение «Я считаю, что данный товар нельзя назвать хорошим!». Слово «хорошим» без учета контекста отнесет данное предложение к тексту с положительной эмоциональной окраской. Но если бы мы могли учесть слово «нельзя», то поняли бы, что предложение явно с негативной окраской.

Методы сверточных нейронных сетей, перенесенные на «плоские» структуры, помогают нам добиться учета контекста.

Соответственно, необходимо поменять формат входных данных и формат ядра свертки.

В целом сохраняются все правила и принципы двумерной свертки с некоторыми оговорками:

➤ длина последовательности после операции свертки уменьшается на размер ядра, но это изменение происходит в одном измерении, а не в двух;

➤ значения ядра свертки подбираются в процессе обучения нейронной сети, то есть являются весами нейронной сети;

➤ ядро свертки может быть различных размеров, возможно применение и четных размеров ядра. Учитывая, что ядро одномерное, можно применять ядра большего размера.

➤ Одномерная сверточная нейронная сеть также использует слой подвыборки для уменьшения размерности, последовательности и получения представлений более высокого уровня. Работает она точно также, как и в двумерном случае: оставляет элемент с максимальным значением либо среднее по всем значениям, попавшим в «окно».

После свертки у нас был следующий вектор:

Простая одномерная свертка позволяет анализировать текстовые последовательности с учетом контекста. Причем после выполнения нескольких операций свертки и подвыборки получается отследить достаточно далекие зависимости слов.

Одномерные свертки также позволяют анализировать временные ряды, потому что они так же нуждаются в обработке информации с учетом последовательности. Но гораздо эффективней для данной задачи показали себя рекуррентные нейронные сети.

Мы уже поняли, что при обработке текстовой информации важны: порядок следования слов и понимание контекста между словами, расположенными как рядом, так и далеко друг от друга. Сверточная одномерная сеть частично решает эти проблемы. Она позволяет видеть контекст слов, но длина контекста для решения некоторых задач недостаточно большая. Вследствие того, что мы двигаемся «окном» (ядром свертки) по последовательности, мы информацию о последовательности извлекаем частично, но внутри «окна» слова просто суммируются с некоторыми коэффициентами, что не позволяет учитывать там порядок. В целом, это не мешает сверточной нейронной сети отлично работать при решении задачи классификации текстов. И в целом, для данной задачи чаще используются именно одномерные сверточные нейронные сети, так как они дают в среднем такое же качество, но работают намного быстрее и требуют меньше ресурсов. Но если взять, к примеру, задачу автоматической генерации текстов, то тут уже сверточная сеть

явно уступает рекуррентной, так как имеет меньшую глубину контекста.

Пример определения тональности текста

Далее мы рассмотрим практический пример - определение тональности текстов отзывов на фильмы.

Зададим параметры: количество рассматриваемых слов десять тысяч, максимальная анализируемая длина отзыва — 100:

```
num_words = 10000
max_review_len = 100
```

Импортируем библиотеки:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Embedding, GRU, LSTM
from tensorflow.keras import utils
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.callbacks import ModelCheckpoint
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
```

Далее осуществим загрузку исходных данных с сайта YELP:

```
!wgethttps://www.dropbox.com/s/ufbhk3kadtnn6h0/yelp_review_polarity_csv.tgz?dl=1 -O yelp_review_polarity_csv.tgz
!tar -xvf yelp_review_polarity_csv.tgz
```

Читаем данные:

```
train = pd.read_csv('yelp_review_polarity_csv/train.csv', header=None, names=['Class', 'Review'])
```

Выделяем данные для обучения:

```
reviews = train['Review']
```

Выделяем правильные ответы:

```
y_train = train['Class'] - 1
```

На данном этапе нам нужно провести разбиение текста на лексические единицы. Создадим токенизатор Keras и обучим его на имеющихся отзывах:

```
tokenizer = Tokenizer(num_words=num_words)
tokenizer.fit_on_texts(reviews)
```

Преобразуем отзывы Yelp в числовое представление:

```
sequences = tokenizer.texts_to_sequences(reviews)
```

Ограничиваем длину отзывов нашим параметром:

```
x_train = pad_sequences(sequences, maxlen=max_review_len)
```

Далее создаем и компилируем нейронную сеть с ячейками LSTM или GRU:

```
model = Sequential()
model.add(Embedding(num_words, 64, input_length=max_review_len))
# Выберите один вариант - LSTM или GRU
model.add(LSTM(128))
#model.add(GRU(128))
model.add(Dense(1, activation='sigmoid'))

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=[
    'accuracy'])
```

Обучим нейронную сеть:

```
history = model.fit(x_train, y_train, epochs=5, batch_size=128,
    validation_split=0.1)
```

Сеть готова! Загружаем набор данных для тестирования ее работы:

```
test = pd.read_csv('yelp_review_polarity_csv/test.csv',
    header=None, names=['Class', 'Review'])
```

Преобразуем отзывы в числовое представление. При этом обратите внимание, что нужно использовать токенизатор, обученный на наборе данных train:

```
test_sequences = tokenizer.texts_to_sequences(test['Review'])
x_test = pad_sequences(test_sequences, maxlen=max_review_len)
```

Оцениваем качество работы сети на тестовом наборе данных:

```
model.evaluate(x_test, y_test, verbose=1)
```

Результат:

```
1188/1188 [=====] - 5s  
4ms/step - loss: 0.1499 - accuracy: 0.9459
```

AutoML – ИИ совершенствует себя

Одним из кошмаров людей, опасаящихся, что ИИ захватит мир, является, помимо «сверхсильного» ИИ, искусственный разум, совершенствующий сам себя. Действительно, если предположить, что возможности ИИ сравниваются и даже превзойдут человеческие, .

А ведь уже сейчас сделаны первые многообещающие шаги на этом направлении! Родилось направление AutoML – Automated Machine Learning, или автоматизированное машинное обучение.

Как, например, подобрать лучшую нейросеть для решения конкретной прикладной задачи? Одна из них может лучше справляться в одном случае, другая — в другом. Выбор сети, обеспечивающей наилучшую точность (метрику качества решения) — своего рода искусство, причем подбирать придется и параметры (число слоев, функции активации, виды слоев), и гиперпараметры сети.

Мы уже убедились на примере ансамблирования, например, алгоритмов случайного леса и бустинга, что может оказаться, что мощности одного метода машинного обучения недостаточно, и можно для получения лучшего решения использовать их комбинацию. Как правило, специалист по обработке данных тратит большую часть своего времени на предварительную обработку, инженерию признаков, выбор и настройку моделей, а затем оценку результатов. AutoML может автоматизировать эти задачи, предоставляя базовый результат, а также может обеспечить высокую производительность при определенных проблемах и дать понимание того, где можно продолжить исследование

Оказывается, и этому можно научить сам компьютер!

При этом получается, что достаточно квалифицированному человеку-разработчику для качественного решения той или иной задачи с применением машинного обучения, нейросетей часто необходимо один-два дня, а средству AutoML - всего несколько часов. Системы AutoML имеют преимущество: они способны создавать огромное количество тестовых моделей за очень короткий промежуток времени

Модели создаются и обучаются с использованием стекинга. При этом в AutoML *не стараются использовать обязательно глубокие нейронные сети, более простая модель, например случайный лес, может оказаться более подходящей и выигрывать в производительности.*

Среди популярных сегодня инструментов AutoML можно выделить следующие:

- H₂O - полностью открытая распределенная платформа машинного обучения на базе Java, кою можно использовать для автоматизации процесса ML; включает автоматическое обучение и настройку многих моделей в течение заданного пользователем времени;
- Автоматизация создания пайплайнов с помощью библиотеки TPOT (Tree-based Pipeline Optimization Tool) - AutoML пакет, который использует концепции генетического программирования для оптимизации конвейера машинного обучения;
- AutoKeras - открытая - библиотека для автоматизированного поиска архитектуры модели глубокого обучения, совместима с языком Питон и библиотекой Tensorflow; ей указывается, сколько различных архитектур необходимо попробовать, на скольких эпохах каждую из них обучать по исходным данным, и пр.

Существуют и другие средства, например Auto-Sklearn.

Рассмотрим прежде всего использование пакета H₂O. H₂O — это программное обеспечение на основе Java. H₂O поддерживает наиболее широко используемые классические и ансамблевые алгоритмы машинного обучения, включая XGBoost, обобщенные линейные модели, глубокое обучение, и многое другое.

Текущая версия может обучать и выполнять кросс-валидацию для случайного леса, экстремально случайного леса, градиентного бустинга, случайной глубокой нейросети, а затем обучать составной ансамбль, используя все модели (см. рисунок).

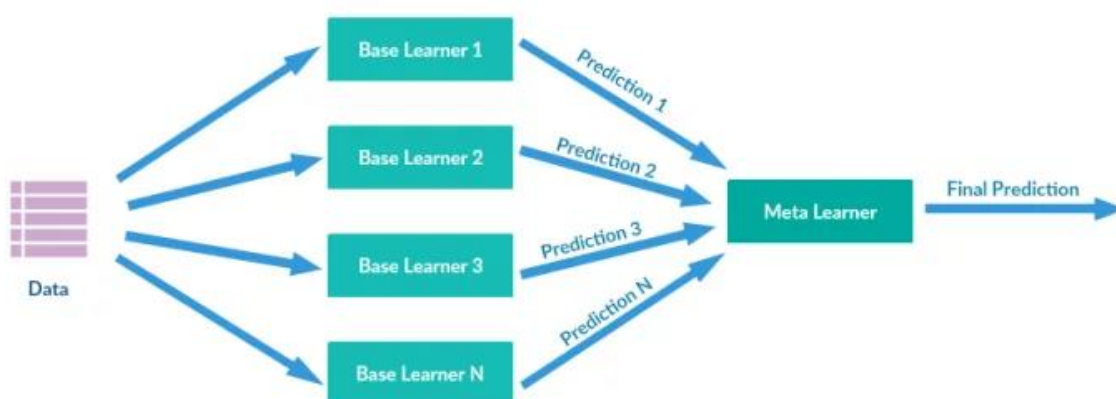


Рис. 3.20. Схема работы H2O

Нетрудно видеть, что используемый подход напоминает ансамблирование моделей машинного обучения, с которым мы познакомились ранее.

Рассмотрим пример, связанный с грибами. Набор данных о них содержит 23 категориальных признака и более 8000 записей. Грибы классифицируются на две категории: съедобные и несъедобные. Классы распределяются достаточно равномерно, с 52% съедобных. В данных отсутствуют пропуски. Это — достаточно популярный и известный набор данных, который мы можем использовать, чтобы увидеть, насколько хорошо AutoML работает по сравнению с традиционными методами.

При использовании языка Питон сначала нужно установить и импортировать модуль H2O и класс H2OAutoML, как и в любой

другой библиотеке, и инициализировать локальный кластер H2O (мы используем Google Colab).

```
import h2o
from h2o.automl import H2OAutoML
h2o.init()
```

Затем нам нужно загрузить данные, это можно сделать прямо в «H2OFrame» или в pandas DataFrame, чтобы мы могли применить *label encoding* и затем преобразовать их в H2OFrame. Как и многие вещи в H2O, H2OFrame работает очень похожим образом на Pandas DataFrame, но с небольшими отличиями и другим синтаксисом.

```
# Load data into H2O
path = "../gdrive/My Drive/Mushrooms/mushrooms.csv"
# df = h2o.import_file(path=path, header =1)

df = pd.read_csv(path)
labelEncoder = preprocessing.LabelEncoder()
for col in df.columns:
    df[col] = labelEncoder.fit_transform(df[col])

df = h2o.H2OFrame(df)
df = df.asfactor()
```

Хотя AutoML будет выполнять даже на начальных этапах большую часть работы за нас, важно, чтобы у нас все еще было хорошее понимание данных, которые мы пытаемся проанализировать, чтобы мы могли умело опираться на итоги работы машинного обучения.

Подобно функциям в sklearn, мы можем создать разделение на обучающую и тестовую выборки, чтобы можно было проверить работу модели в невидимом и предотвратить переобучение.

Важно отметить, что H2O предназначен для работы с [большими данными](#) с использованием метода вероятностного разделения.

Например, при указании разделения 0,70/0,15 H2O будет производить разделение на обучающую и тестовую выборки с вероятностным значением 0,70/0,15, а не точным 0,70/0,15:

```
train, test, valid = df.split_frame ( ratios = [ .7 , .15 ])
```

Затем нам нужно получить имена столбцов, чтобы мы могли передать их в функцию.

В AutoML есть несколько параметров, которые должны быть определены: `x`, `y`, `training_frame` и `validation_frame`, из которых `y` и `training_frame` — обязательные, а остальные - нет. Можно дополнительно настроить значения для `max_runtime_sec` и `max_models`. `max_runtime_sec` является обязательным параметром, а `max_model` — необязательным. Если вы не передаете какой-либо параметр, он принимает по умолчанию значение `NULL`. Параметр `x` является вектором признаков из `training_frame`. Если вы не хотите использовать все признаки из переданного фрейма, вы можете пропустить параметр `x`.

Мы собираемся использовать все параметры в `x` (кроме целевого) и установить значение `max_runtime_sec` в 10 минут (некоторые из этих моделей занимают много времени). Теперь пришло время запустить AutoML:

```
y = "class"
x_train = train.columns
x_train.remove(y)
aml = H2OAutoML(max_runtime_secs=600, seed = 1)
aml.train(x = x_train, y = y, training_frame = train)
```

Здесь была указана функция для запуска 10 минутного периода обучения, но вместо этого можно было указать максимальное количество моделей. Если вы хотите настроить процесс работы AutoML, есть также множество дополнительных параметров, которые вы можете передать для этого:

- `validation_frame`: Этот параметр используется для ранней остановки отдельных моделей в AutoML. Это набор данных, который вы передаете для проверки модели, или он может быть частью обучающих данных, если вы их не пропустили.
- `leaderboard_frame`: Если указано, модели будут оцениваться в соответствии с этими значениями вместо использования показателей кросс-валидации. Опять же, значения являются частью обучающей выборки, если они не пропущены вами.

- `nfold`s: количество блоков в *k-fold* кросс-валидации. По умолчанию 5, может способствовать снижению производительности модели.
- `fold_columns`: Указывает индекс для кросс-валидации.
- `weights_column`: Если вы хотите указать весовые коэффициенты для определенных столбцов, вы можете использовать этот параметр. Назначение веса 0 означает, что вы исключаете столбец.
- `ignored_columns`: Обратно параметру `x`.
- `stopping_metric`: Указывает метрику для ранней остановки поиска. По умолчанию для моделей используется значение `logloss` для классификации и *абсолютное отклонение* для регрессии.
- `sort_metric`: Параметр для сортировки моделей списка лидеров. По умолчанию используется метрика AUC для бинарной классификации, `mean_per_class_error` для многоклассовой классификации и абсолютное отклонение для регрессии.

После запуска моделей можно посмотреть, какие модели работают лучше всего и выбрать лучшие.

```
lb = aml.leaderboard
lb.head()
```

model_id	auc	logloss	mean_per_class_error	rmse	mse
GBM_grid_1_AutoML_20190611_150204_model_3	1	0.00151987		0 0.00978945	9.58333e-05
GBM_3_AutoML_20190611_153332	1	5.07116e-12		0 3.03503e-10	9.2114e-20
GBM_2_AutoML_20190611_153332	1	7.28875e-12		0 4.36864e-10	1.9085e-19
GLM_grid_1_AutoML_20190611_150204_model_1	1	0.00253685		0 0.0164994	0.000272229
DeepLearning_grid_1_AutoML_20190611_153332_model_1	1	0.000501174		0 0.011126	0.000123788
DeepLearning_grid_1_AutoML_20190611_150204_model_1	1	7.86037e-05		0 0.00252565	6.3789e-06
GBM_grid_1_AutoML_20190611_153332_model_1	1	0.0100281		0 0.0117962	0.00013915
DeepLearning_grid_1_AutoML_20190611_153332_model_3	1	0.00480379		0 0.0389828	0.00151966
GBM_1_AutoML_20190611_145434	1	0.00166966		0 0.018618	0.00034663
GBM_1_AutoML_20190611_150204	1	0.00166966		0 0.018618	0.00034663

Чтобы убедиться, что модель не была переобучена, мы теперь запускаем ее на тестовых данных:

```
preds = aml.predict(test)
```

Результат для лучшей модели из AutoML на тестовых данных: значения accuracy и F1-score 1,0 на тестовых данных, что означает, что модель не была переобучена.

Confusion Matrix:

```
[[609    0]
```

```
[    0 602]]
```

Accuracy: 1.00

F1 Score: 1.00

Понятно, что это исключительный случай, обычно мы не можем улучшить accuracy до 100% на нашем тестовом наборе без тестирования большего количества данных.

Следующим шагом будет сохранение обученной модели. Существует два способа сохранить модель лидера — бинарный формат и формат MOJO. Если вы используете свою модель при решении практических задач, то рекомендуется использовать формат MOJO.

Теперь, когда мы нашли лучшую модель для данных, можно провести дальнейшее исследование. Возможно, лучшая модель на тренировочных данных переобучена, и для валидационных данных предпочтительнее окажется другая топ-модель. Возможно, для некоторых моделей данные лучше подготовить или отобрать только наиболее важные признаки. Многие из лучших моделей в H2O AutoML используют ансамблевые методы, и, возможно, модели, используемые в ансамблях, могут быть доработаны.

AutoML может работать с различными задачами, включая бинарную классификацию (как показано здесь), многоклассовую классификацию, а также работать с задачами регрессии.

Библиотека TPOT, созданная поверх Scikit-learn, использует генетическое программирование для оптимизации конвейера машинного обучения.

Например, после подготовки ваших данных вам нужно знать, какие функции вводить в модель для обучения, а затем настраивать её гиперпараметры для получения оптимальных результатов.

ТРОТ автоматизирует все эти шаги для вас и строит оптимальную последовательность действий - «пайплайн» с помощью генетического алгоритма.

Генетические алгоритмы — ещё одна из важных современных методик в ИИ. Они основаны на процессе «естественного отбора» и используются для генерации решений в задачах оптимизации. Шаги алгоритма обычно следующие:

1. Выбор. Имеется совокупность возможных решений некой проблемы, описываемых набором параметров, и функция пригодности. На каждой итерации оценивают, как совместить решение с позиций пригодности.

2. Скрещивание. Затем выбирают наиболее подходящие из решений и выполняется «скрещивание» - часть параметров берётся у одного, часть — у другого.

3. Мутация. Берутся решения и некоторые их параметры изменяются с помощью случайной модификации; процесс повторяется, пока не получится наилучшее решение.

Автоматизированная подготовка пайплайна машинного обучения — это преимущественно задача комбинаторной оптимизации или поиска наилучшего сочетания возможных моделей. Лучшая цепочка действий может быть экспортирована функцией вида `tpot.export(name.py)` в исходный текст на языке Питон, где будут содержаться все действия по подключению нужных библиотек, чтению и предобработке данных, обучению и вызову выбранной модели.

Рассмотрим пример с уже известными нам ирисами Фишера:

```
# еще пример с Ирисами Фишера
pip install tpot
from sklearn.model_selection import train_test_split

from sklearn.datasets import load_iris # данные прямо в SKlearn
iris = load_iris() # загружаем данные

X_train, X_test, y_train, y_test = train_test_split(iris.data,
                                                    iris.target,
                                                    train_size=0.75)
```

```

tpot      =      TPOTClassifier(generations=10,      verbosity=2,
max_time_mins=0.5,  random_state=42)

tpot.fit(X_train, y_train) # обучение модели

print( tpot.score(X_test, y_test) ) # проверяем качество
tpot.export('tpot_iris.py') # это всё!

```

AutoKeras — специализированный фреймворк для построения нейросетей с выбором лучшего решения. Он предоставляет высокоуровневые сквозные API, такие как ImageClassifier или TextClassifier, для решения задач, например, классификации изображений или текстов, в несколько строк, а также гибкие строительные блоки для выполнения поиска архитектуры.

Оптимизация архитектуры модели — а при построении нейросети нам нужно определиться с числом слоев, видами слоев, функциями активации, и пр. - производится путем поиска в пространстве архитектур с применением байесовской модели.

AutoKeras реализует алгоритм поиска, который использует предварительные знания о пространстве поиска. Основная идея здесь заключается в том, чтобы начать поиск с хороших конфигураций — а под *конфигурацией* здесь понимается *полный набор значений гиперпараметров*, - *кой строит и обучает модель* и продолжать исследовать их окрестности. Гиперпараметры иерархически группируются в подмодули в соответствии с их расположением в модели. Подмодулем может быть один гиперпараметр, слой или вся модель. Чтобы чтобы обновленная конфигурация была похожа на текущую наилучшую, при каждом обновлении выбирается только один из подмодулей и все значения его гиперпараметров перебираются.

Ниже приводится пример использования AutoKeras для работы с изображениями.

```

import autokeras as ak
auto_model = ak.ImageClassifier()
auto_model.fit(x_train, y_train)
z = auto_model.predict(x_test)

```

Возможна и более точная подстройка сети в AutoKeras, см. пример ниже задаются желаемые слои в модели:

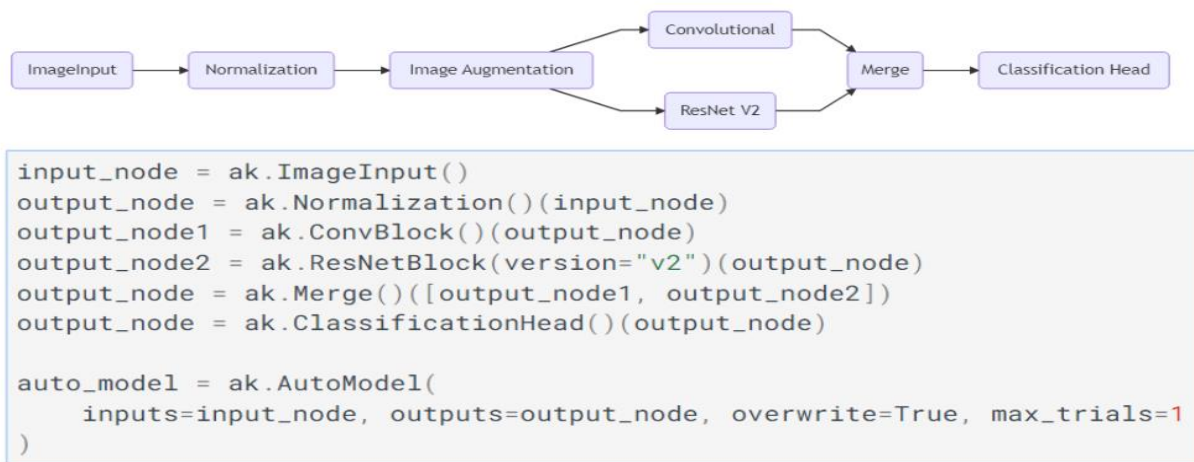


Рис. 3.21. Схема пайплайна

Однако, нужно отметить следующее:

1. Платформа H2O, например, позволяет реализовать отбор признаков и конструирование моделей машинного обучения, однако эти разработки сосредоточены главным образом на повышении скорости генерации готовых моделей, а не на изучении того, как можно улучшить технологию для решения более сложных проблем.

2. Выбор правильной модели машинного обучения и наилучших гиперпараметров для этой модели по своей сути является задачей оптимизации, для решения которой можно использовать генетическое программирование. После использования ТРОТ весь исходный текст программы для предобработки данных, обучения и предсказания занимает буквально полстраницы.

3. Возможности AutoML не безграничны. Например, AutoKeras справляется с простыми типовыми задачами — классификация изображений, текстов, табличных данных, регрессия табличных данных, прогнозирование временного ряда, сегментация изображений с помощью U-net. AutoML не справляется или с трудом справляется со сложными большими задачами — детектированием объектов, распознаванием речи, сложными генеративными нейросетями.

Помимо рассмотренных выше средств, для облегчения программирования приложений с ИИ, существует уже достаточно много доступных фреймворков. Это и Prophet, и MobileNet, и Easy-OCR, и MediaPipe, и Ultralytics, и Fedot, и LightGBM, и др. В

основном они подразумевают применение языка программирования Питон.

Нейрострудники

Весьма «горячей» темой сейчас является возможная замена людей на их рабочих местах искусственным интеллектом. Российская компания «Университет ИИ», занимающаяся как обучением, так и созданием своих решений в области нейросетей и ИИ, ввела в обращение понятие “нейросотрудника“ на базе ChatGPT. Идея заключается в том, что имеющуюся большую языковую модель дообучают на материалах конкретной компании.

Что можно сказать про «сотрудника», являющегося программным обеспечением на основе искусственных нейросетей? Он способен функционировать в режиме 24/7, поскольку ему не нужны ни обеденные перерывы, ни сон. Скорость ответа при обращении, например, клиентов, будет весьма и весьма быстрой.

Обходиться нейросотрудник будет весьма недорого, например, если использовать запросы к ChatGPT через API, одно обращение может стоить в буквальном смысле копейки. А если применять свою большую языковую модель, развернутую на собственных серверах (что особенно актуально в случае, если мы хотим защитить конфиденциальные данные компании) — то и вообще затраты будут лишь на инфраструктуру или даже на её аренду при использовании облачных сервисов.

Какие же конкретные примеры, что может «нейросотрудник»?

Во-первых, это нейромаркетолог. Кто сейчас занимается сочинением рекламных слоганов, текстов, звучащих в рекламе, и т.д. Этот род деятельности, как можно узнать, в частности, из романа В. Пелевина «Поколение П», называется «криэйтор» (не путать с *творцом!*), или «копирайтер». Подразумевается, что они создают нечто оригинальное, идеи, позволяющие лучше продавать те или иные товары.

Как оказалось, «творчества» у людей, занимающихся сейчас подобной деятельностью, в основном не шибко больше, нежели у ИИ. Организация РФ «Университет ИИ» уже получила положительный опыт создания продвигающих текстов на основе дообучения большой языковой модели на материалах конкретной компании, не уступающих подготовленным человеком, а часто — и превосходящих их.

Сюда примыкает «нейроконсультант» — например, на «горячей линии», отвечающий на вопросы о продукции компании, и «нейропродажник».

Автору, зарабатывающему на жизнь преподаванием, нелегко дается написание этого подраздела. Однако, действительность такова, что на базе больших языковых моделей уже вполне успешно создаются и «нейропреподаватели» на базе ИИ. Скажем, преподаватели английского языка. Или — увы! - нейропреподаватели по теме «ИИ и искусственные нейронные сети». От выше рассмотренного нейроконсультанта ведь недалеко до нейрокуратора при изучении учебной дисциплины, ведь читатель уже догадался?

При этом ИИ может (и это и подразумевается!) подстраиваться под конкретного ученика, выстраивать, как это модно сейчас говорить, «индивидуальную образовательную траекторию». Кому-то нужно определенные вещи «разжевать», донести более подробно, на примерах, а кто-то может «схватывать» это же «на лету». Кому-то необходимо продвижение строго шаг за шагом с решением задач по каждой теме, другому, например, нужно просто освежить свои знания в этой области. Все это можно задать ИИ в качестве параметров курса обучения. «Нейрометодолог» подготовит программу курса, включая теорию, практические задачи и контрольные. «Нейроэкзаменатор» проверит выполнение домашних заданий и выставит оценки.

Коснемся подробнее вопроса вытеснения человека из ИТ-профессий, в первую очередь из программирования.

Разработка программного обеспечения (ПО) сегодня превратилось в целую индустрию. Самыми дорогими компаниями по рыночной капитализации сейчас являются Microsoft, Google, и другие, занимающиеся производством ПО. Программ нужно много, и разных. Они применяются в торговле, промышленности, индустрии развлечений, и т.д, и т. п.

Неудивительно, что программист превратился в высокооплачиваемую профессию, и даже дающую определенный социальный статус, уважение (как ранее предприниматель, актер, и пр.).

Однако, все может измениться драматическим образом во вполне обозримом будущем!

Глава Nvidia, фирмы-разработчика видеокарт, харизматичный Дженсен Хуанг, а именно на их продукции сейчас, а не центральных процессорах, обучают и применяют наиболее успешные крупные нейросети, заявил вообще, что детей можно прекращать обучать программированию, поскольку скоро это станет ненужным: *«Наша работа – создавать технологии, где никому не нужно программировать. Языком программирования, по сути, станет человеческий, и каждый в мире будет программистом. В этом и заключается феномен искусственного интеллекта»*, – заявил Хуанг.

По мнению главы американской компании, подрастающему поколению сегодня стоит сосредоточиться на более важных видах деятельности: сельском хозяйстве, биотехнологиях, производстве.

Большие языковые модели, например, ChatGPT, Claude, ГигаЧат способны генерировать по запросу на естественном языке текст программы, которая будет решать поставленную задачу. Становится возможным человеку просто сообщить компьютеру на русском, китайском, английском, что нужно — и получить результат без необходимости изучения языков Питон, С# или SQL. Не шуткой стала уже даже возможность сгенерировать работающий Web-сайт по отсканированному карандашному наброску, как примерно должна выглядеть страница.

Это стало возможным на основе обучения нейросетей на огромных массивах исходных текстов программного обеспечения, доступных в электронном виде.

Всем известен уже, наверное, инструмент Copilot, декларируемый как «помощник программиста», внедренный прямо в популярнейшую среди программистов систему контроля версий и совместной работы GitHub. Есть и другие средства, например, можно воспользоваться расширением для среды разработки Visual Studio Code. Популярным стал инструмент Cursor, демонстрирующий нечто похожее на магию в предсказании намерений человека-программиста, и способный анализировать не просто текст одного отдельно взятого программного модуля, а целый набор связанных исходных текстов программ.

Пока к качеству синтезируемых автоматически текстов программ предъявляется множество претензий, программа вполне может сразу не запуститься, работать не так, как хотелось, и нуждаться в доработке. Однако, некоторые не самые сложные, работают правильно сразу! И это уже сегодня — а ИИ ведь постоянно совершенствуется.

На самом деле, современный скачок в этом направлении укладывается в давно существовавшую тенденцию. Чтобы решить задачу на первых ЭВМ в 1940-1950е годы, необходимо было досконально разбираться в особенностях устройства машины, применять двоичный код команд и адреса ячеек памяти. Затем возникла возможность частичной автоматизации, когда можно было использовать уже мнемоники команд вроде ADD или СТОП, и имена вместо адресов. Подобные системы называли *автокодами*, или ассемблерами. Однако всё равно программа пригодна была лишь для одного конкретного компьютера (или серии ЭВМ с совместимой архитектурой) и не могла быть перенесена на другой. Ассемблеры оставались языками так называемого низкого (приближенного к машине) уровня. Затем появились языки программирования высокого уровня (ЯВУ) — Фортран, Бэйсик, Си и пр., в них стало возможным применять достаточно абстрактные конструкции. Забавно, что

трансляторы (программы, переводящие исходные тексты на ЯВУ, в машинный код для исполнения на процессоре), первоначально, по крайней мере в СССР, называли *программирующими программами*. Ещё более высокий уровень абстракции предоставили *декларативные* языки, например Пролог, SQL, когда стало возможным не приводить подробный пошаговый алгоритм решения, а лишь описать, что нужно получить. Правда, всё равно мы оставались в рамках формального, строго определенного синтаксиса.

Весьма интересным направлением стало *визуальное* программирование, получившее успешное применение, например, в системах управления сложными техническими комплексами (Grafset, ГРАФИТ-ФЛОКС, МЭК61131, и пр.) [40], когда, например, не разбирающийся в тонкостях программирования на С++ или Java инженер или специалист по логике управления буквально рисует схему действий в графическом редакторе.

При этом в технологии разработки ПО довольно давно был выдвинут лозунг «программирование без программистов!» (Мартин, 1982 год). Написанные человеком программы содержат ошибки, ведь, как известно, человеку свойственно ошибаться. А от качества и надежности ПО сейчас зависит очень многое, включая жизни и здоровье людей. Ошибки в так называемом «ПО критической важности» (управление атомными электростанциями, космическими кораблями, медицина, и пр.) чреваты катастрофами, огромным материальным ущербом, и пр.

И если ИИ достигнет качества и уровня генерации программного обеспечения, превосходящего человеческий, профессия «программист» вымрет. Особенно если учесть, что ИИ не нужно спать, ездить отпуск, да и не потребует он повышения зарплаты и не перебежит в конкурирующую фирму.

Вообще, есть определенная ирония в современном витке научно-технического прогресса. Испокон веков, механизация и автоматизация подразумевала избавление человека от тяжелого, монотонного, рутинного труда, подразумевая, что он займется более творческой, интеллектуальной деятельностью.

Сейчас же под непосредственной угрозой со стороны больших языковых моделей находятся именно называемый так некоторыми «креативный класс» - журналисты, копирайтеры, программисты.

Не так давно по рунету разошлась шутка из социальных сетей: **«все думали, что роботы будут чистить снег, а человечество заниматься творчеством. Но теперь роботы занимаются творчеством, а человек чистит снег».**

Свою работу сантехник или электрик, работающие руками, не потеряют ещё долго — на пути внедрения роботов встает очень много проблем, включая носящие объективный характер, например, отсутствие емких аккумуляторов и весьма высокую стоимость по сравнению с использованием труда человека.

А вот профессии, связанные с творческой, умственной деятельностью, находятся под угрозой.

ИИ уже сочиняет сценарии фильмов, пишет книги — и они издаются, и раскупаются! Другая нейросеть может снабдить эти книги иллюстрациями. Живых актёров заменяют образы, сгенерированные нейросетями. И так далее. И подобные технологии совершенствуются на наших глазах, они будут становиться все лучше с каждым годом!

Введение в промпт-инжиниринг и контекст-инжиниринг

На данный момент в сети доступно множество нейросетей самого разного назначения. Генерация изображений, распознавание объектов, синтез видео, суммаризация и стилизация текстов, и т.д., и т. п. При этом многие из них весьма эффективны и демонстрируют впечатляющие успехи! У всех на слуху ChatGPT, DeepSeek, Midjourney, существуют и отечественные продукты (GigaChat, YandexGPT, Кандинский и пр.).

Мы уделили внимание внутреннему устройству нейросетей. Однако, стоит и быть их эффективным пользователем. Попробуем рассматривать ИИ не как конкурента, угрожающего нашим рабочим местам, а помощника. Давайте рассмотрим некоторые рекомендации в зависимости от задачи, кою необходимо решить.

Одна из наиболее впечатляющих сфер применения ИИ, как мы уже упоминали выше, это автоматизированная генерация исходных текстов программ по постановке задачи на естественном языке. Иногда даже можно задействовать картинки, например, эскиз желаемой Web-страницы или экранной формы приложения. Известны такие продукты, как Copilot, дополнительные встраиваемые модули для Visual Studio Code, и многие другие. При этом как минимум помощь ИИ здесь может разгрузить от выполнения рутинных задач, напомнить какие-либо особенности синтаксиса языка программирования или обращения к конкретной библиотеке, быть ассистентом в улучшении (рефакторинге) исходного текста, и пр. Автору книги, например, уже неоднократно удавалось быстро генерировать «скелет» приложения, с коим потом можно дальше работать. Удачным получился и опыт автоматического перевода через большую языковую модель с одного языка программирования на другой — это, кстати, касается предлагаемого в рамках нашего курса в качестве лабораторной работы чатбота, когда для переноса его в Телеграм был получен текст на Питоне на базе исходников на C++.

Однако, наша деятельность не ограничивается программированием. Как задействовать большие языковые модели — GPT, Claude, GigaChat для решения повседневных задач? Для этого нужно освоить искусство составления запросов к ним (промптов). При этом для эффективного использования нейросетей вам не нужно быть ни инженером по машинному обучению, ни специалистом по большим данным. Написать промпт может каждый.

Промпт-инжиниринг, по некоторым громким заявлениям, скоро станет востребованной профессией. Какие же есть советы по эффективному составлению промптов?

Хороший запрос может обладать следующими качествами:

1. Конкретность и однозначность.
2. Указание роли ИИ при создании текста, «от кого» исходит.

3. Выверенная структура, задание границ при генерации ответа, в частности, можно четко задать длину в словах, предложениях, страницах.

4. Правильное задание *контекста*, четкое определение целевой аудитории, например, широкая публика, официальные органы, научные работники, и пр., нередко стоит предоставить дополнительно свой источник информации, коий нужно задействовать при синтезе текста

5. Указание формата ответа, стиля письма, и пр.

При этом хороший промпт должен быть лаконичным, но ёмким. Это требование вытекает не просто из общих соображений, но банально вытекает из ограниченности доступных бесплатных запросов или экономии на платных.

Промпт должен быть понятным как для вас, так и для модели. Как правило, если что-то сбивает с толку вас, то, скорее всего, это также будет сбивать с толку и модель. Постарайтесь не использовать переусложненный язык и не предоставлять излишнюю информацию. Хорошей практикой является использование в промпте глаголов: «Проанализируй, Классифицируй, Противопоставь, Сравни, Создай, Опиши, Оцени, Найди, Сгенерируй, Ранжируй, Рекомендуй, Перепиши, Отсортируй, Обобщи, Переводи, ...».

Часто полезно в запросе указать желаемый объем ответа, например: «напиши кратко, двумя предложениями» или «ответ сгенерируй на две страницы текста», или «не более 500 слов».

Модель иногда - не всегда! - отвечает лучше, если ей указать, в качестве кого («от кого») должен выступить ИИ, генерируя текст (например, «Ты — опытный преподаватель...», «Ты — эффективный маркетолог...», «Ты — поэт-романтик...»).

Чтобы избежать негативного эффекта — так называемых «фантазий» ИИ, часто нужно указать в запросе нечто вроде: «руководствуйся только официальными документами (законами,

нормативными актами организации, в рамках приложенного источника, и пр.), «ничего не добавляй от себя и не придумывай». Пример хорошего запроса: “Напиши пост на тему «как правильно хранить книги дома». Не используй вступление, сразу переходи к делу, пиши просто и понятно, короткими предложениями“.

Текстовые нейросети хорошо понимают контекст, поэтому им можно прямо указывать, про что писать, а про что — нет. Пишите, какие слова, термины, стили можно использовать, а что лучше исключить.

При генерации текста можно — и нужно! задавать его стиль. Например, «напиши в академическом научном стиле» или «составь заметку в стиле официальной информации», «напиши ответ в виде стихов с соблюдением рифмы» (здесь на лето 2025 года непревзойденные результаты показывает модель Claude).

Весьма полезным зачастую является приведение в запросе примера, как должен выглядеть ответ — формат, структура, и пр. Если в промпте нет примера ответа, их относят к zero-shot запросам, один пример — one-shot, для уточнения можно привести и несколько примеров — few-shot. *Не забывайте и о негативных примерах!*

Например:

Преврати текст заказа пиццы клиентом в JSON:

ПРИМЕР:

Я хочу малую пиццу с сыром, томатным соусом и перцем.

JSON:

```
{  
  "size": "малая",  
  "type": "normal",  
  "ingredients": ["сыр", "томатный соус", "перец"]  
}
```

ПРИМЕР:

Можно мне большую пиццу с томатным соусом, базиликом и моцареллой?

JSON:

```
{
```

```
"size": "большая",  
"type": "normal",  
"ingredients": [["томатный соус", "базилик", "моцарелла"]]  
}
```

ПРИМЕР:

Я хочу большую пиццу, одна половина с сыром и моцареллой, а вторая с ветчиной, томатным соусом и ананасом

JSON:

```
...  
...  
  
{  
"size": "большая",  
"type": "half-half",  
"ingredients": [["сыр", "моцарелла"], ["томатный сок",  
"ветчина", "ананас"]]  
}  
...  
.
```

Кстати говоря, большие языковые модели отлично умеют переводить с одного языка на другой! Вполне возможно, мы скоро отойдем от использования переводчиков Google и Яндекс, даже откажемся от DeepL, а будем эффективно применять большие языковые модели и для этих целей. При этом, будучи обученными на огромных корпусах текстов, включая специальные, они владеют различной специальной терминологией, профессиональным жаргоном, и пр. Это сильно облегчает задачу.

Учитывая, что уже появились методы синхронной генерации речи с переводом и использованием указанного голоса, можно предположить скорое вымирание профессии переводчика (как и дубляжа фильмов)?

Несмотря на все усилия, не всегда получается получить от ИИ сразу хороший ответ. В подобной ситуации успешно можно применить механизм *пошагового уточнения*. Современные основанные на ИИ чат-боты удерживают контекст — помнят предыдущую переписку. Этим можно воспользоваться. Например,

написать после получения не совсем устраивающего ответа: «Неплохо, но перепиши ответ. Убери пункты о ..., сделай больший акцент на..., добавь информацию о...». Подобный процесс можно повторять итеративно, добиваясь в итоге приемлемого итога.

Полезным может оказаться и обращение к самой языковой модели с просьбой улучшить, доработать, расширить промпт. В этом модели нередко весьма хороши.

Промпт для современных больших языковых моделей может быть и весьма длинным. Некоторые специалисты способны в одном запросе задать целый «коллектив агентов», взаимодействующих между собой для подготовки лучшего диалога с пользователем.

На самом деле, сейчас «промпт-инжиниринг» эволюционирует в контекст-инжиниринг, подразумевающий не единичный промпт и решение узкой задачи, а формирование целого набора инструкций для ИИ, примеров, документации, которые способны задать команде ИИ-агентов целую экосистему, например, для разработки программной системы.

А некоторые специалисты способны даже в одном промпте задать целый «коллектив агентов», взаимодействующих между собой для подготовки лучшего диалога с пользователем.

Многое сказанное выше, применимо и к промптам, адресуемым системам генерации изображений по описанию — DALL-E, Stable Diffusion, Midjourney, Кандинский, Шедеврум, и пр. Кратко остановимся, как применять и их.

Хороший промпт должен быть кратким. При этом начинать запрос надо с описания главного объекта изображения. Далее можно переходить к фону и иным объектам, которые вы хотите видеть на сгенерированном изображении.

Пропишите оттенок, цвет или тональность. Например, «зеленый», «черно-белый», «неоновый», «в пастельных тонах». Бывает полезно задать стиль рисунка. Можно сослаться на конкретного художника или даже нескольких, либо же указать общий жанр.

В дополнение при генерации фотореалистичных изображений вы можете указать модель камеры, фокусное расстояние и тип съемки. Некоторые примеры, кои можно использовать: Kodak Gold 400, Polaroid, или Nikon D600; объектив: 70 мм, макросъемка, телеобъектив; расстояние до объекта: близко, средняя дистанция, съемка издалека; эффекты: глубина резкости, blur, и пр.

Финальная настройка запроса – указание качества, уровня детализации и размеров. Например: “8K рендеринг“, “с очень четкой детализацией“, “фотографическое качество“, “гиперреалистично“, “5000 на 1000 пикселей”.

Контрольные вопросы и упражнения главы

1. Опишите кратко Ваши впечатления от современных успехов нейросетей.
2. Опишите устройство сверточной нейросети.
3. Ваше мнение о нерешенных вопросах нейросетей.
4. Заменят ли в перспективе нейросети человека? В какой степени? Приведите доводы за и против.
5. Что Вы думаете об ошибках нейросетей?
6. В чем заключается проблема «объяснимого» ИИ (Explainable A.I.)? Что Вы об этом думаете?
7. Опишите кратко современные успехи сетей с обратными связями и области их применения.
8. Ваши впечатления от генерации изображений по описанию?
9. Опишите классические задачи компьютерного зрения
10. Приведите известные Вам свои примеры современных успехов нейросетей.
11. Приведите Ваши собственные варианты применения сетей с подкреплением.
12. В каких играх, Вам кажется, ИИ никогда не превзойдет человека? Почему?
12. Чем занимается «промт-инженер», можно ли это считать перспективной профессией?
14. В каких областях деятельности, Вам кажется, ИИ никогда не превзойдет человека? Почему?
15. В примере нейросети для анализа тональности отзывов используйте ячейки GRU вместо LSTM, сравните скорость обучения и качество работы обученной сети.
16. Меняйте гиперпараметры нейросети, чтобы повысить качество работы: длину вектора представления слов в слое Embedding; количество нейронов на рекуррентном слое (LSTM или GRU); число рекуррентных слоев; оптимизатор - `adam`, `rmsprop`; число эпох обучения; размер мини-выборки.

11. ФИЛОСОФСКИЕ, ЭТИЧЕСКИЕ И СОЦИАЛЬНЫЕ ПРОБЛЕМЫ НЕЙРОСЕТЕЙ

Циолковский когда-то писал, что освоение космоса даст человеку «горы хлеба и бездну могущества». Можно ли сказать, что искусственный интеллект по потенциалу сопоставим с космосом?

Мало того, что прикладные системы ИИ давно и успешно вошли в практику самых разных организаций в различных сферах деятельности, сейчас мы переживаем период «революции ИИ», вполне серьезно говорится о создании «сильного ИИ» к 2030 году.

Система AlphaFold демонстрирует использование ИИ в науке, уже сейчас получая многообещающие результаты в сложнейшей проблеме расшифровки структуры белков, что сулит грандиозные достижения в биологии и медицине. Вообще, «ИИ-ученый» потенциально сможет совершать открытия лучше человека — ведь ему не нужно ни спать, ни есть, и ему доступно будет больше информации, чем может прочесть человек в книгах и статьях за всю жизнь, - и тем самым невиданно продвинуть науку и технический прогресс.

Американская компания Profluent представила нейросетевой инструмент OpenCRISPR, который предназначен для генерирования полностью искусственных систем редактирования генома CRISPR-Cas9. А редактирование генома человека — путь к победе над множеством болезней, продлению жизни, и т.п. Перечисленное — безусловные плюсы ИИ для человечества. Но есть и другая сторона медали.

Нерешенные вопросы ИИС

В инженерной деятельности всегда нужно стараться взглянуть на тот или иной метод или алгоритм с разных позиций, выявить его слабые и сильные стороны, область предпочтительного применения.

Мы рассмотрели выше, бесспорно, впечатляющие современные успехи искусственных нейросетей.

А какие у них существуют проблемы? В первую очередь надо назвать проблему «Explainable A.I.» – проблему «понятного», или «объяснимого» ИИ. Дело в том, что, как правило, непонятно, как именно нейронная сеть приходит к решению. Метод решения «рассредоточен» в весах синапсов обученной сети. Кто может поручиться за качество проведенного обучения? Известен эффект переобучения, о коем говорилось выше.

Далее. А вдруг обучающая выборка окажется недостаточной при переходе к промышленной эксплуатации системы? Разберем пример

из области транспорта, где весьма популярно сейчас практическое применение нейросетей – управление беспилотным автомобилем.

В случае наезда на пешехода кто будет отвечать? Тот, кто построил весь автомобиль? Кто писал программу, моделирующую работу нейросети до обучения? Кто выбирал архитектуру сети и выбрал, например, неподходящую? Тот, кто проводил обучение?

Где искать «неверное правило», приведшее к катастрофе? Если бы у нас была экспертная система с продукционной базой знаний вида «ЕСЛИ – ТО», причину установить можно было бы с точностью до конкретного правила – и поправить его. В случае нейросетей это не так.

Уже сейчас достаточно громкий резонанс получили некоторые частные случаи ошибок нейросетей. Например, в сервисе распознавания изображений Google групповой портрет негров получил подпись «гориллы». Что, само собой, не понравилось многим узнавшим об этом инциденте, а в США с их «политкорректностью» разгорелся большой скандал [40].

Возможно, путь к решению проблемы объяснимого ИИ лежит через интеграцию логического, или символического подхода к искусственному интеллекту, где сильной стороной является способность к рассуждениям, с искусственными нейронными сетями.

Страшен ли ИИ для человека – сейчас и в будущем? Является ли этот вопрос философским? [3, 9, 18]. Действительно, взаимоотношения человека и мира, место человека в мире – один из основополагающих вопросов философии. Может ли создание «сильного» ИИ бросить вызов человеку как «царю природы», единственному существу, способному осознавать себя, мыслить и творить? Ответ на этот вопрос, видимо, положительный.

Многие публичные личности, например, выдающийся ученый Хокинг [42], предприниматель в сфере инноваций Маск, другие известные люди поднимали вопрос о потенциальной опасности ИИ для человечества [43]. Однако сама принципиальная возможность создания «сильного» и «сверхсильного» ИИ на данный момент научно не доказана [7, 9].

В то же время есть и более «утилитарные» и насущные вопросы, актуальность которых не вызовет сомнений уже в скором будущем.

Интеллектуальные системы, несомненно, угрожают сфере занятости. Есть мнение, что целый ряд профессий в связи с их применением исчезнет, как, например, практически исчезли профессии машинистки или извозчика гужевого транспорта. Высказываются мнения, что под реальной угрозой находится, например, профессия бухгалтера. Сбербанк России уже начал эксперимент по замене корпоративных юристов специальным интеллектуальным программным обеспечением. Аналогичные работы идут в сфере внедрения ИИ – операторов колл-центров, которые способны уже на достаточно «умные» беседы в некоторой ограниченной предметной области, причем качество синтеза речи уже практически не позволяет отличить речь машины от человеческой!

Уже сейчас существуют и могут быть поставлены «под ключ» почти полностью автоматизированные склады с роботами-погрузчиками, автоматически перемещающимися между стеллажами тележками и центральным «умным» диспетчерским пунктом, управляющим десятками исполняющих необходимые операции устройств.

Многие считают, что профессия водителя автомобильного транспорта исчезнет уже в обозримом будущем. Неудивительно, учитывая впечатляющий прогресс беспилотных легковых и грузовых автомашин. А еще раньше автоматическое вождение найдет массовое применение на морском и железнодорожном транспорте.

Американская компания Goldman Sachs заменила трейдеров, занимавшихся торговлей акциями по поручению крупных клиентов банка, на автоматически работающие программы на базе ИИ. Сейчас из 600 человек, работавших в 2000 году в этой должности, осталось два – остальных заменили торговые роботы, к обслуживанию которых привлечено 200 инженеров.

На электронной площадке Amazon арбитражем взаимных претензий покупателей и продавцов товаров занимаются программы-роботы. При этом они обрабатывают свыше 60 млн претензий в год,

что в несколько раз больше числа всех поданных исков через традиционную судебную систему США.

Некоторые считают, что с повсеместным внедрением дистанционного обучения под угрозу попадает профессия преподавателя.

ИИ внедряют в сферу найма на работу (HR-менеджмент), например, автоматической обработки анкет кандидатов. Однако здесь столкнулись с дискриминацией со стороны машинного разума по расовому и половому признаку! Данный пример демонстрирует еще один важнейший – этический – аспект проблематики искусственного интеллекта.

Но проблема не только в вытеснении человека с его рабочих мест. В конце концов, с этим мы сталкиваемся на протяжении веков, начиная с механизации и автоматизации ручного труда. На автомобильных заводах давно сварку корпусов, как и многие иные операции, производят не устающие, точные и быстрые роботы.

Еще более острые вопросы этического характера встают в медицине. Система IBM Watson была «принята на работу» (пока, правда, результат эксперимента скорее негативный) в качестве врача в медицинский центр [44]. В Индии в одном из лечебных учреждений обнаружили высокий риск неправильных диагнозов. Причина – Watson «учился» на данных американских пациентов. А, как мы понимаем, образ жизни и условия в США и Индии сильно различны, как и другие особенности. Еще хуже оказался опыт применения системы в Медицинском центре университета Гашон Гил в Южной Корее.

Что, если лечение, назначенное ЭВМ, окажется неверным и здоровью пациента будет нанесен ущерб? В случае ошибки человека-врача возможны лишение его лицензии и ответственность вплоть до уголовной.

А беспилотный транспорт? В случае наезда на пешехода автомобиля без водителя кто виноват и на кого возлагать ответственность? На разработчика всего авто, на создателя программного обеспечения ИИ или на непосредственного владельца машины? Увы, но данная проблема уже не просто абстрактная, были ДТП с человеческими жертвами по вине ИИ. Представим еще одну

проблемную с точки зрения морали ситуацию для автопилота. В случае если на дороге неожиданно выскакивают дети, чью жизнь должен спасти ИИ – детей ценой выезда на встречную полосу и неминуемого столкновения с фурой или пассажиров, сидящих в салоне? В США уже несколько лет проходят международные форумы по этическим проблемам ИИ.

Все перечисленное требует, в том числе, государственного регулирования и внесения необходимых изменений и дополнений в законодательство. Можно ли обжаловать решение ИИ в суде?

Вернемся, однако, к потенциальным угрозам человеку со стороны «сильного», или «универсального», искусственного интеллекта. Это надо сделать, учитывая, насколько быстро он развивается, во многих случаях превосходя первоначальные ожидания [45].

Возможные проблемы развития ИИ, красочно представленные в многочисленных фантастических литературных произведениях и фильмах, начиная с Франкенштейна, связаны с обращением ИИ против человека, «порабощением» и «угнетением» людей вообще со стороны машин. Широко известны сюжеты «Космической Одиссеи» Артура Кларка, «Терминатора» и «Матрицы».

Если действительно ИИ представляет угрозу, возможно, стоит запретить данные работы? Например, заключить международную конвенцию? Илон Маск, создатель электрокаров «Тесла» и многоразовых космических носителей, уверен [43]: «ИИ – тот редкий случай, когда нам необходимо быть активными в вопросах регулирования». По его словам, в противном случае, если человек будет лишь реагировать на изменения, уже произошедшие в отрасли, «будет уже слишком поздно». Маск подчеркнул, что искусственный интеллект может оказаться для человека опаснее, чем дорожно-транспортные происшествия, авиакатастрофы, наркотики или плохая пища.

Вряд ли можно считать подобные меры действительно эффективными. Международные конвенции, подписанные ведущими странами, существуют давно, например, в сфере ограничения разработки химического и биологического оружия. Но остановлены ли полностью эти работы в действительности? Не может ли найтись некий безумный миллионер или диктатор в какой-либо стране, который будет

продолжать разработки, невзирая на договоренности? Риторический вопрос.

До тех пор, пока владение некой технологией будет способно принести серьезное преимущество в войне одной из сторон, весьма сложно вести речь о действенных мерах ограничения подобных работ.

Почти все сходятся во мнении, что искусственный интеллект может быть применен в военной сфере. Прикладные разработки в области интеллектуальных систем военного назначения ведутся давно и активно. Страна-обладатель «сильного ИИ», не исключено, получит весомое преимущество на поле боя.

В свете сказанного, если не произойдет глобального катаклизма, мировой войны или подобного по масштабу события, видимо, развитие технологий ИИ остановить невозможно, и запретить создание «сильного» не получится даже при желании.

Несколько слов о киборгах

Многие считают, что опасных последствий создания «сильного» ИИ можно избежать путем сближения, интеграции человека и машины, «слияния» естественного и искусственного разума. Это можно назвать формой «киборгизации».

Издавна человечеству известны очки, костыли, контактные линзы. Достаточно давно широко применяются, например, кохлеарные импланты, позволяющие вернуть слух тысячам людей, и кардиостимуляторы, диктующие ритм биения сердца пациентам с соответствующими проблемами. Современные протезы конечностей являются весьма гибкими и функциональными. Можно ли считать человека с искусственной рукой или электронной схемой внутри киборгом?

В настоящее время ведутся активные и частично успешные работы в области создания нейроинтерфейсов – способов подключения к ЭВМ сигналов, исходящих непосредственно из нервной системы человека. При этом отпадает необходимость задействовать пальцы руки или язык для ввода информации в ЭВМ. Парализованным людям исследователи с помощью роботизированных рук и экзоскелетов возвращают возможность

движения. Еще в 2014 году на чемпионате мира по футболу в Бразилии первый удар по мячу совершил парализованный человек в экзоскелете [46]. При этом дело не ограничивается антропоморфными конечностями. Пациентка в США с имплантированными в мозг электродами смогла управлять «силой мысли» истребителем в компьютерном симуляторе. Сигнал, считанный с мозга, может передаваться на значительные расстояния, как в знаменитых экспериментах Мигеля Николелиса [47, 48].

Весьма широкую известность получила активность Илона Маска в этом направлении с интерфейсом Neuralink.

Все это весьма впечатляет. Кстати, возможно и побуждение человеку, например, за счет магнитного импульса или ультразвука, выполнить то или иное действие, например, нажать на кнопку.

Российская компания «НейроЧат» [49] предоставляет возможность за счет некоего аналога «шлема», считывающего, подобно электроэнцефалографу, электрическую активность головного мозга, парализованным людям общаться с близкими людьми. Взаимодействие происходит путем набора сообщений буква за буквой. Буквы при этом быстро «вспыхивают» на экране.

Говоря о будущем развитии систем ИИ, их известный популяризатор и разработчик Сергей Марков пишет [50]: «Науке давно известен эффект нейропластичности: мозг очень хорошо адаптируется к поступающим в него сигналам. Первые эксперименты, продемонстрировавшие нейропластичность, провёл ещё в XIX веке французский врач и физиолог Мари-Жан-Пьер Флуранс. Флуранс брал петуха, перерезал ему нервы, ведущие к мышцам – сгибателям и разгибателям крыла, и сшивал их крест-накрест. Сигнал, которым птица пыталась согнуть крыло, теперь попадал в мышцу-разгибатель, и наоборот. Первое время петух не мог летать, но позже мозг приспособился к изменившейся ситуации, и птица снова выучилась полёту... Множество случаев травм головного мозга показывали, что даже с очень серьёзными функциональными повреждениями нейронной сети человек в состоянии сохранить свою личность, активность, воспоминания и т.д., хотя и с некоторыми провалами. Приведём в пример аппараты искусственного зрения. Сигнал попадает не совсем туда, куда он

попадает от настоящего глаза. Требуется время, чтобы мозг приспособился к восприятию этой картинки.

Если мы подключим к мозгу вторичную искусственную нейронную сеть, можно ожидать, что за счёт нейропластичности наше сознание постепенно освоит новое пространство, распространится на него, и на втором этапе мы получим некое новое сознание: модификацию нашего сознания, существующую на комбинированном субстрате. Первая часть субстрата – тот биологический мозг, который у нас был вначале, а вторая часть – искусственная нейронная сеть. Затем, например, биологическая часть отрезается, отмирает... Допустим, мы так рассчитали размеры новой нейронной сети и её структуру, что на её фоне изначальное «обиталище» нашего сознания стало сравнительно малой и несущественной частью этого большого мозга. Точно так же, как изъятие небольших частей нервной ткани из мозга человека зачастую не приводит к фатальным утратам для его сознания». Всем известный Илон Маск тоже хочет объединить мозг человека и ИИ, чтобы развитие ИИ шло не как нечто враждебное человеку. Получается, ИИ – не угроза, а, напротив, путь человека к бессмертию?

Контрольные вопросы главы

1. Каково Ваше мнение об опасности ИИ для человека?
2. Как Вам кажется, возможен ли «сильный» ИИ? Если нет – почему? Если да – когда, по Вашему мнению, он появится (5–15 лет, 20–100 лет, 300–500 лет)?
3. Ваше мнение о вытеснении СИИ людей с их рабочих мест.
4. Кто должен отвечать за ошибки ИИ?
5. Необходима ли «красная кнопка» для вмешательства человека в случае опасности?
6. Что Вы думаете о проблеме социальной и расовой дискриминации людей, проявляемой системами ИИ?
7. Сформулируйте Ваше мнение о военных системах ИИ.
8. В каких областях деятельности, Вам кажется, ИИ никогда не превзойдет человека – или их нет? Почему?
9. Что Вы думаете о киборгизации? Станут ли все люди киборгами и когда?
10. Есть ли проблема в доступе различных социальных слоев к технологиям киборгизации, по Вашему мнению?
11. Станет ли киборгизация путем к бессмертию?

ЗАКЛЮЧЕНИЕ

Поистине впечатляюще выглядит сфера потенциального использования нейросетей. Конечно, в рамках одного учебного пособия невозможно охватить все применяемые подходы и способы решения вариативных, сложно поддающихся формализации задач – предметной области искусственного интеллекта.

Нами рассмотрены применение и современные успехи искусственных нейронных сетей, включая сверточные и рекуррентные.

Приводятся практические примеры на языке Питон на базе библиотек SciKitLearn, Pandas, Keras и др.

Затронуты важные вопросы этического, социального и философского характера, относящихся как к существующим системам «слабого» ИИ, так и к потенциальным, связанным с перспективой появления «сильного» ИИ.

БИБЛИОГРАФИЧЕСКИЙ СПИСОК

1. Указ Президента Российской Федерации от 10.10.2019 № 490 «О развитии искусственного интеллекта в Российской Федерации» [Электронный ресурс] // Официальный Интернет-портал правовой информации. URL: <http://publication.pravo.gov.ru/Document/View/0001201910110003?rangeSize=10/> (дата обращения: 29.11.2019) .
2. Дружинин В.Б., Конторов Д.С. Проблемы системологии (проблемы теории сложных систем) / С предисловием акад. В.М. Глушкова. – М.: Сов. радио, 1976. – 296 с.
3. Марков С. Охота на электроовец. Большая книга искусственного интеллекта. — Москва, 2024. — 568 с. (1 том)
4. Марков С. Охота на электроовец. Большая книга искусственного интеллекта. — Москва, 2024. — 718 с. (2 том)
5. Шанкин Г.П. Ценность информации. Вопросы теории и приложений. – М.: Филоматис, 2004. – 128 с.
6. Тюгашев А.А. Компьютерные средства искусственного интеллекта: учеб. пособие. – Самара: Изд-во СамГТУ, 2020. – 162 с.
7. Лорьер Ж.Л. Системы искусственного интеллекта. – М.: Мир, 1991. – 568 с.
8. Хофштадтер Д. Гедель, Эшер Бах: эта бесконечная гирлянда. – Самара: Изд-во «Бахрах-М», 2001. – 752 с.
9. Искусственные нейронные сети и приложения: учеб. пособие / Ф.М. Гафаров, А.Ф. Галимянов. – Казань: Изд-во Казан. ун-та, 2018. – 121 с.
10. Поспелов Д.А. Из истории искусственного интеллекта: история искусственного интеллекта до середины 80-х годов // Новости искусственного интеллекта. – 1994. – №4. – С.70–90.
11. Мак-Каллок У.С., Питтс В. Логическое исчисление идей, относящихся к нервной активности // В сб.: «Автоматы» под ред. К.Э. Шеннона и Дж. Маккарти. – М.: Изд-во иностр. лит., 1956. – С.363–384.
12. Минский М., Пейперт С. Персептроны = Perceptrons. – М.: Мир, 1971.
13. Башмаков А.И., Башмаков И.А. Интеллектуальные информационные технологии: учеб. пособие. – М.: Изд-во МГТУ им. Н.Э. Баумана, 2005. – 304 с.
14. Постолит А.В. Основы искусственного интеллекта в примерах на Python – Спб.: БХВ-Петербург, 2024 — 448 с.,ил.
15. Лахман К. Самое главное о нейронных сетях. Лекция в Яндексе [Электронный ресурс] // Хабрахабр <https://habr.com/ru/company/yandex/blog/307260/> (дата обращения: 08.08.2020).

16. Беспилотники уже вовсю ездят по Москве [Электронный ресурс] // Russia beyond. URL: <https://ru.rbth.com/read/470-yandex-driverless-cars> (дата обращения 11.08.2020).
17. Тьюринг А. Может ли машина мыслить? – М.: Физматлит, 1960. – 67 с.
18. Searle, J. Minds, brains, and programs // Behavioral and brain sciences. – 1980. – Vol. 3. – № 3 (September). – Pp. 417–424.
19. [Электронный ресурс] // Russia beyond. URL: [Электронный ресурс] // Хабрахабр. Мария Жарова URL: <https://ru.rbth.com/read/470-yandex-driverless-cars> (дата обращения 31.07.2024)..
20. Современный компьютер: Сб. науч.-попул. статей: пер. с англ. / Под ред. В.М. Курочкина. – М.: Мир, 1986. – 212 с.: ил.
21. Сергей Марков: AI и естественный язык [Электронный ресурс] // Интернет-портал YouTube. URL: <https://www.youtube.com/watch?v=JscFBSFqG3k> (дата обращения: 29.11.2019)
22. Девятков В.В. Системы искусственного интеллекта: учеб. пособие для вузов. – М.: Изд-во МГТУ им. Н.Э. Баумана, 2001. – 352 с.
23. Системы искусственного интеллекта. Практический курс: учеб. пособие / В.А. Чулюков [и др.]. – М.: БИНОМ. Лаборатория знаний, 2008. – 292 с.
24. Кибер-Пушкин: стихи из машины. [Электронный ресурс] // Спецпроект Сергея Тетерина: машина пишет и читает стихи. URL: <http://www.teterin.ru/pushkin/> (дата обращения: 29.11.2019).
25. Пекелис В.Д. Маленькая энциклопедия о большой кибернетике. – М.: Изд-во «Дет. литература», 1973. – 415 с.
26. Поспелов Д.А. Ситуационное управление: теория и практика. – М.: Наука. Гл. ред. физ.-мат. лит., 1986.
27. Корнилов Е.Н. Программирование шахмат и других логических игр. – СПб.: БХВ-Петербург, 2005.
28. Гаврилова Г.А., Хорошевский В.Ф. Базы знаний интеллектуальных систем. – СПб.: Питер, 2001 – 384 с.
29. Методы представления знаний: метод. указ. / Сост. И.Л. Коробова. – Тамбов: Изд-во Тамб. гос. техн. ун-та, 2003. – 24 с.
30. Николенко С., Кадурин А., Архангельская Е. Глубокое обучение. – СПб.: Питер, 2018. – 480 с.: ил.
31. ИИ и машинное обучение [Электронный ресурс] // Центр Архэ видео. URL: <https://www.youtube.com/watch?v=z5mVKhjOxQc>
32. Галушкин А.И. Синтез многослойных систем распознавания образов. – М.: Энергия, 1974.
33. Rumelhart D.E., Hinton G.E., Williams R.J., Learning Internal Representations by Error Propagation. In: Parallel Distributed Processing, vol. 1, pp. 318–362. Cambridge, MA, MIT Press., 1986.

34. Барцев С.И., Охонин В.А. Адаптивные сети обработки информации. – Красноярск: Ин-т физики СО АН СССР, 1986. Препринт N 59Б. – 20 с.
35. Y. LeCun, B. Boser, J.S. Denker, D. Henderson, R.E. Howard, W. Hubbard and L.D. Jackel: Backpropagation Applied to Handwritten Zip Code Recognition, Neural Computation, 1(4). – 1989. – Pp. 541–551.
36. Круглов В.В., Борисов В.В. Искусственные нейронные сети. Теория и практика. – М.: Горячая линия – Телеком, 2002. – 382 с.
37. Нейронные сети ассоциативной памяти. Сети Хопфилда [Электронный ресурс] // Лекции. орг. URL: <https://lektsii.org/5-41962.html> (дата обращения 09.08.2020).
38. Andrej Karpathy. [Hackers guide to Neural Networks](#) // [Electronic resource] // Andrej Karpathy blog. URL: <http://karpathy.github.io/neuralnets/> (дата обращения: 09.08.2020).
39. Google ‘fixed’ its racist algorithm by removing gorillas from its image-labeling [Electronic resource] // The Verge. URL: <https://www.theverge.com/2018/1/12/16882408/google-racist-gorillas-photo-recognition-algorithm-ai> (дата обращения 09.08.2020).
40. Тюгашев А.А. Графические языки программирования и их применение в системах управления реального времени. -Самара: Изд-во Самарского научного центра РАН, 2009 — 99 с., ил.
41. Стивен Хокинг. Искусственный интеллект сулит опасность человечеству [Электронный ресурс] // Портал открытых систем OSP.ru. URL: <https://www.osp.ru/news/articles/2014/47/13044144/> (дата обращения: 29.11.2019).
42. Человечество в опасности: Илон Маск призвал регулировать искусственный интеллект [Электронный ресурс] // Журнал «Форбс», 17.07.2017. URL: <https://www.forbes.ru/tehnologii/347945-chelovechestvo-v-opasnosti-ilon-mask-prizval-regulirovat-iskusstvennyy-intellekt> (дата обращения: 29.11.2019).
43. Как IBM Watson не оправдал ожиданий в области применения ИИ в медицине [Электронный ресурс] // Журнал IEEE Spectrum. URL: <https://spectrum.ieee.org/biomedical/diagnostics/how-ibm-watson-overpromised-and-underdelivered-on-ai-health-care> (дата обращения: 29.11.2019)
44. Китайский ИИ получил лицензию доктора [Электронный ресурс] // Портал новостей робототехники RoboNews. URL: <https://robonews.su/21563-Kitaiyskiiy-II-poluchil-licenziyu-doktora.html> (дата обращения: 29.11.2019).
45. Человек в экзоскелете открыл Чемпионат мира по футболу [Электронный ресурс] // Научно-популярный портал «Занимательная робототехника». URL: <http://edurobots.ru/2014/06/chelovek-v-ekzoskete-otkryl-chempionat-mira-po-futbolu-v-brazilii/> (дата обращения: 29.11.2019).

46. Обезьяна управляет роботом [Электронный ресурс] // Газета Нью-Йорк Таймс, 15 января 2008 г. URL: <https://www.nytimes.com/2008/01/15/science/15robo.html?pagewanted=allD7c> (дата обращения: 29.11.2019).
47. Мигель Николелис: Настало время межмозговой коммуникации [Электронный ресурс] // Лекции. URL: <https://ideanomics.ru/lectures/13445> (дата обращения 09.08.2020).
48. Компания «Нейрочат» [Электронный ресурс] // Главная. URL: <http://neurochat.pro/> (дата обращения 09.08.2020).
49. Номо ex machina: перспективы перемещения сознания на другой носитель [Электронный ресурс] // Интернет-портал «Хабрахабр». URL: <https://habr.com/ru/company/mailru/blog/397671/> (дата обращения: 29.11.2019).
50. Пекелис В.Д. Маленькая энциклопедия о большой кибернетике. – М.: Изд-во «Дет. литература», 1973. – 415 с.
51. Поспелов Д.А. Ситуационное управление: теория и практика. – М.: Наука. Гл. ред. физ.-мат. лит., 1986.

Оглавление

ВВЕДЕНИЕ 1

1. ОСНОВЫ МАШИННОГО ОБУЧЕНИЯ 3

История машинного обучения	6
Классические задачи машинного обучения	8
Язык программирования Питон в ИИ	13

2. ВВЕДЕНИЕ В ИСКУССТВЕННЫЕ НЕЙРОННЫЕ СЕТИ 23

История искусственных нейронных сетей	23
Основные классы задач, решаемые ИНС	25
Биологический нейрон	27
Структура формального нейрона	28
Архитектуры нейронных сетей	32
Теоремы Колмогорова – Арнольда и Хехт-Нильсена	35
Обучение нейронной сети	37
<i>Обучение нейросети с учителем</i>	<i>38</i>
<i>Обучение без учителя</i>	<i>42</i>
Пример создания нейросети на языке Питон	44
Контрольные вопросы и упражнения главы	54

3. ПРИМЕНЕНИЕ СОВРЕМЕННЫХ НЕЙРОСЕТЕЙ 55

Сверточные сети	55
Рекуррентные нейронные сети	65
Архитектура «Трансформер»	71
Успехи обучения с подкреплением	74
Тенденции развития больших языковых моделей	83
Использование библиотек языка Питон для создания ИНС	96
<i>Применение библиотеки Keras</i>	<i>96</i>
<i>Использование библиотеки PyTorch</i>	<i>107</i>
<i>Распознавание медицинских изображений</i>	<i>113</i>
<i>PyTorch для предсказания стоимости квартир</i>	<i>119</i>
Современные достижения компьютерного зрения	129
Нейросети для анализа естественного языка	132
Одномерные сверточные нейронные сети	139
Пример определения тональности текста	141
AutoML – ИИ совершенствует себя	143
Нейрострудники	152
Введение в промпт-инжиниринг	158
Контрольные вопросы и упражнения главы	163

11. ФИЛОСОФСКИЕ, ЭТИЧЕСКИЕ И СОЦИАЛЬНЫЕ ПРОБЛЕМЫ ИИ 164

Нерешенные вопросы ИНС	164
Несколько слов о киборгах	169
Контрольные вопросы главы	171

ЗАКЛЮЧЕНИЕ 172

БИБЛИОГРАФИЧЕСКИЙ СПИСОК 17